Software Requirements Specification

For

UniMarket

A Student Marketplace for Buying, Selling, Renting, and Exchanging Items

Prepared by

Ayoraf Hossain
ID: ASH2225001M

Jannatul Fardous Shorna

ID: JSH2325035F

Saifa Sikder Shauly

ID: BKH2325013F

Institute of Information Technology

Noakhali Science and Technology University

Date: 15-06-2026

Table of Contents

1. Introduction 1

1.1 Problem Statement 1

1.2 Purpose 1

1.3 Project Scope 2

2. Stakeholders 3

2.1 Primary Stakeholders 3

2.1.1 Buyers / Student Customers 3

2.1.2 Sellers / Student Product Owners 5

2.1.3 Renters / Exchange Requesters 7

2.2 Secondary Stakeholders 8

2.2.1 Admin / System Moderator 8

2.2.2 Payment Service Providers 10

2.2.3 Firebase and Cloud Service Providers 11

2.3 Development Stakeholders 13

2.3.1 Developers 13

2.3.2 Testers 13

3. Design and Implementation Constraints 14

3.1 Frontend Technologies 14

3.1.1 Flutter 14

3.1.2 Dart 15

3.2 Backend Technologies 15

3.2.1 Firebase Authentication 15

3.2.3 Cloud Firestore 15

3.3 Cloud Services 15

3.3.1 Firebase Cloud Messaging 15

3.3.2 Cloudinary Image Hosting 15

3.4 State Management 16

3.5 External Packages and APIs 16
3.6 Payment Technology 16
3.6.1 Manual Payment System 16
3.6.2 Online Payment Gateway System 16
3.7 Security Measures 17
3.7.1 HTTPS 17
3.7.2 Authentication and Authorization 17
3.7.3 Firestore Security Rules 17
4. Requirement Specification 17
4.1 Functional Requirements 17
4.1.1 User Registration and Login 17
4.1.2 User Profile Management 18
4.1.3 Add Product Listing 18
4.1.4 Edit Product Listing 18
4.1.5 Delete Product Listing 19
4.1.6 Product Search and Filter 19
4.1.7 Product Details View 19
4.1.8 Wishlist Management 20
4.1.9 Cart and Checkout System 20
4.1.10 Manual Payment System 20
4.1.11 Online Payment Gateway System 21
4.1.12 Rent System 21
4.1.13 Book Exchange System 22
4.1.14 Real-Time Chat System 22
4.1.15 Rating and Review System 22
4.1.16 Stock Management 23
4.1.17 Seller Dashboard 23
4.1.18 Notification System 23
4.1.19 Price Drop Notification 24

4.1.20 Admin Panel 24
4.1.21 AI Smart Chatbox 24
4.2 Data Requirements 25
4.2.1 User Information 25
4.2.2 Product Data 25
4.2.3 Cart and Order Data 25
4.2.4 Payment Data 25
4.2.5 Wishlist Data 26
4.2.6 Chat Data 26
4.2.7 Rent Request Data 26
4.2.8 Exchange Request Data 26
4.2.9 Review and Rating Data 27
4.2.10 Notification Data 27
4.3 Performance Requirements 27
4.3.1 Speed and Latency Requirements 27
4.3.2 Real-Time Synchronization Requirements 28
4.3.3 Capacity Requirements 28
4.4 Business Requirements 29
4.5 Business Rules 30
4.6 Dependability Requirements 33
4.6.1 Reliability and Availability Requirements 33
4.6.2 Robustness / Fault-Tolerance Requirements 33
4.6.3 Safety-Critical Requirements 34
4.7 Maintainability and Supportability Requirements 34
4.7.1 Maintainability Requirements 34
4.7.2 Supportability Requirements 35
4.7.3 Adaptability Requirements 35
4.8 Security Requirements 35
4.8.1 Access Requirements 36

4.8.2 Integrity Requirements 36
4.8.3 Privacy Requirements 36
4.9 Usability and Human-Interaction Requirements 37
4.9.1 Ease of Use Requirements 37
4.9.2 Personalization and Internationalization Requirements 37
4.9.3 Understandability and Politeness Requirements 37
4.9.4 Accessibility Requirements 37
4.9.5 User Documentation Requirements 38
4.9.6 Training Requirements 38
4.10 Look and Feel Requirements 38
4.10.1 Appearance Requirements 38
4.10.2 Style Requirements 38
4.11 Operational and Environmental Requirements 39
4.11.1 Expected Physical Requirements 39
4.11.2 Interface Requirements 39
4.11.3 Release Requirements 39
5. Requirement Engineering Process 39
5.1 Requirement Elicitation Techniques 40
5.1.1 Interviews 40
5.1.2 Questionnaires 41
5.1.3 Focus Group Discussions 42
5.1.4 User Interface Analysis 43
5.2 Sample of Requirement Collection 43
5.2.1 Requirement Collection – Buyers / Student Customers 43
5.2.2 Requirement Collection – Sellers / Student Product Owners 44
5.2.3 Requirement Collection – Renters and Exchange Requesters 45
5.3 Conflicting Requirements and Negotiation 45
5.4 Requirement Validation 46
5.4.1 Review the Requirements 47

5.4.2 Test the Requirements 47
5.4.3 Simulate the Requirements 48
5.5 Change Management 48
6. Use Case Diagram 50
7.Use Case Descriptions 51
7.1 UC-01: Manage Profile 51
7.2 UC-02: Search Product 52
7.3 UC-03: Filter Product 53
7.4 UC-04: View Product Details 54
7.5 UC-05: Add to Wishlist 55
7.6 UC-06: Add to Cart 57
7.7 UC-07: Checkout Product 58
7.8 UC-08: Make Payment 59
7.9 UC-09: Send Message 60
7.10 UC-10: Request Rent 61
7.11 UC-11: Request Exchange 62
7.12 UC-12: Give Rating and Review 63
7.13 UC-13: View Notifications 64
7.14 UC-14: Post Product 65
7.15 UC-15: Edit Product 67
7.16 UC-16: Delete Product 68
7.17 UC-17: Manage Seller Dashboard 69
7.18 UC-18: View Orders / Sold Items 70
7.19 UC-19: Accept / Reject Rent Request 71
7.20 UC-20: Accept / Reject Exchange Request 72
8. Requirement Traceability Matrix (RTM) – UniMarket 74
8.1 Selected Core Use Cases 74
8.2 RTM Table 74
9. Requirement Prioritization 75

9.1 Requirement Prioritization Table 75
10. Activity Diagram 78
10.1 Activity Diagram of Manage Profile 78
10.2 Activity Diagram of Search and Filter Product 79
10.3 Activity Diagram of View Product Details 80
10.4 Activity Diagram of Post Product 81
10.5 Activity Diagram of Add to Cart and Checkout 82
10.6 Activity Diagram of Make Payment 83
10.7 Activity Diagram of Send Message 84
10.8 Activity Diagram of Request Rent 85
10.9 Activity Diagram of Request Exchange 86
10.10 Activity Diagram of Manage Seller Dashboard 87
11. Sequence Diagram 88
11.1 Sequence Diagram of Manage Profile 88
11.2 Sequence Diagram of Search and Filter Product 89
11.3 Sequence Diagram of View Product Details 90
11.4 Sequence Diagram of Post Product 91
11.5 Sequence Diagram of Add to Cart and Checkout 92
11.6 Sequence Diagram of Make Payment 93
11.7 Sequence Diagram of Send Message 94
11.8 Sequence Diagram of Request Rent 95
11.9 Sequence Diagram of Request Exchange 96
11.10 Sequence Diagram of Manage Seller Dashboard 97
12. Swimlane Diagram 98
12.1 Swimlane Diagram of Manage Profile 98
12.2 Swimlane Diagram of Search and Filter Product 99
12.3 Swimlane Diagram of View Product Details 100
12.4 Swimlane Diagram of Post Product 101
12.5 Swimlane Diagram of Add to Cart and Checkout 102

12.6 Swimlane Diagram of Make Payment 103
12.7 Swimlane Diagram of Send Message 104
12.8 Swimlane Diagram of Request Rent 105
12.9 Swimlane Diagram of Request Exchange 106
12.10 Swimlane Diagram of Manage Seller Dashboard 107
13. Database Design 108
13.1 Entity Relationship Diagram 108
13.2 Relational Schema 109
14.Class Diagram 110
15. Conclusion 111
16. References 111
17.Appendix 111

List of Figures

Figure 1: Use Case Diagram of UniMarket .....**Error! Bookmark not defined.**
Figure 2: Activity Diagram of Manage Profile 78
Figure 3:Activity Diagram of Search and Filter Product 79
Figure 4: Activity Diagram of View Product Details 80
Figure 5:Activity Diagram of Post Product 81
Figure 6:Activity Diagram of Add to Cart and Checkout 82
Figure 7:Activity Diagram of Make Payment 83
Figure 8: Activity Diagram of Send Message 84
Figure 9:Activity Diagram of Request Rent 85
Figure 10:Activity Diagram of Request Exchange 86
Figure 11:Activity Diagram of Manage Seller Dashboard 87
Figure 12: Sequence Diagram of Manage Profile 88
Figure 13:Sequence Diagram of Search and Filter Product 89
Figure 14: Sequence Diagram of View Product Details 90
Figure 15:Sequence Diagram of Post Product 91
Figure 16:Sequence Diagram of Add to Cart and Checkout 92

Figure 17:Sequence Diagram of Make Payment 93
Figure 18:Sequence Diagram of Send Message 94
Figure 19:Sequence Diagram of Request Rent 95
Figure 20: Sequence Diagram of Request Exchange 96
Figure 21:Sequence Diagram of Manage Seller Dashboard 97
Figure 22: Swimlane Diagram of Manage Profile 98
Figure 23: Swimlane Diagram of Search and Filter Product 99
Figure 24:Swimlane Diagram of View Product Details 100
Figure 25:Swimlane Diagram of Post Product 101
Figure 26:Swimlane Diagram of Add to Cart and Checkout 102
Figure 27: Swimlane Diagram of Make Payment 103
Figure 28:Swimlane Diagram of Send Message 104
Figure 29:Swimlane Diagram of Request Rent 105
Figure 30:Swimlane Diagram of Request Exchange 106
Figure 31:Swimlane Diagram of Manage Seller Dashboard 107
Figure 32:Entity Relationship Diagram of UniMarket 108
Figure 33:Relational Schema of UniMarket 109
Figure 34:Class Diagram of UniMarket 110

1. Introduction

1.1 Problem Statement

In university life, students often need to buy, sell, rent, or exchange different items such as books, electronics, calculators, lab equipment, bicycles, hostel accessories, and study materials. However, most students currently depend on Facebook groups, Messenger chats, notice boards, or personal contacts to find buyers and sellers. These methods are often unorganized, time-consuming, and unreliable. Students may face several problems such as fake product posts, difficulty finding trusted sellers, lack of proper search and filter options, no dedicated rent or exchange system, and poor communication between buyers and sellers. In addition, there is no structured platform where university students can manage their own listings, Wishlist products, chat with sellers, receive notifications, and track marketplace activities in one place. There is a need for a mobile-based student marketplace platform that allows university students to buy, sell, rent, and exchange items securely and conveniently. UniMarket aims to provide a trusted campus-focused marketplace with product listing, search and filter, real-time chat, Wishlist, rent request, exchange request, seller dashboard, rating and review, and notification features. This system will make student-to-student trading easier, faster, more organized, and more reliable.

1.2 Purpose

The purpose of "UniMarket: A University Student Marketplace" is to help university students buy, sell, rent, and exchange products within a trusted student community. The system is designed to reduce the difficulties of informal buying and selling by providing a structured and secure marketplace platform. UniMarket allows students to create accounts, manage profiles, post products, upload product images, search and filter products, add items to Wishlist, communicate through real-time chat, and manage products from a seller dashboard. The system also supports rent-based listings where students can set per-day rent prices and buyers can select rental duration. In addition, the book exchange system allows students to post exchange items without price and coordinate through chat. The system also includes cart and checkout, manual payment verification, rating and review, stock management, notification system, and price drop alerts. Through these

features, UniMarket provides a complete campus marketplace experience that is simple, reliable, and suitable for university students.

1.3 Project Scope

UniMarket is a mobile application focused on university student marketplace activities. The system is designed to be simple, secure, and useful for students who want to buy, sell, rent, or exchange items inside their campus community.

The scope of UniMarket includes:

• Student registration and login
• University email-based authentication
• User profile management with name, department, contact information, and profile picture
• Product posting with image, price, description, category, and condition
• Buy and sell system for student marketplace items
• Product search by name and filter by price and category
• Product details page with image gallery, description, seller information, chat, and add-to-cart option
• Cart and checkout system with total price calculation
• Rent system with per-day price, rental duration, and automatic total calculation
• Book exchange system with no price field and chat-based coordination
• Wishlist system for saving favorite products
• Price drop notification for Wishlist items
• Real-time chat system between buyer and seller
• Rating and review system after successful transaction
• Stock management with quantity tracking
• Seller dashboard to add, edit, delete, and view own listings
• Admin panel for user management, product control, report handling, and analytics
• AI smart chatbox with quick reply suggestions

2. Stakeholders

2.1 Primary Stakeholders

2.1.1 Buyers / Student Customers

Description:

Buyers are university students who use UniMarket to search, view, rent, exchange, or purchase products from other students. They mainly interact with product listings, wishlist, cart, chat, checkout, rent request, exchange request, and review features.

Access level:

- Search and filter products
- View product details
- Add products to wishlist
- Add products to cart
- Send messages to sellers
- Request product rental
- Send exchange requests
- Submit manual payment transaction ID
- Give rating and review after transaction
- Receive notifications

Needs:

- Easy product searching and filtering
- Clear product information with image, price, condition, and seller details
- Safe communication with sellers
- Wishlist support for saving favorite products
- Simple cart and checkout system
- Transparent rent and exchange request tracking

Responsibilities:

- Provide correct profile and contact information
- Communicate respectfully with sellers
- Submit correct payment transaction ID
- Use wishlist, cart, rent, and exchange features responsibly
- Give honest rating and review after transaction

Key interactions/screens:

- Home screen
- Product details page
- Search and filter screen
- Wishlist screen
- Cart and checkout screen
- Chat room
- Rent request screen
- Exchange request dialog
- Notification screen
- Profile screen

Data created/consumed:

- **Created:** wishlist items, cart items, chat messages, rent requests, exchange requests, reviews, payment transaction ID
- **Consumed:** product listings, seller information, product details, notifications, order status, rent request status, exchange request status
- **Privacy:** buyer profile data, chat messages, wishlist, payment reference, and transaction information must be protected securely

Constraints:

- Buyer must be logged in before using marketplace actions
- Internet connection is required for real-time updates
- Buyer depends on seller accuracy and response time

- Manual payment verification may take time

2.1.2 Sellers / Student Product Owners

Description:

Sellers are university students who use UniMarket to post products for sale, rent, or exchange. They manage their own listings, communicate with buyers, respond to rent and exchange requests, and update product availability.

Access level:

- Create product listings
- Upload product images
- Set product category, condition, price, and stock
- Set listing type: Sell Only, Rent Only, Sell & Rent, or Exchange Only
- Set daily rent price for rent items
- Add wanted item and exchange notes for exchange listings
- Edit own product listings
- Delete own product listings
- View own active listings
- Accept or reject rent requests
- Accept or reject exchange requests
- Chat with interested buyers

Needs:

- Simple product posting process
- Easy listing management
- Secure image upload support
- Clear seller dashboard
- Notification for new messages, rent requests, exchange requests, and order updates
- Product analytics such as total products, active listings, sold items, and earnings

Responsibilities:

- Post accurate and genuine product details
- Upload valid product images
- Keep product price, stock, and availability updated
- Respond to buyer messages and requests responsibly
- Remove unavailable, sold, or expired products
- Verify payment information carefully

Key interactions/screens:

- Add product screen
- Edit product screen
- Seller dashboard
- My listings section
- Product details page
- Chat room
- Rent requests section
- Exchange requests section
- Notification screen
- Profile screen

Data created/consumed:

- **Created**: product listings, product images, rent price, exchange notes, seller messages, listing updates, request responses
- **Consumed:** buyer messages, rent requests, exchange requests, order status, wishlist interest, reviews, payment information
- **Privacy:** seller personal information, chat data, and transaction records must be protected securely

Constraints:

- Seller must be authenticated before posting products
- Seller can only edit or delete own listings
- Incorrect product information may create disputes

- Seller response delay may affect buyer experience

2.1.3 Renters / Exchange Requesters

Description:

Renters and exchange requesters are students who use UniMarket specifically to rent items for a selected duration or request item-to-item exchange. They may also be buyers, but their interaction is different because rent and exchange do not always follow the normal purchase flow.

Access level:

- View rentable and exchangeable products
- Select rental start and end date
- Send rent request
- Send exchange request
- Add introductory exchange message
- View rent request status
- View exchange request status
- Chat with product owner

Needs:

- Clear rent price per day
- Automatic rental duration calculation
- Automatic total rent calculation
- Easy exchange request process
- Clear wanted item information
- Chat-based coordination
- Request status tracking

Responsibilities:

- Select valid rental dates
- Provide clear exchange message

- Communicate honestly with product owner
- Return rented products according to agreement
- Avoid fake rent or exchange requests

Key interactions/screens:

- Product details page
- Rent request dialog
- Exchange request dialog
- Chat room
- Notification screen
- Profile / My Rentals screen

Data created/consumed:

- **Created:** rent requests, exchange requests, chat messages
- **Consumed:** rent price, rental duration, wanted item, exchange notes, request status
- **Privacy:** request information and chat communication must be protected

Constraints:

- Rent request depends on product availability
- Exchange depends on seller acceptance
- Rental agreement may require manual coordination between students

2.2 Secondary Stakeholders

2.2.1 Admin / System Moderator

Description:

Admin or system moderator is responsible for managing UniMarket, monitoring user activity, handling reports, controlling inappropriate listings, and maintaining a safe marketplace environment.

Access level:

- View registered users
- Ban or delete users if necessary
- Approve, reject, or remove product listings
- View reported products or users
- Manage product categories
- Monitor marketplace analytics
- Review suspicious activities

Needs:

- Clear admin dashboard
- User management tools
- Product control tools
- Report handling system
- Analytics dashboard
- Secure admin access

Responsibilities:

- Maintain marketplace safety
- Remove fake or inappropriate products
- Handle user reports
- Take action against rule violations
- Protect system integrity
- Ensure fair use of UniMarket

Key interactions/screens:

- Admin dashboard
- User management screen
- Product control screen
- Report handling screen

- Analytics dashboard

Data created/consumed:

- **Created:** admin decisions, report status updates, moderation records
- **Consumed:** user data, product data, reports, analytics, transaction summaries
- **Privacy:** admin access must be restricted and securely controlled

Constraints:

- Admin panel requires strong access control
- Wrong moderation decisions may affect user trust
- Admin actions should be logged for accountability

2.2.2 Payment Service Providers

Description:

Payment service providers such as bKash, Nagad, banks, or payment gateways support transaction-related activities in UniMarket. In the initial version, UniMarket uses manual payment verification through transaction ID.

Objectives:

- Support payment between buyer and seller
- Provide transaction ID after payment
- Help verify payment completion
- Support future payment gateway integration

Responsibilities:

- Process payment outside the UniMarket application
- Provide valid transaction reference
- Maintain secure payment process
- Support error handling through their own service

Interest in System:

- Reliable transaction reference handling
- Secure payment information management
- Smooth future integration with payment gateway

Data created/consumed:

- Created: transaction ID and payment reference
- Consumed: payment amount and receiver payment number
- Privacy: payment reference information must be handled securely

Constraints:

- Manual verification may take time
- Incorrect transaction ID may cause dispute
- Full automated gateway may not be available in the first version

2.2.3 Firebase and Cloud Service Providers

Description:

Firebase and cloud service providers support UniMarket's backend services such as authentication, database, notification, and real-time synchronization. Cloudinary is used for image hosting.

Access level:

- Firebase Authentication
- Cloud Firestore
- Firebase Cloud Messaging
- Cloudinary image hosting
- Local notification support

Needs:

- Secure configuration
- Reliable internet connection
- Proper database rules

- Scalable storage and synchronization
- Secure image upload process

Responsibilities:

- Provide authentication support
- Store marketplace data
- Support real-time chat
- Support push notifications
- Store product/profile image URLs
- Maintain backend availability

Key interactions/screens:

- Login/signup flow
- Product posting system
- Chat system
- Wishlist system
- Notification center
- Seller dashboard
- Admin panel

Data created/consumed:

- Created: authentication tokens, database documents, notification tokens, image URLs
- Consumed: user credentials, product data, chat messages, wishlist data, order data, notification payloads
- Privacy: all sensitive data must be protected using proper security rules

Constraints:

- Internet connection is required
- Firebase quota and billing limits may affect large-scale usage
- Incorrect rules may create security risks

2.3 Development Stakeholders

2.3.1 Developers

Description:

Developers are responsible for designing, implementing, testing, and maintaining the UniMarket system.

Needs:

- Clear and complete requirements
- Well-defined modules
- Maintainable architecture
- Proper database structure
- Reusable widgets and services
- Testable system behavior

Responsibilities:

- Develop frontend using Flutter and Dart
- Integrate Firebase services
- Implement authentication, product, chat, Wishlist, rent, exchange, notification, and dashboard modules
- Maintain clean code structure
- Fix bugs and improve performance
- Ensure secure data handling

2.3.2 Testers

Description:

Testers are responsible for validating functional and non-functional behavior of UniMarket.

Needs:

- Clear functional requirements

- Testable use cases
- Expected outputs
- Test accounts and test data
- Defined error handling scenarios

Responsibilities:

- Test registration and login
- Test product posting, editing, and deletion
- Test search, filter, wishlist, cart, chat, rent, and exchange features
- Test payment transaction ID submission
- Test notification behavior
- Verify UI responsiveness and usability
- Report bugs and confirm fixes

3. Design and Implementation Constraints

To ensure the successful development and operation of the UniMarket: A University Student Marketplace application, a set of design and implementation constraints has been defined. These constraints describe the technologies, tools, cloud services, and security measures used in the system. They provide guidelines for developers and testers to design, implement, test, and maintain the application's user interface, backend services, database, notification system, image upload system, and marketplace features. These constraints also help maintain system reliability, usability, scalability, and data security.

3.1 Frontend Technologies

3.1.1 Flutter

UniMarket is developed using Flutter, a cross-platform mobile application development framework. Flutter allows the application to run on Android and other supported platforms using a single codebase. It is used to design and implement the mobile user interface of UniMarket, including product listing, product details, Wishlist, cart, checkout, chat, seller dashboard, rent request, exchange request, and notification features. Flutter provides a widget-based architecture,

which helps create reusable and responsive UI components. This makes the application easier to maintain and helps ensure a consistent user experience for university students.

3.1.2 Dart

Dart is used as the primary programming language for UniMarket. It handles the application logic, UI behavior, form validation, navigation, data processing, Firebase communication, and service-layer operations. Dart works closely with Flutter to build fast and interactive mobile application features.

3.2 Backend Technologies

3.2.1 Firebase Authentication

Firebase Authentication is used for user registration and login. It verifies user identity and manages secure user sessions. Only authenticated users can access important marketplace features such as posting products, chatting, adding wishlist items, adding products to cart, sending rent requests, and sending exchange requests.

3.2.3 Cloud Firestore

Cloud Firestore is used as the main real-time cloud database for UniMarket. It stores user profiles, product listings, cart items, wishlist items, chat messages, notifications, rent requests, exchange requests, reviews, payment records, and seller dashboard data. Firestore supports real-time updates, which is useful for chat, wishlist, notification, and marketplace listing features.

3.3 Cloud Services

3.3.1 Firebase Cloud Messaging

Firebase Cloud Messaging (FCM) is used to support push notifications in UniMarket. It helps notify users about new chat messages, Wishlist price drops, rent requests, exchange requests, order updates, and other important marketplace activities.

3.3.2 Cloudinary Image Hosting

Cloudinary is used for uploading and hosting product images and profile pictures. When a user uploads an image, the image is stored in Cloudinary and the image URL is saved in Cloud

Firestore. This allows UniMarket to display product and profile images efficiently across the application.

3.4 State Management

The Provider package is used for state management in UniMarket. It helps manage shared data across different screens, such as current user information, navigation state, cart data, wishlist data, and other UI states. Provider makes the application easier to maintain and improves separation between UI and business logic.

3.5 External Packages and APIs

UniMarket uses several external packages and APIs to support important application features. The image_picker package is used to select images from the user's gallery or camera for product posting, product editing, and profile picture updates. The http package is used to communicate with external services such as image upload APIs. The flutter_local_notifications package is used to display local notifications on the user's device, especially when the application is running in the foreground. The intl package is used to format dates, times, timestamps, and Bangladeshi currency values such as product prices, rent prices, and checkout totals. These packages help reduce development complexity and improve the overall functionality of the application.

3.6 Payment Technology

3.6.1 Manual Payment System

UniMarket supports manual payment through bKash and Nagad. Buyers' complete payment outside the app and submit the transaction ID in UniMarket. The seller or admin can verify the payment manually.

3.6.2 Online Payment Gateway System

UniMarket supports an online payment gateway system for secure digital transactions during checkout. Buyers can complete payments using different options such as SSLCommerz, bKash, Nagad, card payment, online banking, and wallet services. After a payment attempt, the system receives the transaction response and updates the order status as successful, failed, or cancelled. For production deployment, payment verification and transaction validation should be handled through a secure backend server to ensure reliability, data integrity, and user security

3.7 Security Measures

3.7.1 HTTPS

UniMarket uses HTTPS-based communication for secure data transmission between the application and cloud services. This helps protect sensitive information such as user profile data, chat messages, payment transaction references, and authentication data.

3.7.2 Authentication and Authorization

Authentication mechanisms are implemented to verify user identity before allowing access to marketplace features. Authorization ensures that users can only access permitted data. For example, sellers can edit or delete only their own product listings, and users can access only their own cart, Wishlist, chat, and notification records.

3.7.3 Firestore Security Rules

Firestore security rules are used to protect database access. These rules help prevent unauthorized users from reading, writing, updating, or deleting sensitive data. They ensure that user information, product ownership, chat messages, payment data, rent requests, and exchange requests are accessed only by authorized users

4. Requirement Specification

The complete requirement specification based on the requirement elicitation process is described in this section. This section defines the functional and non-functional requirements of the UniMarket system to ensure correct, reliable, secure, and user-friendly marketplace operation for university students.

4.1 Functional Requirements

Functional requirements describe the mandatory functions that the UniMarket system must perform. These requirements define how buyers, sellers, renters, exchange requesters, admins, and the system interact with the application and how the system responds.

4.1.1 User Registration and Login

FR 1	User registration and authentication

Description:	The system shall allow university students to register and log in securely using valid credentials. Authentication ensures that only authorized users can access marketplace features such as posting products, chatting, wishlisting, cart management, rent requests, and exchange requests.		
Stakeholders	Buyers, Sellers, Developers	**Priority**	High

4.1.2 User Profile Management

FR 2	Create and update user profile		
Description:	The system shall allow users to create and update profile information such as name, department, contact information, and profile picture. The profile data helps buyers and sellers identify each other in the campus marketplace.		
Stakeholders	Buyers, Sellers	**Priority**	High

4.1.3 Add Product Listing

FR 3	Add product details		
Description:	The system shall allow sellers to add product listings with product title, image, description, category, condition, price, stock quantity, and listing type. The listing type may be Sell Only, Rent Only, Sell & Rent, or Exchange Only.		
Stakeholders	Sellers	**Priority**	High

4.1.4 Edit Product Listing

FR 4	Update product listing

Description:	The system shall allow sellers to edit their own product information, including title, description, price, category, condition, stock, rent price, wanted item, and exchange notes. The updated information shall be reflected on the Home screen and Product Details page.		
Stakeholders	Sellers	**Priority**	High

4.1.5 Delete Product Listing

FR 5	Delete own product listing		
Description:	The system shall allow sellers to delete their own product listings after confirmation. Deleted listings shall no longer appear on the Home screen, Product Details page, or Seller Dashboard.		
Stakeholders	Sellers	**Priority**	High

4.1.6 Product Search and Filter

FR 6	Search and filter products		
Description:	The system shall allow users to search products by name and filter listings by category, price range, condition, and listing type. This helps students quickly find suitable products in the marketplace.		
Stakeholders	Buyers, Renters, Exchange Requesters	**Priority**	High

4.1.7 Product Details View

FR 7	View product details

Description:	The system shall display complete product information including image gallery, title, description, price, rent price, condition, stock, seller details, location, rating, and available action buttons such as Chat, Add to Cart, Rent Now, or Exchange Request.		
Stakeholders	Buyers, Sellers	**Priority**	High

4.1.8 Wishlist Management

FR 8	Save favorite products		
Description:	The system shall allow users to add products to wishlist and remove them when needed. Wishlisted items shall be stored per user and displayed in a dedicated Wishlist screen.		
Stakeholders	Buyers	**Priority**	Medium

4.1.9 Cart and Checkout System

FR 9	Manage cart and checkout		
Description:	The system shall allow users to add sellable products to cart, remove items, update quantity, calculate total price, and proceed to checkout. Cart data shall remain available until checkout or manual removal.		
Stakeholders	Buyers	**Priority**	High

4.1.10 Manual Payment System

FR 10	Submit manual payment transaction ID

Description:	The system shall support manual payment through bKash or Nagad. Buyers shall be able to view payment instructions, submit transaction ID, and wait for seller or admin verification.		
Stakeholders	Buyers, Sellers, Payment Service Providers	**Priority**	High

4.1.11 Online Payment Gateway System

FR 11	Process online payment through payment gateway		
Description:	The system shall support online payment through a secure payment gateway. Buyers shall be able to complete payment using available digital payment options such as SSLCommerz, card payment, mobile banking, online banking, or wallet services. After payment, the system shall update the order status as successful, failed, or cancelled based on the payment response.		
Stakeholders	Buyers, Sellers, Payment Service Providers	**Priority**	High

4.1.12 Rent System

FR 11	Request product rental		
Description:	The system shall allow sellers to post rentable products with daily rent price and allow renters to select rental duration. The system shall automatically calculate total rent and save rent requests with pending, accepted, rejected, or completed status.		
Stakeholders	Renters, Sellers	**Priority**	High

4.1.13 Book Exchange System

FR 12	Request product exchange		
Description:	The system shall allow users to post exchange-only items without price. Buyers shall be able to send exchange requests with an introductory message, and sellers shall be able to accept or reject requests.		
Stakeholders	Exchange Requesters, Sellers	**Priority**	High

4.1.14 Real-Time Chat System

FR 13	Buyer-seller messaging		
Description:	The system shall allow buyers and sellers to communicate using real-time text messaging. Chat messages shall include sender information, message text, timestamp, and related product or chat room information.		
Stakeholders	Buyers, Sellers	**Priority**	High

4.1.15 Rating and Review System

FR 14	Submit rating and review		
Description:	The system shall allow users to provide 1 to 5 star ratings and text reviews after a successful transaction. The system shall update average rating and review count for the product or seller.		
Stakeholders	Buyers, Sellers	**Priority**	Medium

4.1.16 Stock Management

FR 15	Track product quantity		
Description:	The system shall allow sellers to enter product stock quantity and shall reduce available stock after successful purchase. The system shall prevent purchase or cart actions when stock is unavailable.		
Stakeholders	Sellers, Buyers	**Priority**	Medium

4.1.17 Seller Dashboard

FR 17	Manage seller activities		
Description:	The system shall provide a seller dashboard where sellers can view own listings, edit or delete products, view sold items, manage rent and exchange requests, and view basic analytics such as total products, active listings, and earnings.		
Stakeholders	Sellers	**Priority**	High

4.1.18 Notification System

FR 18	Send system notifications		
Description:	The system shall notify users about new chat messages, rent requests, exchange requests, order updates, and other important marketplace activities. Notifications shall be shown in the Notification screen and may also appear as push notifications.		
Stakeholders	Buyers, Sellers	Priority	High

4.1.19 Price Drop Notification

FR 19	Notify wishlist price decrease		
Description:	The system shall notify users when the price of a product saved in their wishlist decreases. The notification shall allow users to open the related Product Details page.		
Stakeholders	Buyers	**Priority**	Medium

4.1.20 Admin Panel

FR 20	Manage users and listings		
Description:	The system shall provide an admin panel for system moderators to manage users, review reports, approve or remove listings, monitor analytics, and maintain marketplace safety.		
Stakeholders	Admin, Developers	**Priority**	Medium

4.1.21 AI Smart Chatbox

FR 21	Quick reply message support		
Description:	The system shall provide a smart chatbox or predefined quick reply system that helps users send common messages such as “Is it available?” or “Last price?”. This feature improves communication speed between buyers and sellers.		
Stakeholders	Buyers, Sellers	**Priority**	Low

4.2 Data Requirements

Data requirements define the essential information that the UniMarket system must capture, store, and manage to support its marketplace services. These data are required for user authentication, product listing, buying, selling, renting, exchanging, chatting, payment verification, wishlist, notification, and review features.

4.2.1 User Information

Description: User name, university email, department, phone number, profile picture, user ID, and account creation information.

Stakeholders: Buyers, Sellers, Admin

Priority: High

4.2.2 Product Data

Description: Product title, description, category, condition, price, image URL, quantity, location, seller ID, seller name, listing type, and product status.

Stakeholders: Buyers, Sellers

Priority: High

4.2.3 Cart and Order Data

Description: Cart item ID, product ID, product name, product image, quantity, price, buyer ID, seller ID, total amount, order status, and checkout information.

Stakeholders: Buyers, Sellers

Priority: High

4.2.4 Payment Data

Description: Payment method, bKash/Nagad number, transaction ID, payment amount, payment status, buyer ID, seller ID, and payment timestamp.

Stakeholders: Buyers, Sellers, Admin, Payment Service Providers

Priority: High

4.2.5 Wishlist Data

Description: User ID, product ID, product title, product image, price, seller ID, and wishlist creation date.

Stakeholders: Buyers

Priority: Medium

4.2.6 Chat Data

Description: Chat room ID, sender ID, receiver ID, message text, timestamp, product ID, and message status.

Stakeholders: Buyers, Sellers

Priority: High

4.2.7 Rent Request Data

Description: Rent request ID, product ID, renter ID, seller ID, start date, end date, rental duration, daily rent price, total rent amount, request status, and creation time.

Stakeholders: Renters, Sellers

Priority: High

4.2.8 Exchange Request Data

Description: Exchange request ID, product ID, requester ID, owner ID, wanted item, exchange message, request status, and creation time.

Stakeholders: Exchange Requesters, Sellers

Priority: High

4.2.9 Review and Rating Data

Description: Review ID, product ID, user ID, user name, rating value, review comment, average rating, review count, and review date.

Stakeholders: Buyers, Sellers

Priority: Medium

4.2.10 Notification Data

Description: Notification ID, user ID, notification type, title, message body, related product/chat/order ID, read status, and timestamp.

Stakeholders: Buyers, Sellers, Admin

Priority: Medium

4.3 Performance Requirements

Performance requirements ensure that UniMarket operates efficiently, responds quickly, and provides real-time marketplace updates to users.

4.3.1 Speed and Latency Requirements

SLR-1: Fast product loading

Description: Product listings should load quickly on the Home screen so that users can browse marketplace items without delay.

Stakeholders: Buyers, Sellers

Priority: High

SLR-2: Fast search and filter response

Description: Product search and filter results should be displayed within a short time after the user enters a keyword or selects filter options.

Stakeholders: Buyers

Priority: High

4.3.2 Real-Time Synchronization Requirements

RTS-1: Real-time chat update

Description: Chat messages between buyer and seller should appear instantly without requiring manual refresh.

Stakeholders: Buyers, Sellers

Priority: High

RTS-2: Real-time wishlist and notification update

Description: Wishlist changes, price drop alerts, rent requests, exchange requests, and notifications should update in real time whenever possible.

Stakeholders: Buyers, Sellers

Priority: Medium

4.3.3 Capacity Requirements

CR-1: Handle multiple users and listings

Description: The system must support a large number of university students, product listings, chat messages, wishlist items, orders, rent requests, and exchange requests simultaneously.

Stakeholders: Buyers, Sellers, Admin

Priority: Medium

CR-2: Store marketplace media and data efficiently

Description: The system must efficiently handle product images, profile pictures, product records, and user-generated data using cloud services.

Stakeholders: Buyers, Sellers, Developers

Priority: Medium

4.4 Business Requirements

Business Requirements describe the high-level organizational goals and objectives that the UniMarket system must achieve. These requirements explain why the system is needed from a university marketplace and student service perspective rather than how it is technically implemented.

BR-01: Support Student-to-Student Marketplace

The UniMarket system shall provide a dedicated platform where university students can buy, sell, rent, and exchange products within their campus community.

BR-02: Improve Trust and Safety in Campus Trading

The system shall reduce unreliable and unorganized trading by allowing authenticated users to post products, communicate through the app, and manage transactions in a structured way.

BR-03: Organize Product Listings

The system shall organize student products by category, price, condition, listing type, and availability so that users can easily find relevant items.

BR-04: Support Multiple Listing Types

The system shall support different marketplace needs by allowing products to be listed as Sell Only, Rent Only, Sell & Rent, or Exchange Only.

BR-05: Enable Real-Time Buyer-Seller Communication

The system shall provide real-time chat so that buyers and sellers can communicate, negotiate, and coordinate product transactions easily.

BR-06: Support Wishlist and Price Drop Tracking

The system shall allow users to save favorite products and receive notifications when wishlisted product prices decrease.

BR-07: Support Manual Payment Verification

The system shall support manual payment through bKash/Nagad transaction ID submission and seller/admin verification.

BR-08: Support Seller Management

The system shall provide a seller dashboard where sellers can manage their own products, view listings, respond to rent/exchange requests, and track marketplace activities.

BR-09: Improve Marketplace Transparency Through Reviews

The system shall allow users to provide ratings and reviews after successful transactions to improve trust and decision-making.

BR-10: Support System Moderation

The system shall allow admin or moderators to manage users, review reported products, remove inappropriate listings, and maintain a safe marketplace environment.

4.5 Business Rules

Business Rules define mandatory constraints and policies that govern how UniMarket operates. These rules must be enforced consistently across the system regardless of user interface or implementation details.

BRULE-01: Authentication Rule

Only registered and authenticated users shall be allowed to perform marketplace actions such as

posting products, adding items to cart, sending messages, creating rent requests, and sending exchange requests.

BRULE-02: Product Ownership Rule

A seller shall only be allowed to edit or delete products that he/she has posted. Users cannot modify products posted by other sellers.

BRULE-03: Listing Type Rule

Each product listing shall have one valid listing type only:

- Sell Only
- Rent Only
- Sell & Rent
- Exchange Only

No other listing type values are permitted.

BRULE-04: Price Requirement Rule

For Sell Only products, a selling price is required. For Rent Only products, a daily rent price is required. For Sell & Rent products, both selling price and daily rent price are required.

BRULE-05: Exchange Listing Rule

Exchange Only products shall not require a selling price or rent price. Instead, the seller must provide the wanted item and exchange notes.

BRULE-06: Stock Management Rule

Product stock quantity shall be reduced only after a successful purchase or confirmed order. Stock shall not be reduced for wishlist, chat, rent request, or exchange request actions.

BRULE-07: Cart Rule

Only products available for sale shall be added to the cart. Rent Only and Exchange Only products shall not be added to the cart.

BRULE-08: Rent Request Rule

Rent requests can only be created for products marked as Rent Only or Sell & Rent. The rental start date must be earlier than the rental end date.

BRULE-09: Exchange Request Rule

Exchange requests can only be created for Exchange Only products. Each exchange request must include requester information and an introductory message.

BRULE-10: Manual Payment Rule

An order shall be considered pending until the buyer submits a valid transaction ID and the seller/admin verifies the payment.

BRULE-11: Wishlist Price Drop Rule

A price drop notification shall be generated only when the current product price becomes lower than the previous price for a product saved in a user's wishlist.

BRULE-12: Review Eligibility Rule

A user shall be allowed to submit a rating and review only after completing a valid transaction with the product or seller.

BRULE-13: Auto Expiry Rule

A product listing may be hidden or marked as expired if it remains active for more than the defined period, such as 30 days, without renewal or update.

BRULE-14: Report Handling Rule

Reported products or users shall be reviewed by the admin. The admin may remove listings or restrict users if a violation is confirmed.

BRULE-15: Data Integrity Rule

User profiles, product data, chat messages, payment records, rent requests, exchange requests, and reviews shall not be modified without proper authorization and validation.

4.6 Dependability Requirements

Dependability is measured through:

• Availability

• Reliability

• Safety

• Security

UniMarket must operate reliably so that students can access product listings, chat with sellers, manage cart and wishlist items, send rent or exchange requests, and receive notifications without unexpected failure.

4.6.1 Reliability and Availability Requirements

RAR-1: Continuous marketplace availability

Description:

UniMarket should be available whenever students need to browse products, post listings, communicate with sellers, or manage transactions.

Stakeholders: Buyers, Sellers, Admin

Priority: High

RAR-2: Reliable real-time data access

Description:

The system shall reliably synchronize product listings, chat messages, wishlist data, cart items, rent requests, exchange requests, and notifications using cloud services.

Stakeholders: Buyers, Sellers

Priority: High

4.6.2 Robustness / Fault-Tolerance Requirements

RFT-1: Fault-free handling of marketplace requests

Description:

The system shall handle multiple user requests such as product search, product upload, chat messages, wishlist updates, cart operations, and checkout actions without crashing or data loss.

Stakeholders: Buyers, Sellers, Developers
Priority: High

RFT-2: Graceful error handling
Description:
If network failure, image upload failure, payment verification error, or database error occurs, UniMarket shall display clear error messages and prevent incomplete or corrupted data submission.
Stakeholders: Buyers, Sellers
Priority: High

4.6.3 Safety-Critical Requirements

There are no direct safety-critical requirements, as UniMarket does not control medical devices, vehicles, banking systems, or life-critical operations. However, the system must safely handle user data, payment references, chat messages, and marketplace records to prevent fraud, misinformation, and unauthorized access.

4.7 Maintainability and Supportability Requirements

4.7.1 Maintainability Requirements

MR-1: Easy update of product listings
Description:
Sellers shall be able to easily modify or delete their own product listings, including price, description, stock, image, rent price, and exchange details.
Stakeholders: Sellers
Priority: Medium
MR-2: Modular code structure
Description:

The system shall maintain a clean architecture using separate models, services, providers, screens, and widgets so that future features can be added easily.

Stakeholders: Developers

Priority: Medium

4.7.2 Supportability Requirements

The system supports:

• Testability

• Extensibility

• Maintainability

• Compatibility

• Configurability

UniMarket should be designed in a way that allows developers to test modules separately, extend features such as payment gateway, admin panel, AI chatbox, and notification system, and maintain the application without affecting unrelated modules.

4.7.3 Adaptability Requirements

UniMarket should be adaptable for future improvements such as advanced payment gateway integration, improved admin panel, AI-based smart chat replies, expanded notification types, and support for more university campuses. The system should allow future feature expansion without requiring complete redesign.

4.8 Security Requirements

Security is essential because UniMarket manages user accounts, product listings, chat messages, payment transaction references, wishlist data, and order information.

Security features include:

• Secure login

• Firebase Authentication

• Authorization-based access

• Firestore security rules

• Secure cloud data storage
• Protected image upload
• Secure payment transaction reference handling

4.8.1 Access Requirements

SR-1: Secure authentication mechanism

Description:

Only registered and authenticated users shall be allowed to access marketplace actions such as posting products, adding items to wishlist/cart, sending messages, submitting payment transaction IDs, and creating rent or exchange requests.

Stakeholders: Buyers, Sellers, Admin

Priority: High

4.8.2 Integrity Requirements

SR-2: Data Integrity

Description:

All user data, product listings, chat messages, cart items, payment records, rent requests, exchange requests, and reviews must be protected against unauthorized modification. Users shall only be able to update data that they are permitted to access.

Stakeholders: Buyers, Sellers, Admin

Priority: High

4.8.3 Privacy Requirements

SR-3: Privacy Protection

Description:

User profile information, contact details, chat messages, wishlist data, transaction references, and marketplace activity shall only be accessible to authorized users. UniMarket shall follow proper privacy and security practices to protect student information.

Stakeholders: Buyers, Sellers

Priority: High

4.9 Usability and Human-Interaction Requirements

4.9.1 Ease of Use Requirements

EUR-1: User-friendly marketplace interface

Description:

The UniMarket application shall be simple and easy to use for university students. Users should be able to browse products, post listings, chat, add items to wishlist/cart, request rent or exchange, and complete checkout without confusion.

Stakeholders: Buyers, Sellers

Priority: High

4.9.2 Personalization and Internationalization Requirements

The system may support basic personalization such as profile picture, user department, saved wishlist items, notification preferences, and marketplace activity history. The interface should primarily use clear English text, and future versions may support Bangla language for better accessibility among local university students.

4.9.3 Understandability and Politeness Requirements

The system shall provide clear, polite, and easy-to-understand messages for all important actions. Error messages, validation alerts, payment status, rent request status, exchange request status, and notification messages should clearly explain what happened and what the user should do next.

4.9.4 Accessibility Requirements

Basic accessibility support shall be provided through readable fonts, clear icons, simple navigation, proper button spacing, and visually understandable UI elements. The system should be usable on common Android devices with different screen sizes.

4.9.5 User Documentation Requirements

UDR-1: System documentation

Description:

Developer documentation, SRS documentation, and basic user guidelines shall be provided to explain how the UniMarket system works and how users can use major features such as product posting, wishlist, chat, rent, exchange, and checkout.

Stakeholders: Developers, Users

Priority: Medium

4.9.6 Training Requirements

Minimal training is required because the UniMarket interface is designed to be intuitive and student-friendly. Short user guidelines or help instructions may be provided for important actions such as posting a product, submitting transaction ID, requesting rent, and sending exchange requests.

4.10 Look and Feel Requirements

4.10.1 Appearance Requirements

AR-1: Clear UI labels

Description:

Mandatory fields such as product title, category, price, rent price, wanted item, quantity, and transaction ID shall be clearly labeled and visually highlighted where necessary.

Stakeholders: Buyers, Sellers

Priority: Medium

4.10.2 Style Requirements

SR-1: Modern Flutter-based UI styling

Description:

Flutter widgets, Material Design components, Google Fonts, cards, icons, buttons, and

responsive layouts shall control the appearance of the UniMarket application. The interface should look clean, modern, and suitable for a student marketplace.

Stakeholders: Users, Developers

Priority: Medium

4.11 Operational and Environmental Requirements

4.11.1 Expected Physical Requirements

No special physical hardware is required. Users only need an Android smartphone or supported device with internet access to use the UniMarket application.

4.11.2 Interface Requirements

The system interfaces with Firebase services for authentication, Cloud Firestore database, Firebase Cloud Messaging notifications, and Cloudinary for image hosting. It also uses mobile device features such as gallery access for image selection and internet connectivity for real-time updates.

4.11.3 Release Requirements

There are no strict release constraints in the initial version. The first release shall focus on core marketplace features such as authentication, product posting, search/filter, chat, wishlist, cart, rent, exchange, seller dashboard, and notification support. Future releases may add advanced admin features, improved AI chatbox support, and enhanced payment gateway integration.

5. Requirement Engineering Process

Software requirements for the UniMarket: A University Student Marketplace system were identified through a structured Requirement Engineering process. This process focused on understanding student needs, analyzing campus marketplace problems, and validating features required for buying, selling, renting, and exchanging products.

The Requirement Engineering process for UniMarket includes:

- Requirement elicitation
- Requirement analysis and negotiation
- Requirement validation
- Change management

This process helped identify major features such as user authentication, product posting, search and filter, wishlist, chat, cart, manual payment, rent system, exchange system, seller dashboard, notification, and review system.

.

5.1 Requirement Elicitation Techniques

Requirement elicitation is the process of gathering system requirements from stakeholders, existing resources, and real-world usage patterns. For UniMarket, multiple elicitation techniques were used to identify the needs of university students, sellers, buyers, renters, exchange requesters, and system administrators. These techniques helped ensure that the system requirements are complete, realistic, user-centered, and suitable for a campus-based marketplace.

5.1.1 Interviews

Description:

One-to-one interviews were conducted with university students who regularly buy, sell, rent, or exchange items within their campus community.

Stakeholders:

• Buyers / Student customers

• Sellers / Student product owners

• Renters and exchange requester

Purpose:

• Identify common problems in campus buying and selling
• Understand difficulties in Facebook group-based trading
• Identify the need for search, filter, wishlist, chat, and notification features
• Understand student expectations for rent and exchange systems
• Identify safety and trust issues between buyers and sellers

Outcome:

Interviews provided important qualitative insights about student marketplace behavior. They helped identify the need for verified user accounts, organized product listings, real-time chat, wishlist, seller dashboard, rent request, exchange request, and secure payment tracking features.

5.1.2 Questionnaires

Description:

A structured questionnaire was created using Google Forms and distributed among university students to collect feedback about the need for a student marketplace application. The questionnaire was used to understand students' current buying, selling, renting, and exchanging behavior, as well as their expectations from UniMarket.

Purpose:

- Identify how students currently buy and sell used products
- Understand whether students need a dedicated university marketplace app
- Find out the most important features for students
- Collect opinions about rent, exchange, wishlist, chat, and payment systems
- Prioritize UniMarket features based on student responses

Sample Questions:

- Do you currently buy or sell products through Facebook groups or Messenger?
- Would you use a dedicated university student marketplace app?
- Which feature is most important to you? (Search / Chat / Wishlist / Rent / Exchange / Payment / Review)

- Do you want a rent system for books, calculators, or study materials?
- Do you want a book exchange feature inside the app?
- Do you want price drop notifications for wishlist products?
- What problems do you face while buying or selling items online?

Outcome:

The Google Form responses helped identify the major needs of university students. The collected feedback showed that students need a safe, organized, and easy-to-use platform for buying, selling, renting, and exchanging products. The responses also helped prioritize important features such as product search, real-time chat, wishlist, rent system, exchange system, manual payment, and notification support.

5.1.3 Focus Group Discussions

Description:

Focus group discussions were conducted with small groups of university students to discuss shared problems and expectations related to campus marketplace activities.

Purpose:

- Discuss common challenges in buying and selling used student items
- Compare student opinions on rent, exchange, and payment systems
- Identify conflicting needs between buyers and sellers
- Understand trust issues, product condition concerns, and price negotiation behavior
- Collect feedback on the proposed UniMarket workflow

Outcome:

Focus group discussions revealed important usability and trust-related concerns. Students preferred a simple interface, clear product details, real-time chat, wishlist, seller information, and a safe way to manage rent and exchange requests.

5.1.4 User Interface Analysis

Description:

Existing marketplace and e-commerce applications were studied to understand common UI patterns and usability standards.

Purpose:

- Identify useful marketplace UI patterns
- Study product card layouts, search bars, filters, cart screens, and chat interfaces
- Design navigation suitable for university students
- Avoid complex workflows in product posting, checkout, rent, and exchange features
- Improve the appearance of product details, wishlist, seller dashboard, and notification screens

Outcome:

User interface analysis supported the design of a simple, modern, and student-friendly UniMarket interface. It helped improve screen layout, navigation flow, product listing design, action buttons, and overall usability.

5.2 Sample of Requirement Collection

Requirement collection consolidates information gathered through elicitation techniques into structured system needs. For UniMarket, requirements were collected from different stakeholder groups such as buyers, sellers, renters, exchange requesters, and admin/system moderators.

5.2.1 Requirement Collection – Buyers / Student Customers

Techniques Used:

- Interviews
- Questionnaires
- Focus group discussions

Findings:

- Students often use Facebook groups or Messenger to buy and sell products
- They face difficulty finding trusted sellers and accurate product information
- Search, filter, wishlist, and chat features are highly needed
- Students want a simple checkout and payment tracking system

Key Requirements Identified:

- Product search and filter
- Product details page
- Wishlist system
- Real-time chat
- Cart and checkout system
- Manual payment transaction ID submission

5.2.2 Requirement Collection – Sellers / Student Product Owners

Techniques Used:

- Interviews
- Questionnaires
- Focus group discussions

Findings:

- Sellers need an easy way to post products with images and descriptions
- They want to edit or delete their own listings
- They need a dashboard to manage products, rent requests, and exchange requests
- Notifications are needed for new messages and buyer requests

Key Requirements Identified:

- Add product listing
- Edit and delete product
- Seller dashboard
- Product image upload
- Rent request management

- Exchange request management
- Notification system

5.2.3 Requirement Collection – Renters and Exchange Requesters

Techniques Used:

- Interviews
- Questionnaires
- User interface analysis

Findings:

- Students want to rent books, calculators, electronics, and study materials for short periods
- Renters need clear daily rent price and automatic total calculation
- Students also want a book exchange feature without a price field
- Exchange coordination should be done through chat

Key Requirements Identified:

- Rent system
- Rental duration selection
- Auto total rent calculation
- Exchange-only product listing
- Wanted item and exchange notes
- Chat-based coordination

5.3 Conflicting Requirements and Negotiation

CR-01: Feature Richness vs Simplicity

- Conflict: Adding many features such as rent, exchange, wishlist, chat, payment, and admin panel may make the app complex.

- Decision: Core features will be kept simple and beginner-friendly, while advanced features will be added gradually.
- Status: Agreed

CR-02: Buyer Convenience vs Seller Control

- Conflict: Buyers want quick access to products and requests, while sellers need control over product availability, price, rent, and exchange decisions.
- Decision: Buyers can send requests, but sellers will have authority to accept or reject rent and exchange requests.
- Status: Agreed

CR-03: User Privacy vs Marketplace Communication

- Conflict: Buyers and sellers need to communicate, but personal contact details should not be unnecessarily exposed.
- Decision: Real-time in-app chat will be used for communication, and sensitive user data will be protected.
- Status: Agreed

CR-04: Open Product Posting vs Marketplace Safety

- Conflict: Allowing all users to post products freely may increase fake or inappropriate listings.
- Decision: Admin/report handling features will be included to monitor and remove problematic listings.
- Status: Agreed

5.4 Requirement Validation

Requirement validation ensures that the documented requirements for UniMarket: A University Student Marketplace are correct, complete, consistent, and achievable. Some requirements may appear clear during documentation but may reveal missing details, conflicts, or technical limitations during implementation or testing. Therefore, validation activities are important to

ensure that UniMarket meets student needs and supports buying, selling, renting, exchanging, chatting, payment tracking, wishlist, and notification features properly.

5.4.1 Review the Requirements

A structured peer review process is used to examine the UniMarket requirements. This review involves developers, project supervisors, testers, and selected student users who analyze the requirements from different perspectives.

The reviewers carefully inspect:

- Functional and non-functional requirements
- User workflows for buyers, sellers, renters, and exchange requesters
- Product posting, search, chat, wishlist, cart, rent, and exchange flows
- Security, privacy, usability, and marketplace safety requirements

The goal of this review is to detect ambiguities, inconsistencies, missing requirements, and unrealistic expectations early in the development process. Peer review improves requirement quality and reduces costly changes during later implementation.

5.4.2 Test the Requirements

Requirements for UniMarket are validated by deriving test cases directly from the documented requirements. These tests define how the system should behave under specific conditions, such as user login, product posting, wishlist update, chat message sending, checkout, rent request, exchange request, and notification delivery.

Testing helps verify that:

- Each requirement is testable and measurable
- Expected system behavior is clearly defined
- Functional requirements can be correctly implemented
- Error handling works properly for invalid input or failed operations

By testing requirements before and during development, UniMarket ensures that the system behaves as expected and supports student marketplace activities reliably.

5.4.3 Simulate the Requirements

Requirement simulation is performed using low-fidelity and interactive prototypes to represent the behavior of UniMarket before full implementation. Simulation helps stakeholders visualize how product posting, searching, filtering, chatting, checkout, rent request, exchange request, and notification flows will work in real usage.

Simulation activities include:

- Clickable UI mockups
- Product posting flow demonstration
- Search and filter flow demonstration
- Cart and checkout flow demonstration
- Rent and exchange request flow demonstration
- Chat and notification flow demonstration

This approach helps stakeholders understand the proposed system before final implementation. Simulation also helps identify usability issues, missing features, and workflow problems early, improving the overall quality of UniMarket.

5.5 Change Management

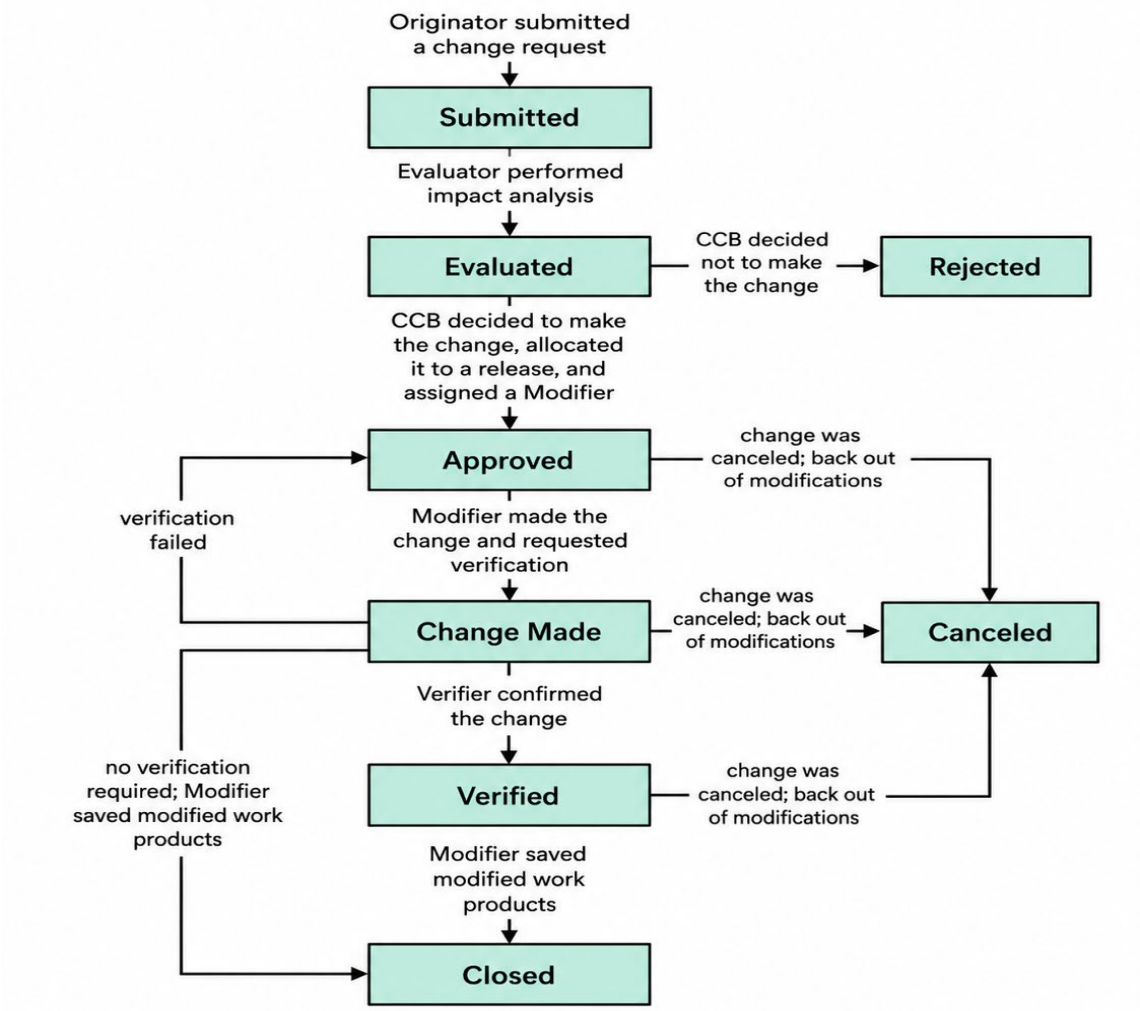
Change management is used in UniMarket to control requirement changes and prevent unnecessary scope expansion. Any change request from users, developers, testers, or the project supervisor must be reviewed before implementation.

Change management activities include:

- Reviewing bug fixes and feature requests
- Checking the impact on buyers, sellers, renters, and admins
- Evaluating effects on modules such as product listing, chat, cart, rent, exchange, and payment
- Adjusting future release scope when needed

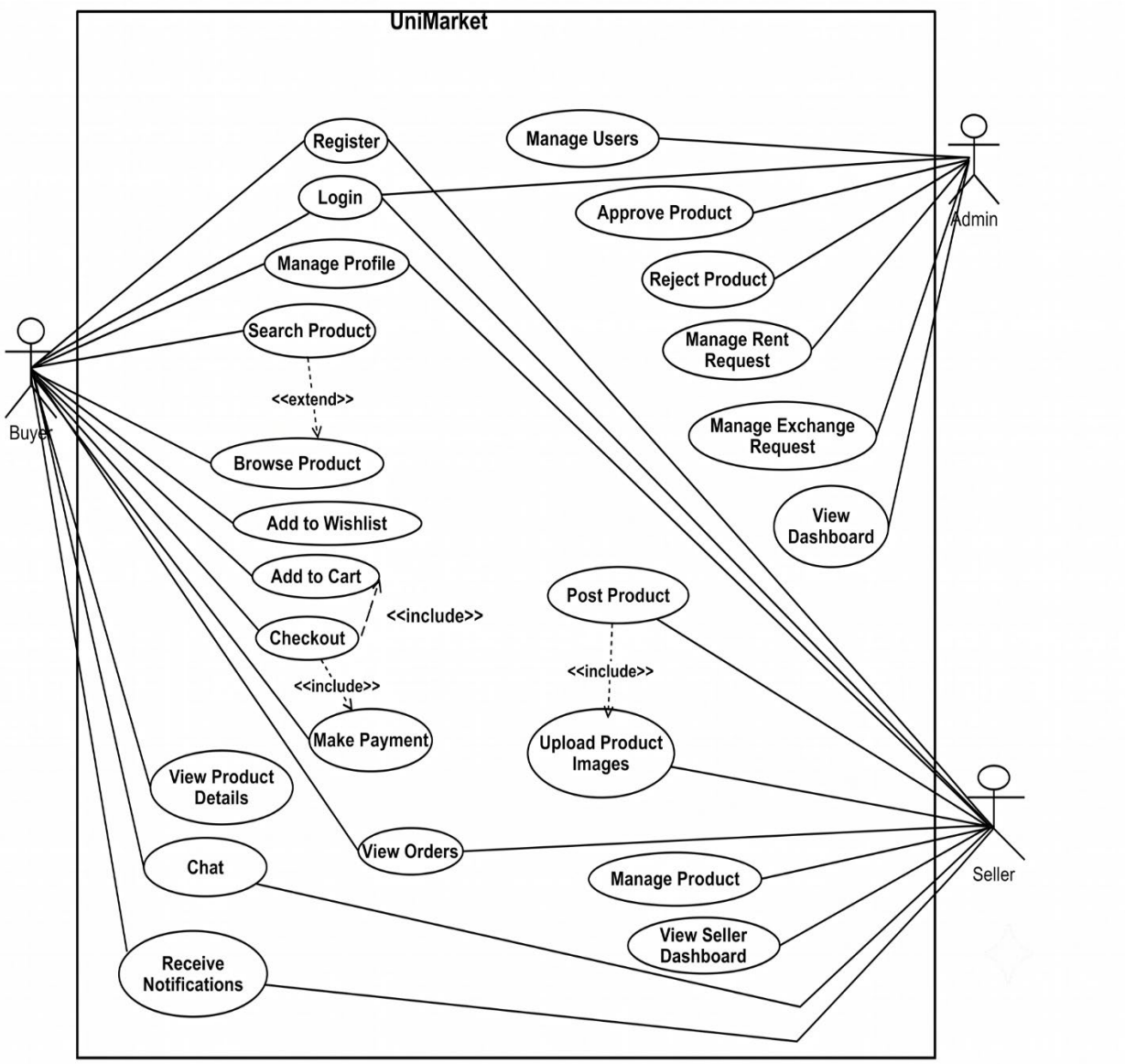
- Approving, delaying, or rejecting changes

This process helps UniMarket remain organized, maintainable, and aligned with its main goal of providing a secure student marketplace.



UniMarket Change Request Flow

6. Use Case Diagram



7.Use Case Descriptions

7.1 UC-01: Manage Profile

Field	Description
Use Case No	UC-01
Use Case	Manage Profile
Goal	Allows buyer/seller to view and update personal profile information in the UniMarket system.
Preconditions	User must be registered and logged into the UniMarket application.
Success End Condition	Profile information is successfully updated and saved in the system.
Failed End Condition	Profile update fails and previous information remains unchanged.
Primary Actors	Buyer, Seller
Trigger	User selects the “Manage Profile” option from the profile or settings menu.
Main Success Flows	1) User opens the profile section. 2) UniMarket displays current profile details. 3) User edits profile information such as name, department, phone number, and profile picture. 4) User submits the updated information. 5) UniMarket validates the input. 6) If a new profile picture is selected, Cloudinary uploads the image and returns the image URL. 7) Firebase System updates the profile data. 8) UniMarket shows a confirmation message.

Alternative Flows	3a) If the user cancels the update, the system discards changes and returns to the profile screen. 4a) If invalid data is entered, the system displays an error message and requests correction. 6a) If image upload fails, the system keeps the previous profile picture and shows an upload error message. 7a) If database update fails, the system shows an error message and keeps previous information unchanged.
Quality Requirements	1) Profile update should complete within 5 seconds under normal internet connection. 2) User profile data shall be stored securely in the database. 3) Validation messages must clearly explain errors in simple language. 4) The profile screen should be easy to use and responsive on mobile devices.

7.2 UC-02: Search Product

Field	Description
Use Case No	UC-02
Use Case	Search Product
Goal	Allows buyer to search for products by entering product name or keyword in the UniMarket application.
Preconditions	Buyer must be logged into the UniMarket application and product listings must be available in the system.
Success End Condition	Matching products are displayed successfully based on the search keyword.
Failed End Condition	No matching product is found or the search result cannot be loaded.
Primary Actors	Buyer
Trigger	Buyer enters a keyword in the search bar on the Home screen.

Main Success Flows	1) Buyer opens the Home screen. 2) Buyer enters a product name or keyword in the search bar. 3) UniMarket receives the search keyword. 4) Firebase System retrieves matching product listings. 5) UniMarket displays products that match the search keyword. 6) Buyer selects a product to view details.
Alternative Flows	2a) If buyer clears the search bar, the system displays all available products again. 4a) If no matching product is found, the system displays a "No products found" message. 4b) If data loading fails, the system shows an error message and allows retry.
Quality Requirements	1) Search results should appear within a short time under normal internet connection. 2) Search should be simple and easy to use. 3) The system should not crash if no product matches the keyword. 4) Error messages should be clear and user-friendly.

7.3 UC-03: Filter Product

Field	Description
Use Case No	UC-03
Use Case	Filter Product
Goal	Allows buyer to filter products based on category, price range, condition, and listing type.
Preconditions	Buyer must be logged into the UniMarket application and product listings must be available.
Success End Condition	Filtered products are displayed according to the selected filter criteria.
Failed End Condition	No products match the selected filters or the filtered result cannot be loaded.
Primary Actors	Buyer

Trigger	Buyer selects filter options from the Home screen or product listing section.
Main Success Flows	1) Buyer opens the Home screen. 2) Buyer selects filter options such as category, price range, product condition, or listing type. 3) UniMarket applies the selected filter criteria. 4) Firebase System retrieves product data according to the selected filters. 5) UniMarket displays the filtered product list. 6) Buyer selects a product to view details.
Alternative Flows	2a) If buyer resets filters, the system displays all available products again. 4a) If no products match the selected filters, the system displays a "No products found" message. 4b) If filter data cannot be loaded, the system shows an error message and allows retry.
Quality Requirements	1) Filter results should load quickly under normal internet connection. 2) Filter options should be clear and easy to understand. 3) The system should support multiple filters without crashing. 4) The interface should remain responsive on mobile devices.

7.4 UC-04: View Product Details

Field	Description
Use Case No	UC-04
Use Case	View Product Details
Goal	Allows buyer to view complete product information before making a purchase, rent request, or exchange request.
Preconditions	Buyer must be logged into the UniMarket application and the selected product must be available in the system.
Success End Condition	Product details are displayed successfully with all necessary information.

Failed End Condition	Product details cannot be loaded or the product is unavailable.
Primary Actors	Buyer
Trigger	Buyer selects a product from the Home screen, search result, wishlist, or product listing section.
Main Success Flows	1) Buyer opens the product list. 2) Buyer selects a product card. 3) UniMarket requests product information from Firebase System. 4) Firebase System retrieves product details. 5) UniMarket displays product images, title, price, description, condition, stock, seller information, rating, and listing type. 6) Buyer can choose actions such as Add to Wishlist, Add to Cart, Chat, Rent Now, or Exchange Request depending on the product listing type.
Alternative Flows	3a) If the product has been deleted, the system displays “Product unavailable.” 4a) If product data fails to load, the system displays an error message and allows retry. 5a) If product image is unavailable, the system displays a placeholder image.
Quality Requirements	1) Product details should load within a short time under normal internet connection. 2) Product information must be clear and readable. 3) The page should be responsive on mobile devices. 4) The system should prevent buyer from adding out-of-stock products to cart.

7.5 UC-05: Add to Wishlist

Field	Description
Use Case No	UC-05
Use Case	Add to Wishlist
Goal	Allows buyer to save favorite products for later viewing.

Preconditions	Buyer must be logged into the UniMarket application and the product must exist in the system.
Success End Condition	Product is successfully added to the buyer's wishlist.
Failed End Condition	Product is not added to wishlist due to system error or invalid product data.
Primary Actors	Buyer
Trigger	Buyer taps the wishlist/heart icon on a product card or Product Details page.
Main Success Flows	1) Buyer views a product card or Product Details page. 2) Buyer taps the wishlist icon. 3) UniMarket checks whether the product is already in the buyer's wishlist. 4) If the product is not already saved, Firebase System stores the wishlist item under the buyer's account. 5) UniMarket updates the heart icon state. 6) UniMarket shows confirmation that the product has been added to wishlist.
Alternative Flows	3a) If the product is already in wishlist, tapping the icon may remove it from wishlist. 4a) If Firebase update fails, the system displays an error message. 4b) If the product no longer exists, the system displays "Product unavailable."
Quality Requirements	1) Wishlist update should happen quickly and reflect in real time. 2) The wishlist icon state must remain synchronized across Home, Product Details, and Wishlist screens. 3) The system should show clear feedback after adding or removing a wishlist item. 4) Wishlist data must be stored securely per user.

7.6 UC-06: Add to Cart

Field	Description
Use Case No	UC-06
Use Case	Add to Cart
Goal	Allows buyer to add sellable products to the cart before checkout.
Preconditions	Buyer must be logged into the UniMarket application, the product must be available for sale, and stock quantity must be greater than zero.
Success End Condition	Product is successfully added to the buyer's cart.
Failed End Condition	Product is not added to cart due to insufficient stock, invalid listing type, or system error.
Primary Actors	Buyer
Trigger	Buyer taps the "Add to Cart" button from the Product Details page.
Main Success Flows	1) Buyer opens the Product Details page. 2) Buyer taps the Add to Cart button. 3) UniMarket checks product listing type and stock availability. 4) If the product is sellable and available, Firebase System adds the product to the buyer's cart. 5) If the product already exists in the cart, the system updates the quantity. 6) UniMarket shows confirmation that the product has been added to cart.
Alternative Flows	3a) If the product is Rent Only or Exchange Only, the Add to Cart option is not available. 3b) If stock is zero, the system displays "Out of Stock." 4a) If cart update fails, the system displays an error message. 5a) If requested quantity exceeds available stock, the system prevents the update and shows a warning.
Quality Requirements	1) Cart update should complete within a short time under normal internet connection. 2) The system must prevent adding unavailable or non-sellable products to cart.

	3) Cart quantity must not exceed available product stock. 4) Cart data must be stored securely for each buyer.

7.7 UC-07: Checkout Product

Field	Description
Use Case No	UC-07
Use Case	Checkout Product
Goal	Allows buyer to review selected cart items, confirm order details, and proceed to payment.
Preconditions	Buyer must be logged into the UniMarket application and must have at least one valid product in the cart.
Success End Condition	Checkout information is confirmed and an order is created successfully.
Failed End Condition	Checkout fails due to empty cart, unavailable product, insufficient stock, or system error.
Primary Actors	Buyer
Trigger	Buyer selects the “Checkout” option from the Cart screen.
Main Success Flows	1) Buyer opens the Cart screen. 2) UniMarket displays selected products, quantity, price, and total amount. 3) Buyer reviews cart items and checkout information. 4) Buyer confirms the checkout. 5) UniMarket validates product availability and stock quantity. 6) Firebase System creates an order record with pending payment status. 7) UniMarket redirects buyer to the payment process.
Alternative Flows	2a) If the cart is empty, the system displays “Your cart is empty.” 5a) If any product is out of stock, the system prevents checkout and displays an error message. 5b) If product quantity exceeds available stock, the system asks buyer to update quantity.

	6a) If order creation fails, the system displays an error message and keeps cart unchanged.
Quality Requirements	1) Checkout summary should be clear and easy to understand. 2) Checkout validation should prevent invalid orders. 3) Order creation should complete within a short time under normal internet connection. 4) Buyer cart and order data must be stored securely.

7.8 UC-08: Make Payment

Field	Description
Use Case No	UC-08
Use Case	Make Payment
Goal	Allows buyer to complete payment for an order using manual payment or SSLCommerz payment gateway.
Preconditions	Buyer must have created a valid order through checkout and the order status must be pending payment.
Success End Condition	Payment is completed or submitted successfully and the order status is updated.
Failed End Condition	Payment fails, is cancelled, or payment information cannot be verified.
Primary Actors	Buyer
Trigger	Buyer selects a payment option after checkout.
Main Success Flows	1) Buyer opens the payment screen after checkout. 2) UniMarket displays available payment options such as manual payment or SSLCommerz gateway. 3) Buyer selects a payment method. 4) For manual payment, buyer enters transaction ID after paying through bKash/Nagad. 5) For SSLCommerz, buyer is redirected to the payment gateway page. 6) Payment Gateway processes the payment and returns payment status. 7) Firebase System updates payment and order status. 8) UniMarket displays payment success or order confirmation message.

Alternative Flows	3a) If buyer cancels payment, the order remains unpaid or cancelled. 4a) If transaction ID is invalid or empty, the system requests correction. 5a) If payment gateway fails to load, the system displays an error message. 6a) If payment fails, the system updates the order as failed and shows payment failure message. 7a) If order update fails, the system displays an error message and stores the issue for retry.
Quality Requirements	1) Payment process should be clear and secure. 2) Sensitive payment information must not be exposed publicly. 3) Payment status should be updated accurately. 4) The system should handle success, failure, and cancellation properly.

7.9 UC-09: Send Message

Field	Description
Use Case No	UC-09
Use Case	Send Message
Goal	Allows buyer and seller to communicate through real-time chat for product discussion, negotiation, rent, or exchange coordination.
Preconditions	Buyer and seller must be logged into the UniMarket application and a valid chat room must exist or be created.
Success End Condition	Message is successfully sent, stored, and displayed in the chat room.
Failed End Condition	Message is not sent due to network error, invalid chat room, or system failure.
Primary Actors	Buyer, Seller
Trigger	User selects the Chat option from Product Details page or Chat screen.
Main Success Flows	1) User opens a chat room. 2) UniMarket displays previous messages between buyer and seller. 3) User enters a text message or selects media such as image, video, audio, or file. 4) If media is selected, Cloudinary uploads the media and returns a secure URL.

	5) Firebase System stores the message data in real time. 6) UniMarket displays the message in the chat room. 7) The receiver gets a notification for the new message.
Alternative Flows	3a) If user cancels message writing, no message is sent. 4a) If media upload fails, the system shows an upload error and does not send invalid media. 5a) If message sending fails, the system shows an error and allows retry. 5b) If chat room data is invalid, the system prevents message sending and displays an error message.
Quality Requirements	1) Text messages should appear in real time under normal internet connection. 2) Media messages should be uploaded securely before sending. 3) Chat interface should be simple and responsive. 4) The system must not send messages with empty user ID, chat ID, or invalid media URL.

7.10 UC-10: Request Rent

Field	Description
Use Case No	UC-10
Use Case	Request Rent
Goal	Allows buyer/renter to request a product for rent by selecting rental duration and confirming total rent amount.
Preconditions	Buyer must be logged into the UniMarket application and the selected product must be available for rent.
Success End Condition	Rent request is successfully submitted and saved with pending status.
Failed End Condition	Rent request fails due to invalid date range, unavailable product, or system error.
Primary Actors	Buyer / Renter
Trigger	Buyer selects the “Rent Now” option from the Product Details page.

Main Success Flows	1) Buyer opens a rentable product details page. 2) Buyer selects the Rent Now option. 3) UniMarket displays date selection options. 4) Buyer selects rental start date and end date. 5) UniMarket calculates rental duration and total rent amount based on daily rent price. 6) Buyer submits the rent request. 7) Firebase System saves the rent request with pending status. 8) Seller receives the rent request and notification.
Alternative Flows	3a) If buyer cancels date selection, no rent request is created. 4a) If selected date range is invalid, the system displays an error message. 5a) If rent price is missing, the system prevents request submission. 7a) If request saving fails, the system displays an error message and allows retry.
Quality Requirements	1) Rent duration and total rent amount must be calculated accurately. 2) Rent request should be saved quickly under normal internet connection. 3) The system should clearly show rent status such as pending, accepted, or rejected. 4) Rent request data must be stored securely and linked to buyer, seller, and product.

7.11 UC-11: Request Exchange

Field	Description
Use Case No	UC-11
Use Case	Request Exchange
Goal	Allows buyer/exchange requester to send an exchange request for an exchange-only product.
Preconditions	Buyer must be logged into the UniMarket application and the selected product must be listed as Exchange Only.
Success End Condition	Exchange request is successfully submitted and saved with pending status.
Failed End Condition	Exchange request fails due to missing message, unavailable product, or system error.

Primary Actors	Buyer / Exchange Requester
Trigger	Buyer selects the “Exchange Request” option from the Product Details page.
Main Success Flows	1) Buyer opens an exchange-only product details page. 2) UniMarket displays wanted item and exchange notes. 3) Buyer clicks the Exchange Request button. 4) UniMarket opens a request message dialog. 5) Buyer enters an introductory exchange message. 6) Buyer submits the exchange request. 7) Firebase System saves the exchange request with pending status. 8) Seller receives the exchange request and notification.
Alternative Flows	3a) If the product is not Exchange Only, the Exchange Request option is not shown. 4a) If buyer cancels the request dialog, no request is created. 5a) If the message field is empty, the system requests the buyer to enter a message. 7a) If request saving fails, the system displays an error message and allows retry.
Quality Requirements	1) Exchange request should be saved quickly under normal internet connection. 2) The system must clearly show exchange status such as pending, accepted, or rejected. 3) The exchange message must be stored securely. 4) The system should notify the seller after a successful exchange request.

7.12 UC-12: Give Rating and Review

Field	Description
Use Case No	UC-12
Use Case	Give Rating and Review
Goal	Allows buyer to provide a star rating and text review after completing a valid transaction.
Preconditions	Buyer must be logged in and must have completed a valid purchase or transaction for the product.

Success End Condition	Rating and review are successfully saved and displayed in the product review section.
Failed End Condition	Review submission fails or buyer is not eligible to review the product.
Primary Actors	Buyer
Trigger	Buyer selects the “Write a Review” option from the Product Details page or completed order section.
Main Success Flows	1) Buyer opens the product details or completed order section. 2) UniMarket checks whether the buyer is eligible to review. 3) Buyer selects a star rating from 1 to 5. 4) Buyer writes a text review. 5) Buyer submits the review. 6) Firebase System saves the review data. 7) UniMarket updates the product’s average rating and review count. 8) The review appears in the review list.
Alternative Flows	2a) If the buyer has not completed a valid transaction, the review option is not shown. 3a) If buyer does not select a rating, the system requests a rating value. 4a) If review text is empty, the system may allow only rating or request a comment depending on validation rules. 6a) If review saving fails, the system displays an error message.
Quality Requirements	1) Rating and review submission should complete within a short time. 2) A user should not submit duplicate reviews for the same product transaction. 3) Average rating should update accurately. 4) Review content must be linked securely to buyer, product, and seller.

7.13 UC-13: View Notifications

Field	Description
Use Case No	UC-13
Use Case	View Notifications

Goal	Allows buyer and seller to view important updates such as chat messages, price drops, rent requests, exchange requests, and order updates.
Preconditions	User must be logged into the UniMarket application.
Success End Condition	Notifications are displayed successfully in the Notification screen.
Failed End Condition	Notifications cannot be loaded due to network error or database issue.
Primary Actors	Buyer, Seller
Trigger	User taps the notification icon or opens the Notification screen.
Main Success Flows	1) User opens the Notification screen. 2) UniMarket requests notification data from Firebase System. 3) Firebase System retrieves notifications related to the current user. 4) UniMarket displays notifications in a list. 5) User taps a notification. 6) UniMarket marks the notification as read. 7) UniMarket navigates the user to the related screen such as Chat Room, Product Details, Seller Dashboard, or Order screen.
Alternative Flows	3a) If no notifications exist, the system displays “No notifications yet.” 3b) If notification loading fails, the system displays an error message and allows retry. 5a) If the related product or chat is unavailable, the system displays a friendly message.
Quality Requirements	1) Notifications should load quickly under normal internet connection. 2) Unread notifications should be clearly marked. 3) Notification navigation should be accurate. 4) User notification data must be private and accessible only to the correct user.

7.14 UC-14: Post Product

Field	Description
Use Case No	UC-14
Use Case	Post Product

Goal	Allows seller to create a new product listing for sale, rent, sell & rent, or exchange.
Preconditions	Seller must be logged into the UniMarket application.
Success End Condition	Product listing is successfully created and displayed in the marketplace.
Failed End Condition	Product listing is not created due to invalid input, image upload failure, or database error.
Primary Actors	Seller
Trigger	Seller selects the “Sell” or “Post Product” option from the app navigation.
Main Success Flows	1) Seller opens the Post Product screen. 2) Seller selects listing type such as Sell Only, Rent Only, Sell & Rent, or Exchange Only. 3) Seller enters product title, description, category, condition, location, price/rent price, stock, wanted item, or exchange notes based on listing type. 4) Seller selects product image from gallery or camera. 5) Cloudinary System uploads the image and returns image URL. 6) UniMarket validates all required fields. 7) Firebase System saves the product information. 8) UniMarket shows confirmation and the product appears in the marketplace.
Alternative Flows	2a) If Exchange Only is selected, price fields are hidden and wanted item is required. 2b) If Rent Only is selected, daily rent price is required and sell price is not required. 4a) If seller does not select an image, the system may show a placeholder or request image upload. 5a) If image upload fails, the system displays an upload error message. 6a) If required fields are missing, the system displays validation errors. 7a) If product saving fails, the system displays an error message and allows retry.
Quality Requirements	1) Product posting should be simple and user-friendly. 2) Product image upload should be secure and reliable.

	3) Required fields should be clearly marked. 4) Product data must be saved correctly with seller ID and listing type.

7.15 UC-15: Edit Product

Field	Description
Use Case No	UC-15
Use Case	Edit Product
Goal	Allows seller to update information of an existing product listing.
Preconditions	Seller must be logged in and must be the owner of the product listing.
Success End Condition	Product information is successfully updated and reflected in the marketplace.
Failed End Condition	Product update fails and previous product information remains unchanged.
Primary Actors	Seller
Trigger	Seller selects the Edit option from Seller Dashboard or My Listings section.
Main Success Flows	1) Seller opens Seller Dashboard. 2) Seller selects a product from My Listings. 3) Seller taps the Edit option. 4) UniMarket displays the existing product information in an editable form. 5) Seller updates product details such as title, description, price, rent price, category, condition, stock, or exchange information. 6) If seller selects a new image, Cloudinary System uploads the image and returns a new image URL. 7) UniMarket validates the updated information. 8) Firebase System updates the product document. 9) UniMarket shows confirmation and updated product details appear in the marketplace.
Alternative Flows	3a) If seller cancels editing, the system returns to Seller Dashboard without saving changes.

	5a) If required fields are empty, the system displays validation messages. 6a) If new image upload fails, the system keeps the previous image and shows an upload error. 8a) If database update fails, the system displays an error message and keeps previous information unchanged. 8b) If seller is not the owner of the product, the system blocks the edit action.
Quality Requirements	1) Product update should complete within a short time under normal internet connection. 2) Only the owner seller should be allowed to edit the product. 3) Product data must remain consistent after update. 4) Validation messages should be clear and easy to understand.

7.16 UC-16: Delete Product

Field	Description
Use Case No	UC-16
Use Case	Delete Product
Goal	Allows seller to remove an existing product listing from the UniMarket marketplace.
Preconditions	Seller must be logged in and must be the owner of the product listing.
Success End Condition	Product listing is successfully deleted or removed from the marketplace.
Failed End Condition	Product deletion fails and the product remains available in the system.
Primary Actors	Seller
Trigger	Seller selects the Delete option from Seller Dashboard or My Listings section.
Main Success Flows	1) Seller opens Seller Dashboard. 2) Seller views products under My Listings. 3) Seller selects the product to delete.

	4) Seller taps the Delete option. 5) UniMarket displays a confirmation dialog. 6) Seller confirms the deletion. 7) Firebase System removes or marks the product as deleted. 8) UniMarket updates the Seller Dashboard and Home screen.
Alternative Flows	5a) If seller cancels the confirmation dialog, no product is deleted. 6a) If seller is not the owner of the product, the system blocks the delete action. 7a) If database deletion fails, the system displays an error message and keeps the product unchanged.
Quality Requirements	1) Only the owner seller should be allowed to delete the product. 2) The system must ask for confirmation before deleting. 3) Deleted products should not appear in the active marketplace list. 4) The deletion process should complete quickly under normal internet connection.

7.17 UC-17: Manage Seller Dashboard

Field	Description
Use Case No	UC-17
Use Case	Manage Seller Dashboard
Goal	Allows seller to view and manage product listings, orders, rent requests, exchange requests, and dashboard summary.
Preconditions	Seller must be logged into the UniMarket application.
Success End Condition	Seller dashboard loads successfully with listings, requests, and summary information.
Failed End Condition	Seller dashboard data cannot be loaded due to network or database error.
Primary Actors	Seller
Trigger	Seller selects the “Seller Dashboard” option from the Profile screen.

Main Success Flows	1) Seller opens the Profile screen. 2) Seller selects Seller Dashboard. 3) UniMarket requests seller-related data from Firebase System. 4) Firebase System retrieves seller products, orders, rent requests, and exchange requests. 5) UniMarket displays dashboard summary such as total products, active listings, sold items, and total earnings. 6) Seller views My Listings, Rent Requests, Exchange Requests, and Orders sections. 7) Seller performs actions such as edit product, delete product, accept/reject requests, or view orders.
Alternative Flows	4a) If no product exists, the system displays "No listings yet." 4b) If no rent or exchange requests exist, the system displays an empty state message. 4c) If data loading fails, the system displays an error message and allows retry.
Quality Requirements	1) Dashboard data should load within a short time under normal internet connection. 2) Dashboard cards and sections should be clear and easy to understand. 3) Seller should only see data related to his/her own listings. 4) The dashboard should be responsive on mobile devices.

7.18 UC-18: View Orders / Sold Items

Field	Description
Use Case No	UC-18
Use Case	View Orders / Sold Items
Goal	Allows seller to view purchase orders, sold products, payment status, and transaction information.
Preconditions	Seller must be logged in and at least one order or purchase request may exist for the seller's product.
Success End Condition	Seller can view order details and sold item information successfully.

Failed End Condition	Order or sold item data cannot be loaded.
Primary Actors	Seller
Trigger	Seller opens the Orders or Sold Items section from Seller Dashboard.
Main Success Flows	1) Seller opens Seller Dashboard. 2) Seller selects Orders or Sold Items section. 3) UniMarket requests order data related to the seller. 4) Firebase System retrieves matching order records. 5) UniMarket displays product title, buyer name, quantity, total amount, payment method, transaction ID, payment status, and order status. 6) Seller reviews the order details. 7) Seller may confirm manual payment or mark an order as completed if applicable.
Alternative Flows	4a) If no order exists, the system displays "No orders yet." 4b) If order data cannot be loaded, the system displays an error message and allows retry. 5a) If transaction ID is missing for manual payment, the system displays payment status as pending.
Quality Requirements	1) Order information must be accurate and linked to the correct seller. 2) Payment status should be clearly displayed. 3) Seller should not see other sellers' order data. 4) Order records must be stored securely.

7.19 UC-19: Accept / Reject Rent Request

Field	Description
Use Case No	UC-19
Use Case	Accept / Reject Rent Request
Goal	Allows seller to accept or reject rent requests submitted by buyers/renters.
Preconditions	Seller must be logged in and a rent request must exist for one of the seller's rentable products.

Success End Condition	Rent request status is successfully updated as accepted or rejected.
Failed End Condition	Rent request status cannot be updated due to system error.
Primary Actors	Seller
Trigger	Seller opens the Rent Requests section from Seller Dashboard.
Main Success Flows	1) Seller opens Seller Dashboard. 2) Seller opens Rent Requests section. 3) UniMarket displays pending rent requests. 4) Seller reviews product title, renter name, rental duration, daily rent price, and total rent amount. 5) Seller selects Accept or Reject. 6) Firebase System updates the rent request status. 7) UniMarket notifies the renter about the seller's decision.
Alternative Flows	3a) If no rent request exists, the system displays "No rent requests yet." 5a) If seller cancels the action, the request status remains unchanged. 6a) If status update fails, the system displays an error message and allows retry.
Quality Requirements	1) Rent request status should update accurately. 2) Only the product owner should accept or reject rent requests. 3) Renter should receive notification after seller decision. 4) Rent request information must remain secure and consistent.

7.20 UC-20: Accept / Reject Exchange Request

Field	Description
Use Case No	UC-20
Use Case	Accept / Reject Exchange Request
Goal	Allows seller to accept or reject exchange requests submitted for exchange-only products.
Preconditions	Seller must be logged in and an exchange request must exist for one of the seller's exchange products.

Success End Condition	Exchange request status is successfully updated as accepted or rejected.
Failed End Condition	Exchange request status cannot be updated due to system error.
Primary Actors	Seller
Trigger	Seller opens the Exchange Requests section from Seller Dashboard.
Main Success Flows	1) Seller opens Seller Dashboard. 2) Seller opens Exchange Requests section. 3) UniMarket displays pending exchange requests. 4) Seller reviews requester name, product title, wanted item, and exchange message. 5) Seller selects Accept or Reject. 6) Firebase System updates the exchange request status. 7) UniMarket notifies the exchange requester about the seller's decision. 8) If accepted, buyer and seller may continue coordination through chat.
Alternative Flows	3a) If no exchange request exists, the system displays "No exchange requests yet." 5a) If seller cancels the action, the request status remains unchanged. 6a) If status update fails, the system displays an error message and allows retry. 8a) If the related product is deleted, the system shows "Product unavailable."
Quality Requirements	1) Exchange request status should update correctly. 2) Only the product owner should accept or reject exchange requests. 3) Requester should receive notification after seller decision. 4) Exchange request data must be stored securely and linked to correct product and users.

8. Requirement Traceability Matrix (RTM) – UniMarket

8.1 Selected Core Use Cases

- UC1 – Manage Profile
- UC2 – Search / Filter Product
- UC3 – View Product Details
- UC4 – Add to Wishlist
- UC5 – Add to Cart & Checkout
- UC6 – Make Payment
- UC7 – Send Message
- UC8 – Request Rent
- UC9 – Request Exchange
- UC10 – Manage Seller Dashboard

8.2 RTM Table

Req ID	Requirement Description	UC1	UC2	UC3	UC4	UC5	UC6	UC7	UC8	UC9	UC10
R1	User can create and update profile	◄┘									
R2	User can search and filter products		◄┘								
R3	User can view complete product details			◄┘							
R4	User can add products to wishlist				◄┘						
R5	User can add products to cart and checkout					◄┘					
R6	User can make payment for orders						◄┘				
R7	Buyer and seller can communicate through chat							◄┘			

R8	Buyer can request product rental								◄┘		
R9	Buyer can request product exchange									◄┘	
R10	Seller can manage product listings										◄┘
R11	Seller can manage orders and sold items										◄┘
R12	Seller can manage rent and exchange requests										◄┘
R13	User can receive important notifications										◄┘
R14	System can upload profile, product, and chat media							◄┘			◄┘
R15	System restricts access to authenticated NSTU students	◄┘									

9. Requirement Prioritization

9.1 Requirement Prioritization Table

Important–Urgent Matrix

Req ID	Requirement Description	Important	Urgent	Priority	Remarks
R1	User can create and update profile	Yes	Yes	High	Needed for user identity management
R2	User can search products by keyword	Yes	Yes	High	Core marketplace browsing feature

R3	User can filter products	Yes	Yes	High	Helps users find products quickly
R4	User can view product details	Yes	Yes	High	Essential before buying, renting, or exchanging
R5	User can add products to wishlist	Yes	No	Medium	Useful for saving favorite items
R6	User can add products to cart	Yes	Yes	High	Required for purchase flow
R7	User can checkout products	Yes	Yes	High	Main order placement process
R8	User can make payment	Yes	Yes	High	Required to complete purchase
R9	Buyer and seller can send messages	Yes	Yes	High	Essential for negotiation and coordination
R10	Buyer can request rent	Yes	No	Medium	Important for rent system
R11	Buyer can request exchange	Yes	No	Medium	Important for exchange system
R12	Buyer can give rating and review	Yes	No	Medium	Improves trust and transparency
R13	User can view notifications	Yes	No	Medium	Helps users track updates
R14	Seller can post products	Yes	Yes	High	Core seller functionality
R15	Seller can edit product listings	Yes	Yes	High	Needed for listing maintenance
R16	Seller can delete product listings	Yes	Yes	High	Needed to remove unavailable products
R17	Seller can manage dashboard	Yes	Yes	High	Central control panel for sellers

R18	Seller can view orders and sold items	Yes	No	Medium	Needed for sales tracking
R19	Seller can accept/reject rent requests	Yes	No	Medium	Required for rent workflow
R20	Seller can accept/reject exchange requests	Yes	No	Medium	Required for exchange workflow
R21	System can upload media using Cloudinary	Yes	Yes	High	Needed for product, profile, and chat media
R22	System can store and sync data using Firebase	Yes	Yes	High	Core backend requirement
R23	System can send notifications	Yes	No	Medium	Improves user engagement and awareness
R24	System restricts access to NSTU students	Yes	Yes	High	Important for university-only marketplace
R25	Admin can manage users, products, reports, and analytics	Yes	No	Medium	Important for moderation and safety

10. Activity Diagram

An activity diagram represents the workflow of a system or process. It shows the sequence of activities, decision points, branching paths, initial state, and final state. In UniMarket, activity diagrams are used to describe important workflows such as profile management, product posting, searching, checkout, payment, rent request, exchange request, and seller dashboard management. All diagrams are created by using PlantUML.

10.1 Activity Diagram of Manage Profile

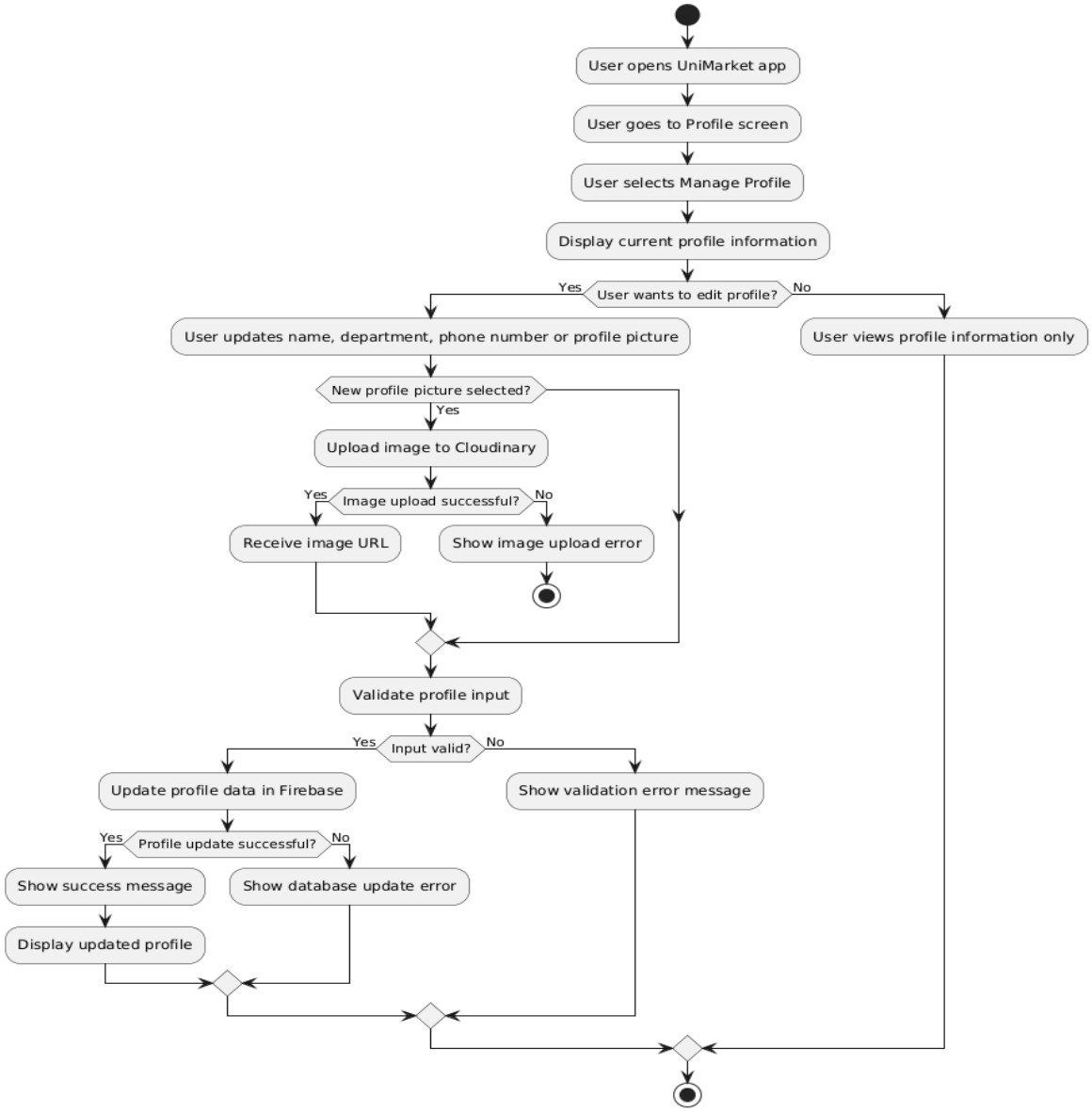


Figure 1: Activity Diagram of Manage Profile

10.2 Activity Diagram of Search and Filter Product

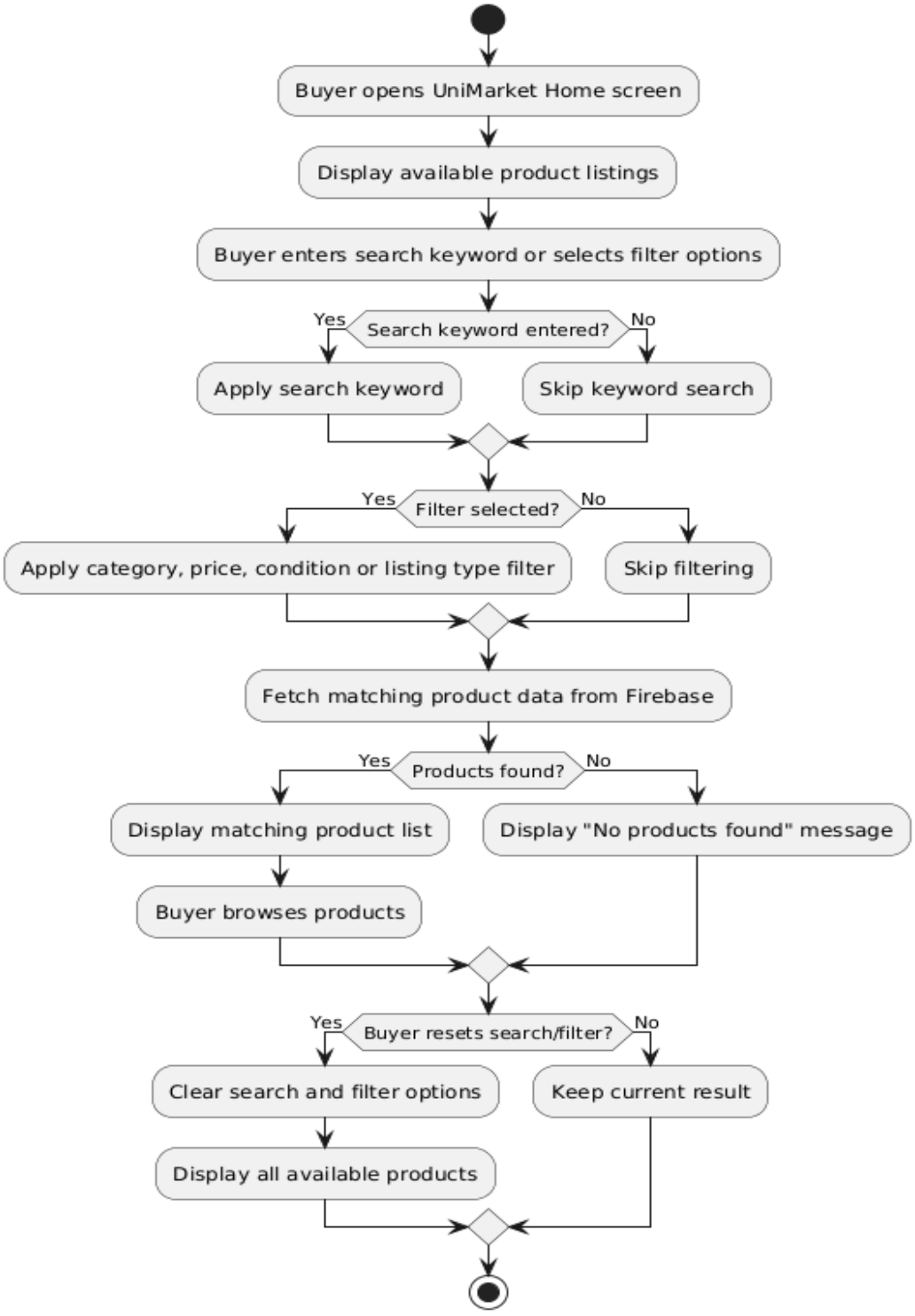


Figure 2:Activity Diagram of Search and Filter Product

10.3 Activity Diagram of View Product Details

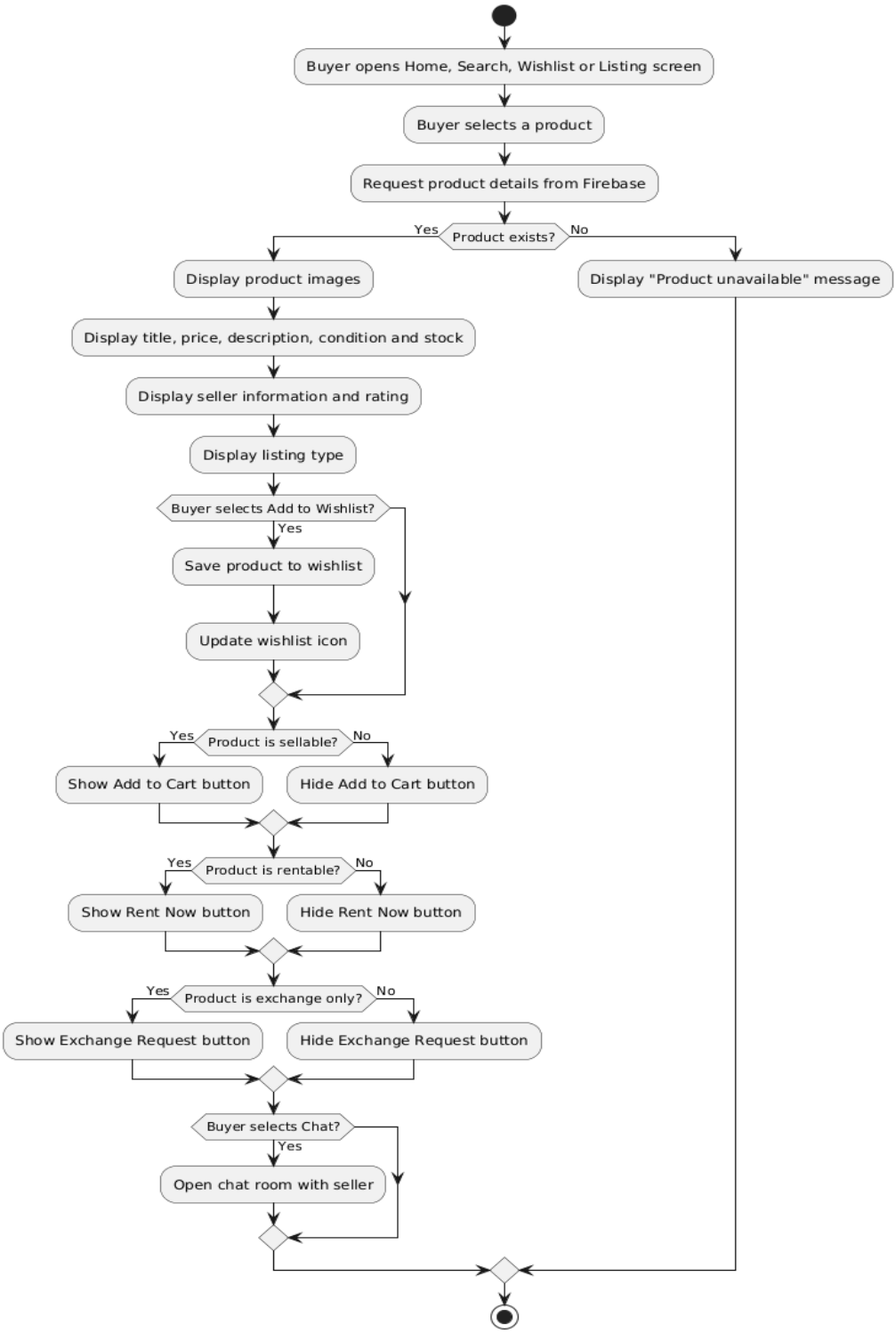


Figure 3: Activity Diagram of View Product Details

10.4 Activity Diagram of Post Product

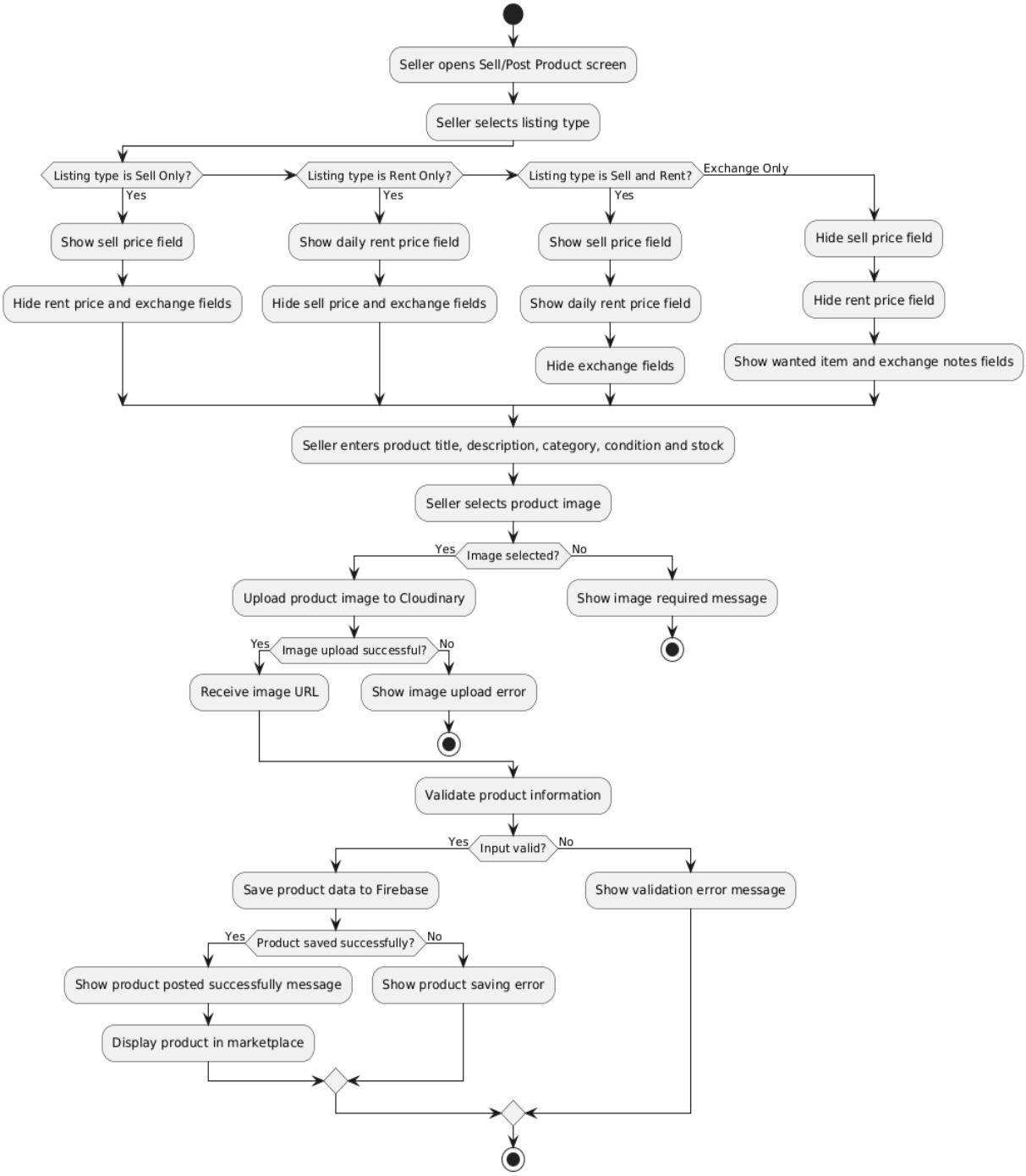


Figure 4:Activity Diagram of Post Product

10.5 Activity Diagram of Add to Cart and Checkout

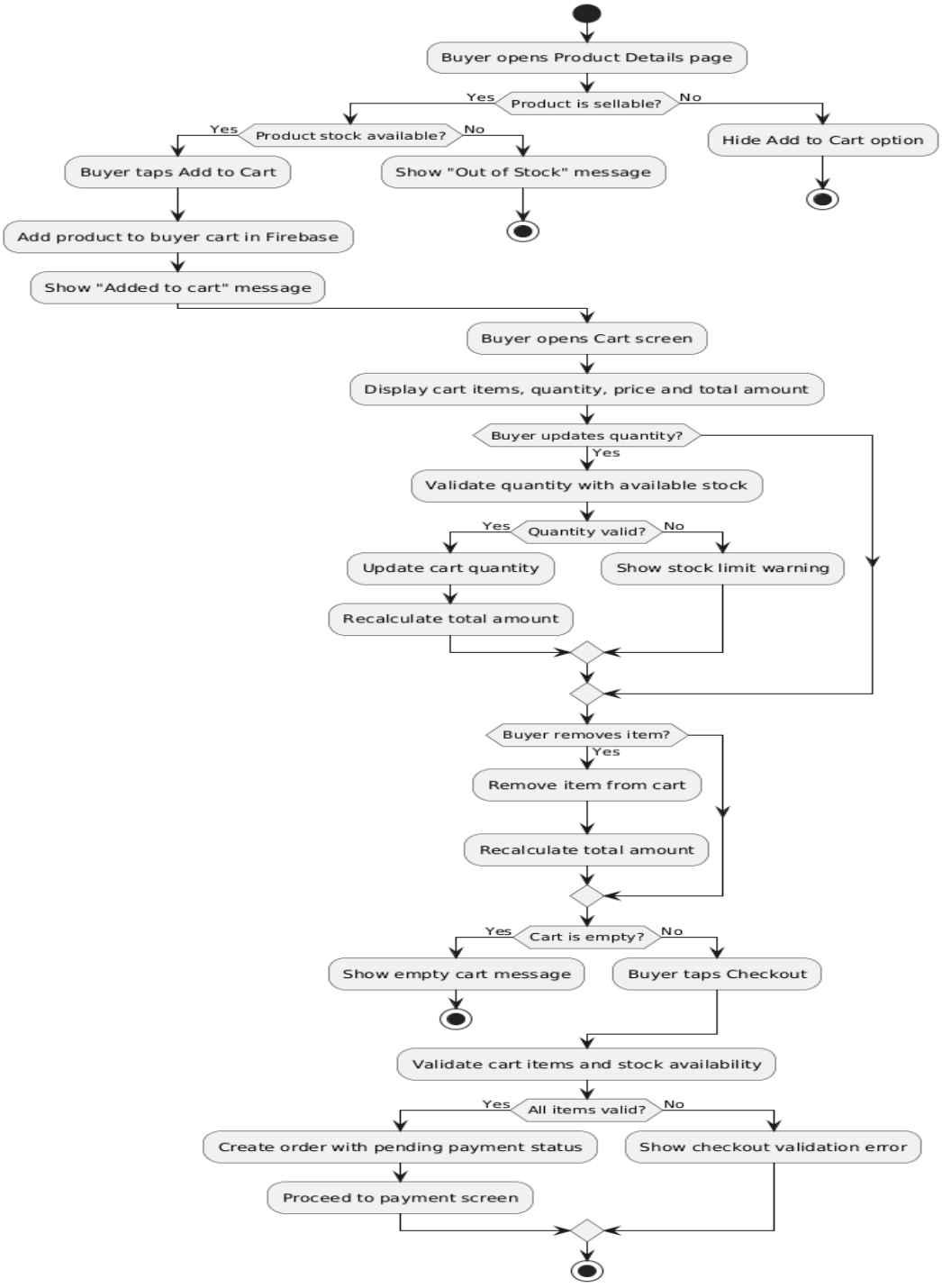


Figure 5:Activity Diagram of Add to Cart and Checkout

10.6 Activity Diagram of Make Payment

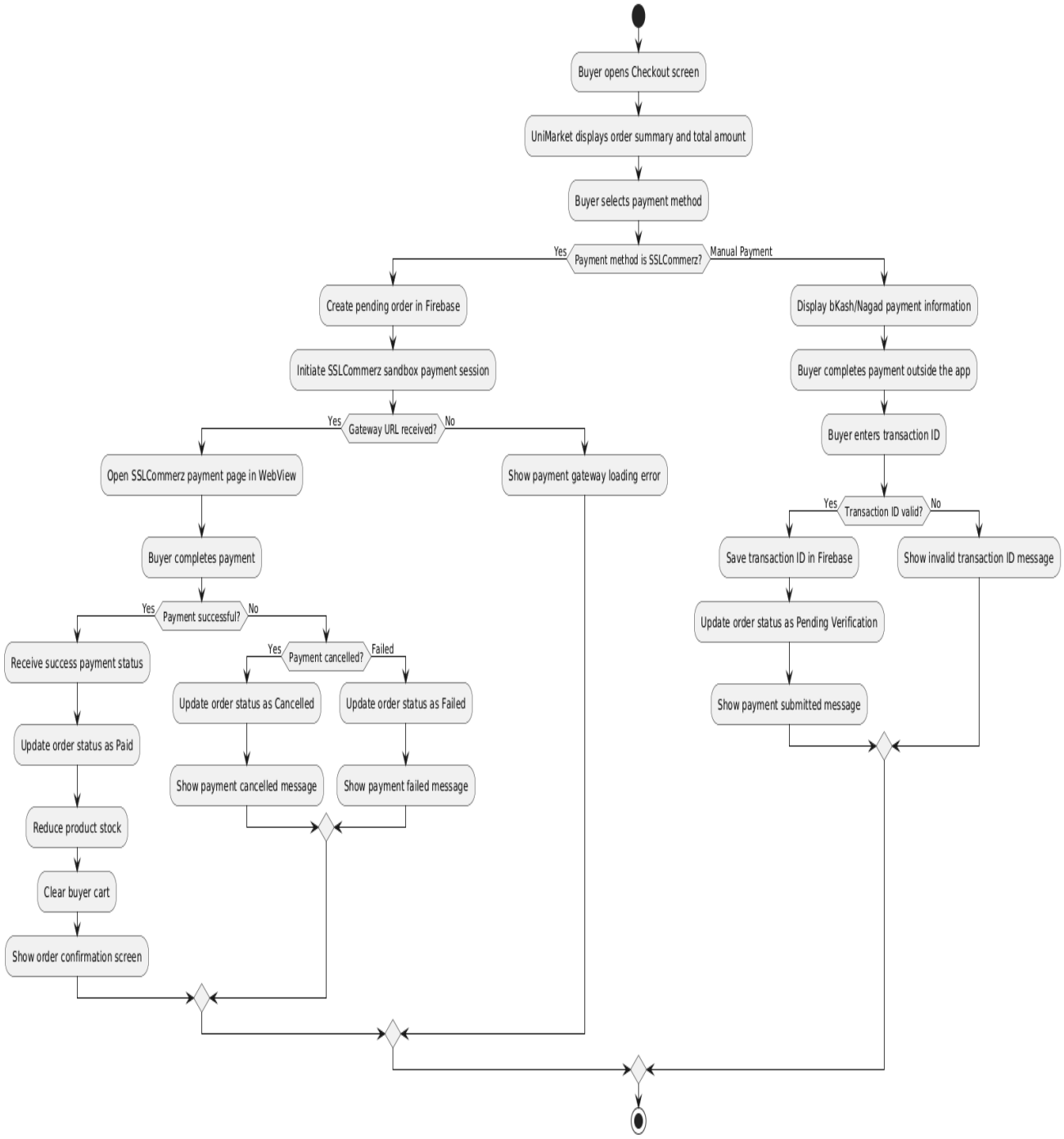


Figure 6:Activity Diagram of Make Payment

10.7 Activity Diagram of Send Message

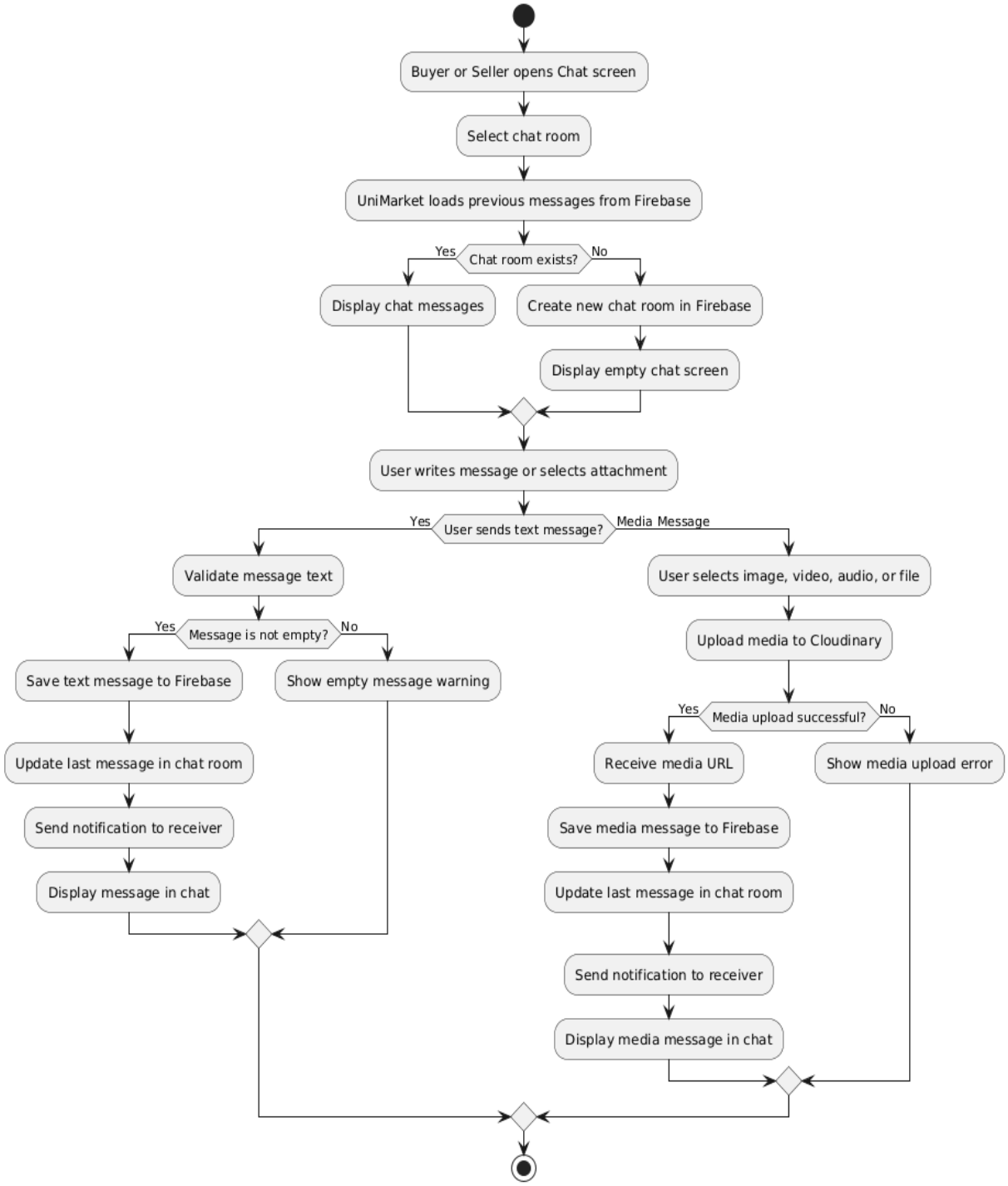


Figure 7: Activity Diagram of Send Message

10.8 Activity Diagram of Request Rent

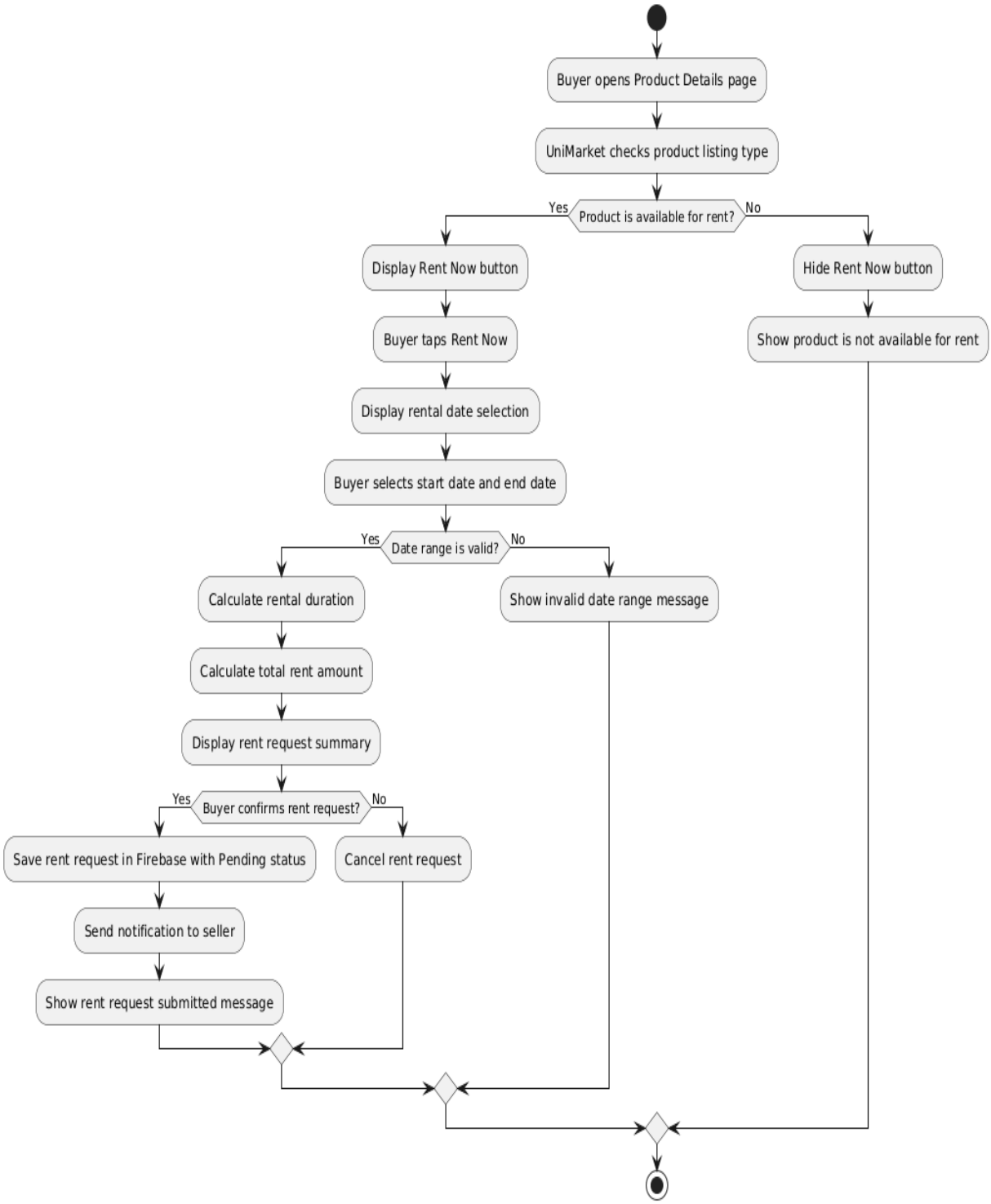


Figure 8:Activity Diagram of Request Rent

10.9 Activity Diagram of Request Exchange



Figure 9:Activity Diagram of Request Exchange

10.10 Activity Diagram of Manage Seller Dashboard

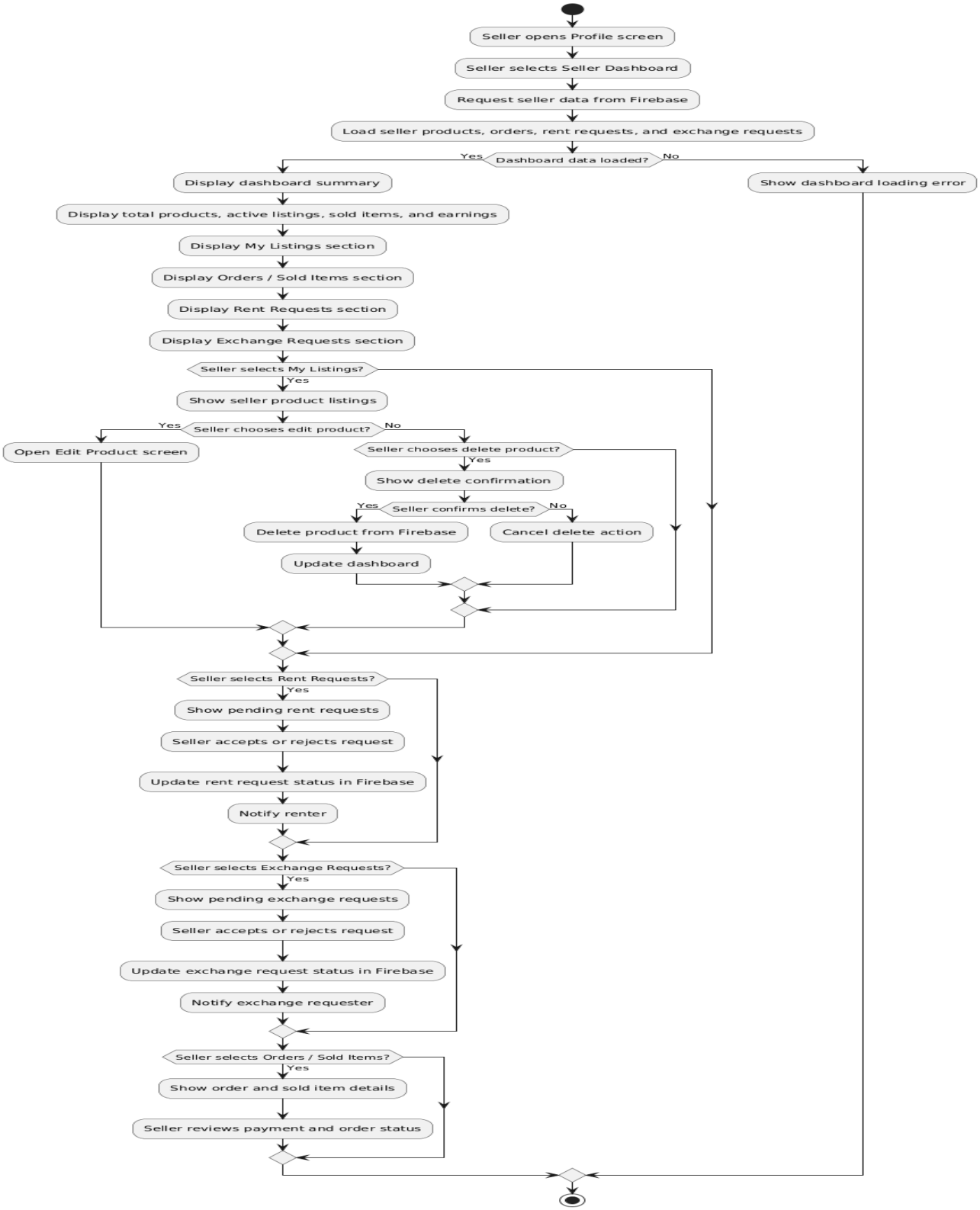


Figure 10:Activity Diagram of Manage Seller Dashboard

11. Sequence Diagram

A Sequence Diagram shows the time-based interaction between actors and system components. The flow of messages is arranged top to bottom using arrows. It is used to represent the dynamic behavior of a system. All sequence diagrams are created using PlantUML.

11.1 Sequence Diagram of Manage Profile

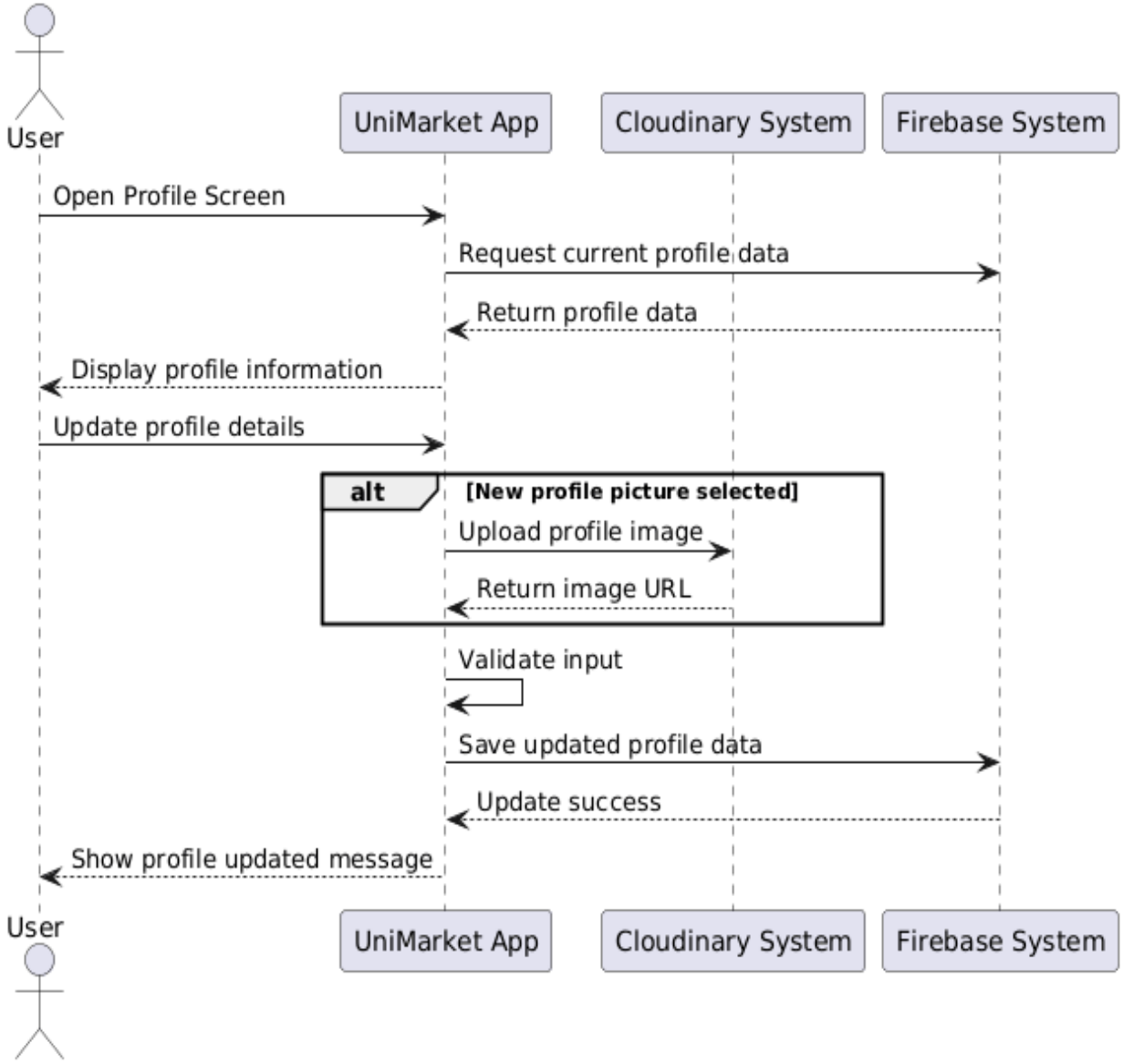


Figure 11: Sequence Diagram of Manage Profile

11.2 Sequence Diagram of Search and Filter Product

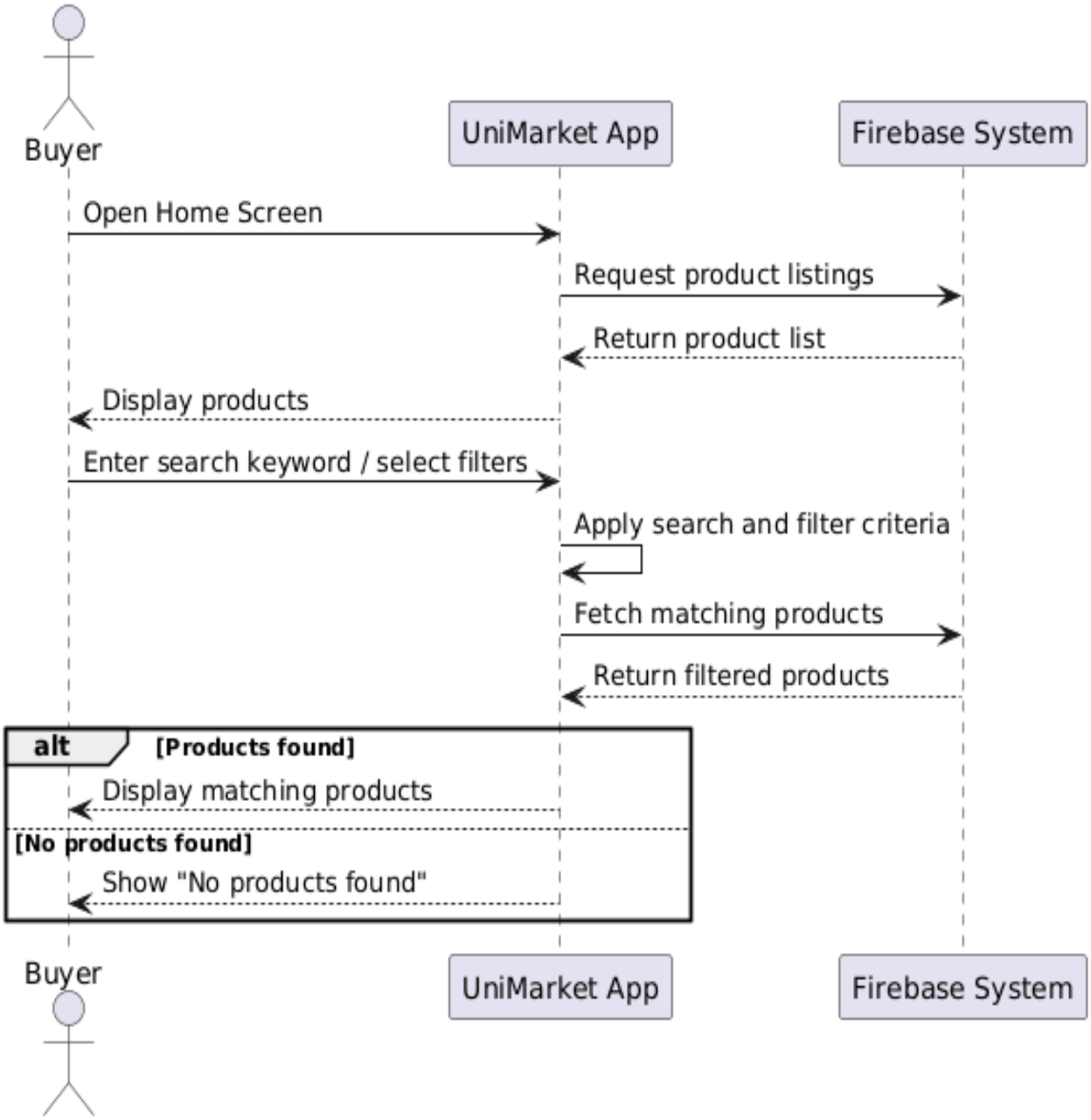


Figure 12:Sequence Diagram of Search and Filter Product

11.3 Sequence Diagram of View Product Details

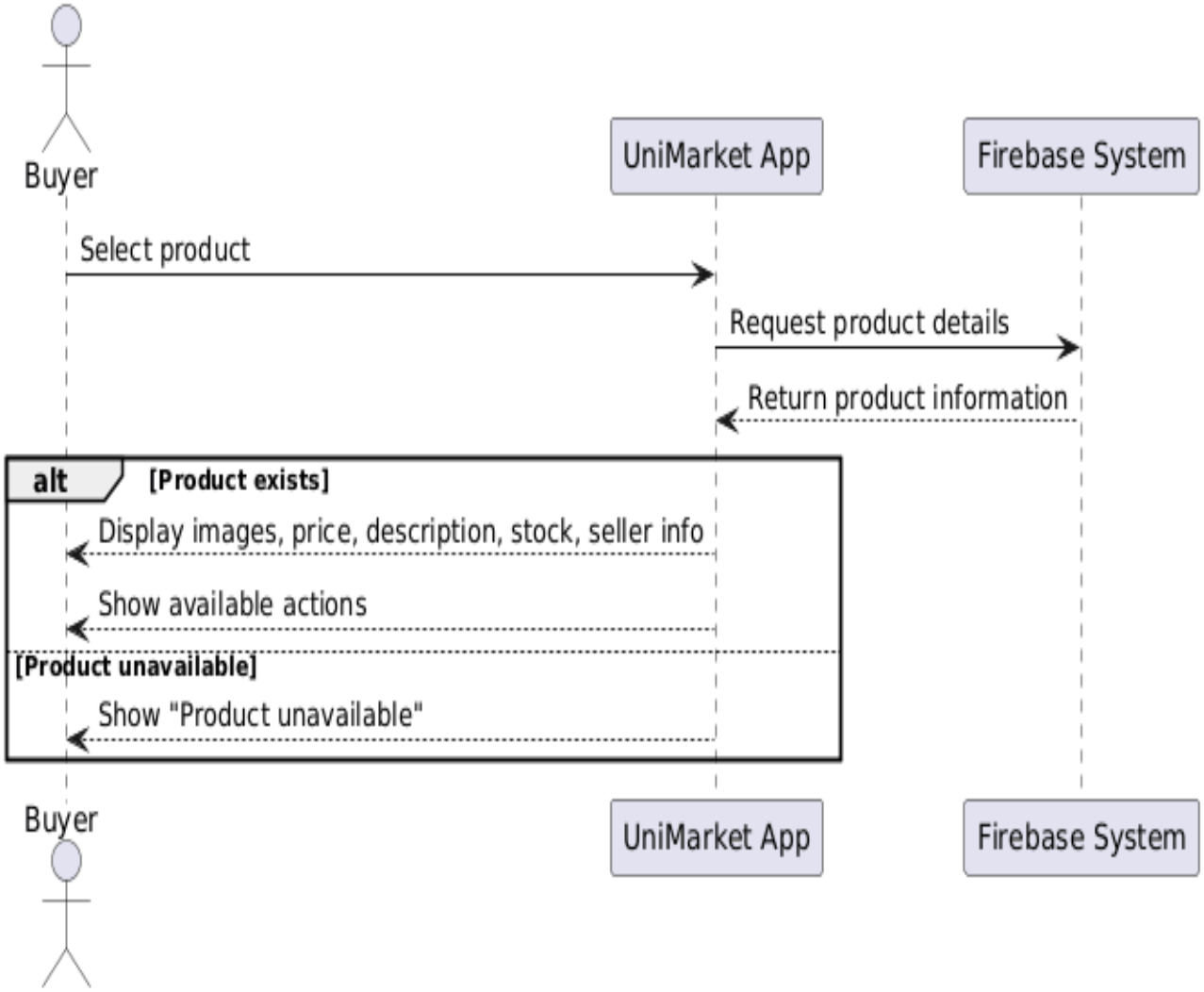


Figure 13: Sequence Diagram of View Product Details

11.4 Sequence Diagram of Post Product

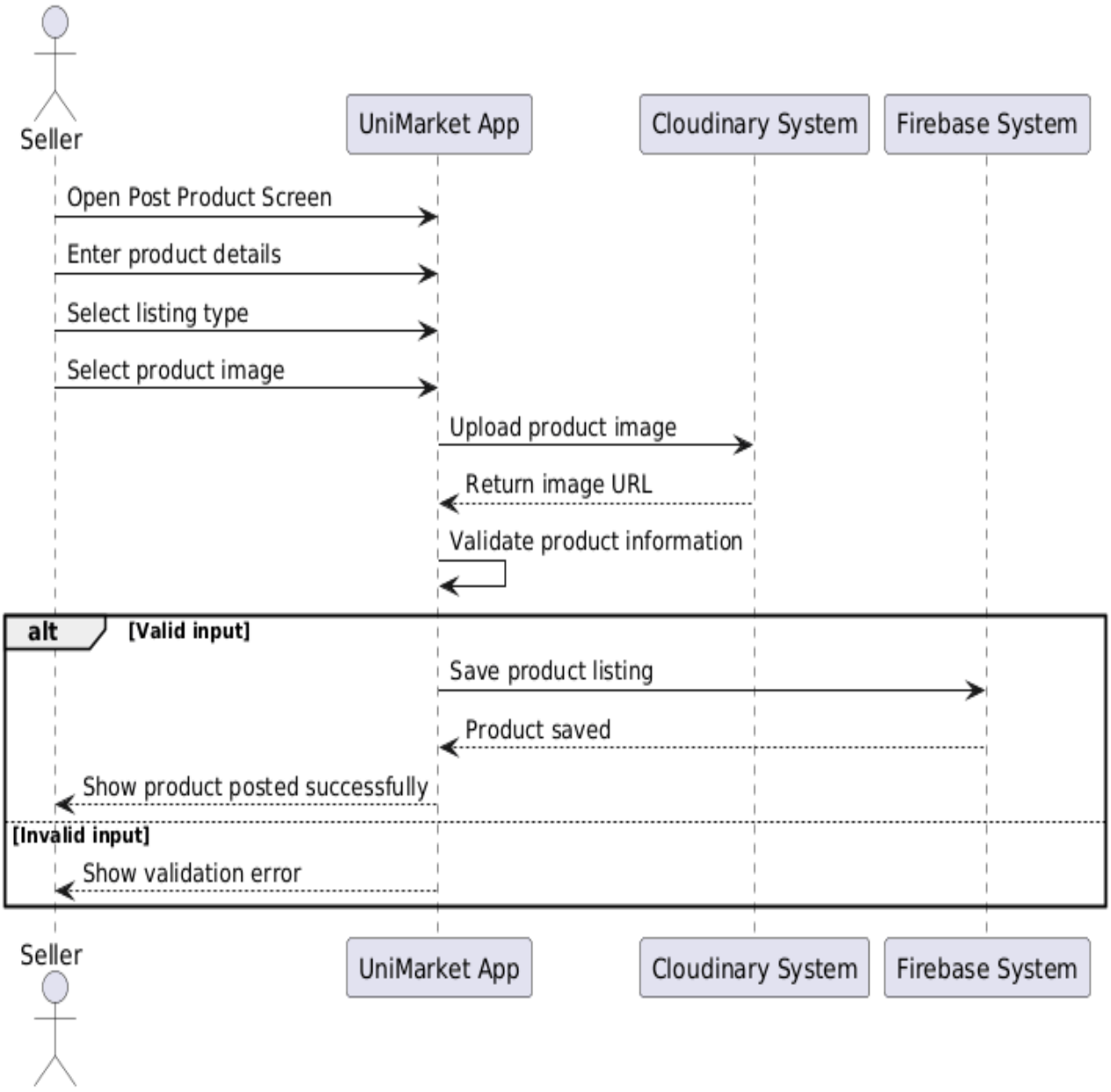


Figure 14:Sequence Diagram of Post Product

11.5 Sequence Diagram of Add to Cart and Checkout

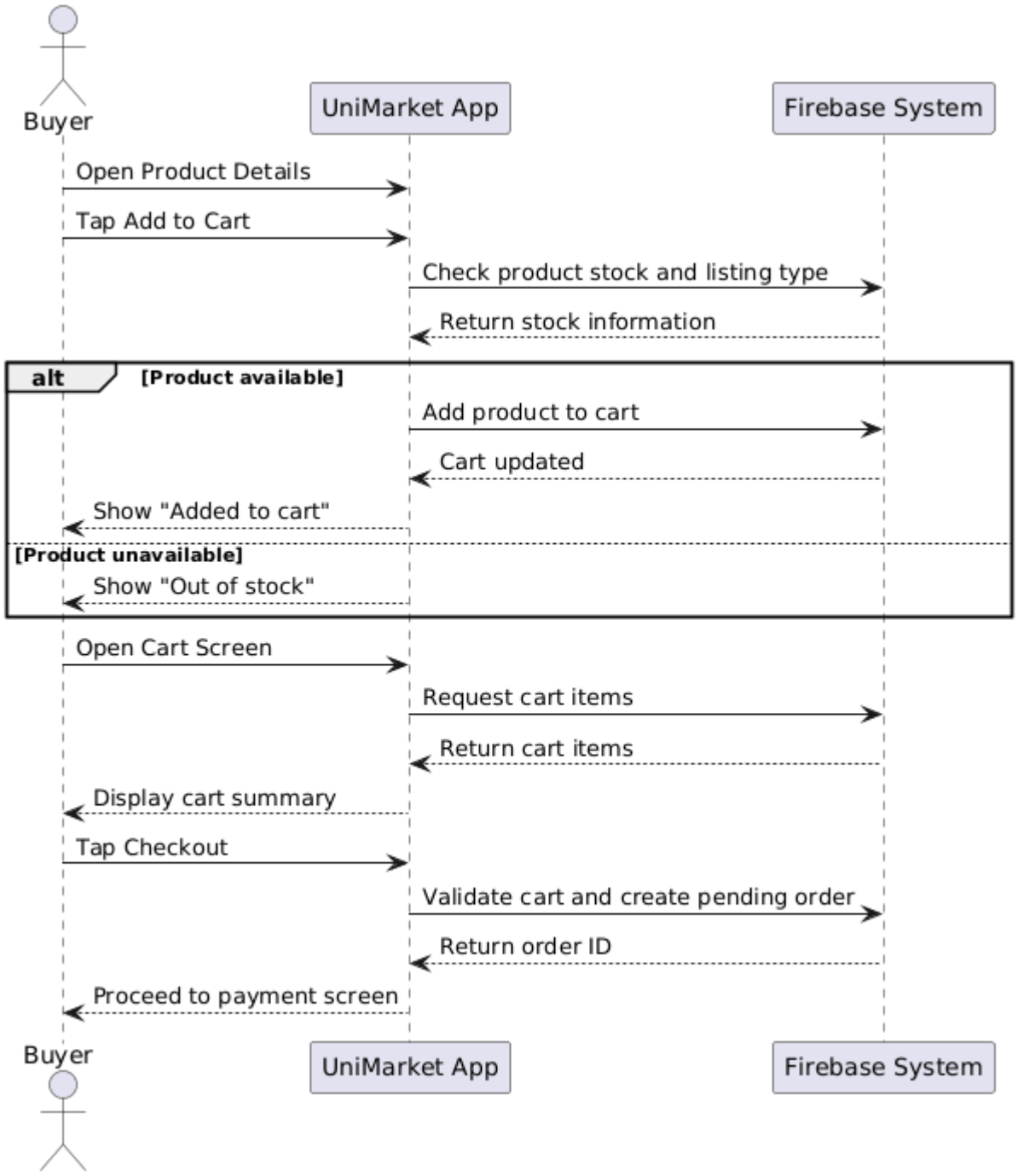


Figure 15:Sequence Diagram of Add to Cart and Checkout

11.6 Sequence Diagram of Make Payment

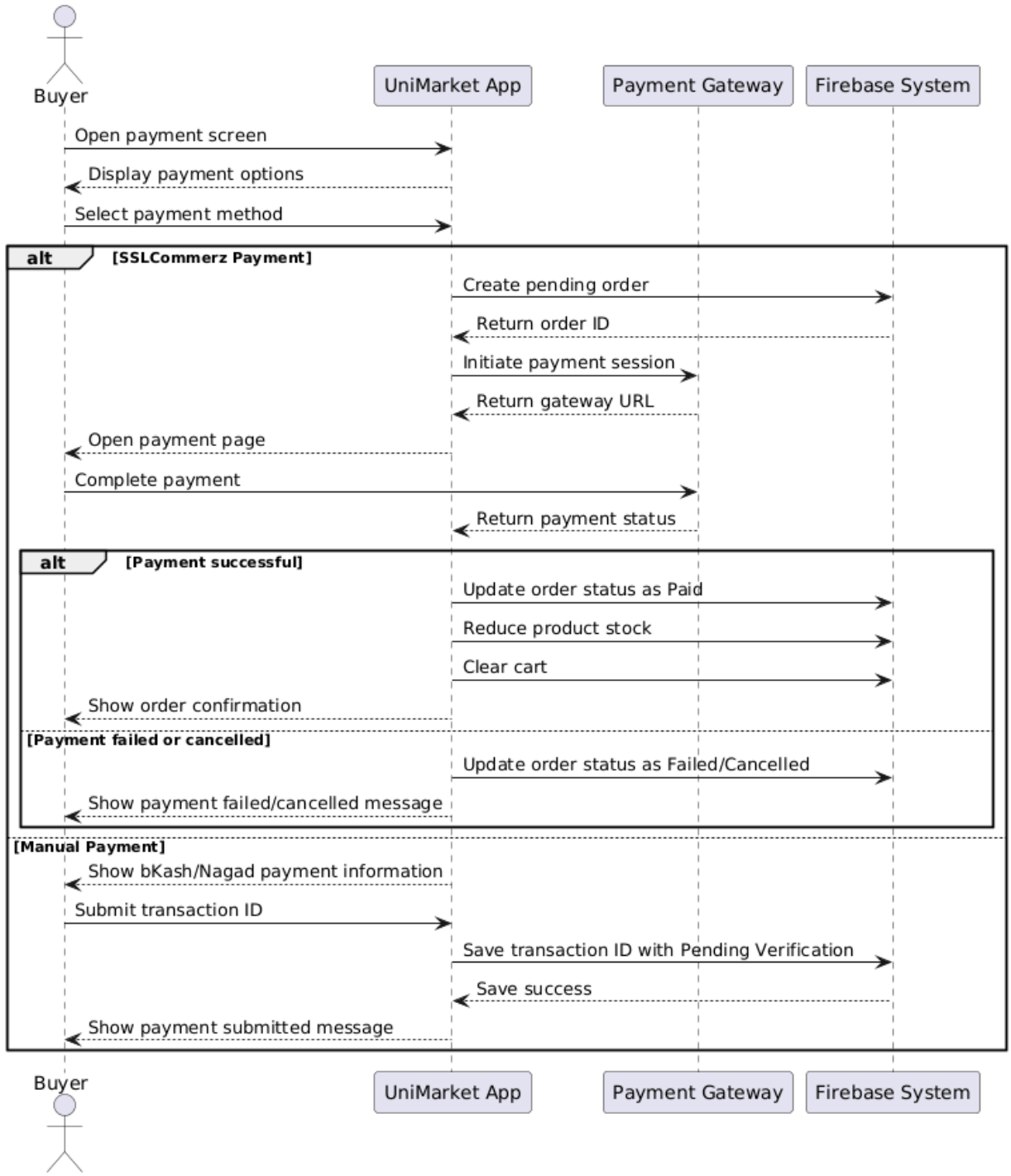


Figure 16:Sequence Diagram of Make Payment

11.7 Sequence Diagram of Send Message

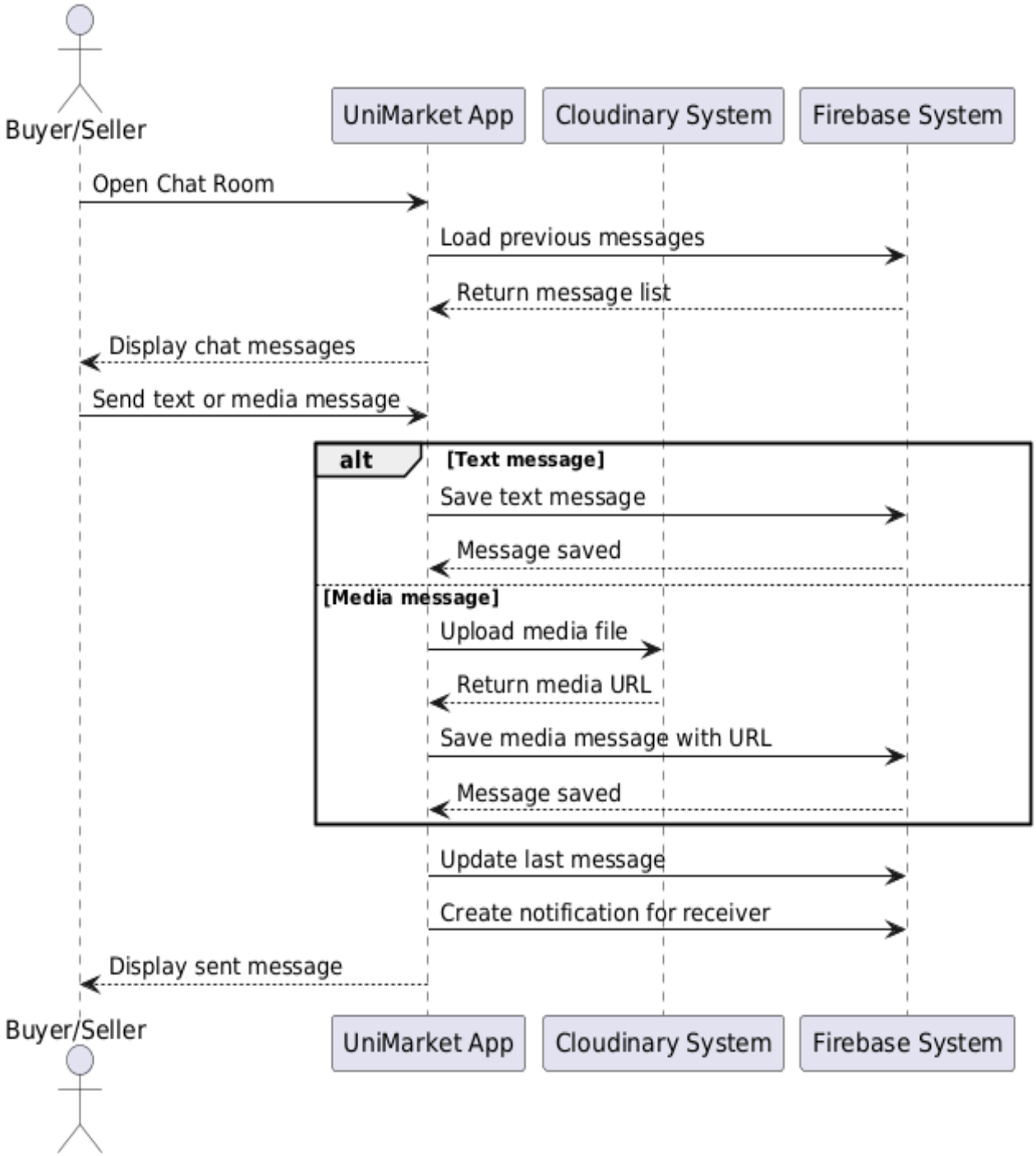


Figure 17:Sequence Diagram of Send Message

11.8 Sequence Diagram of Request Rent

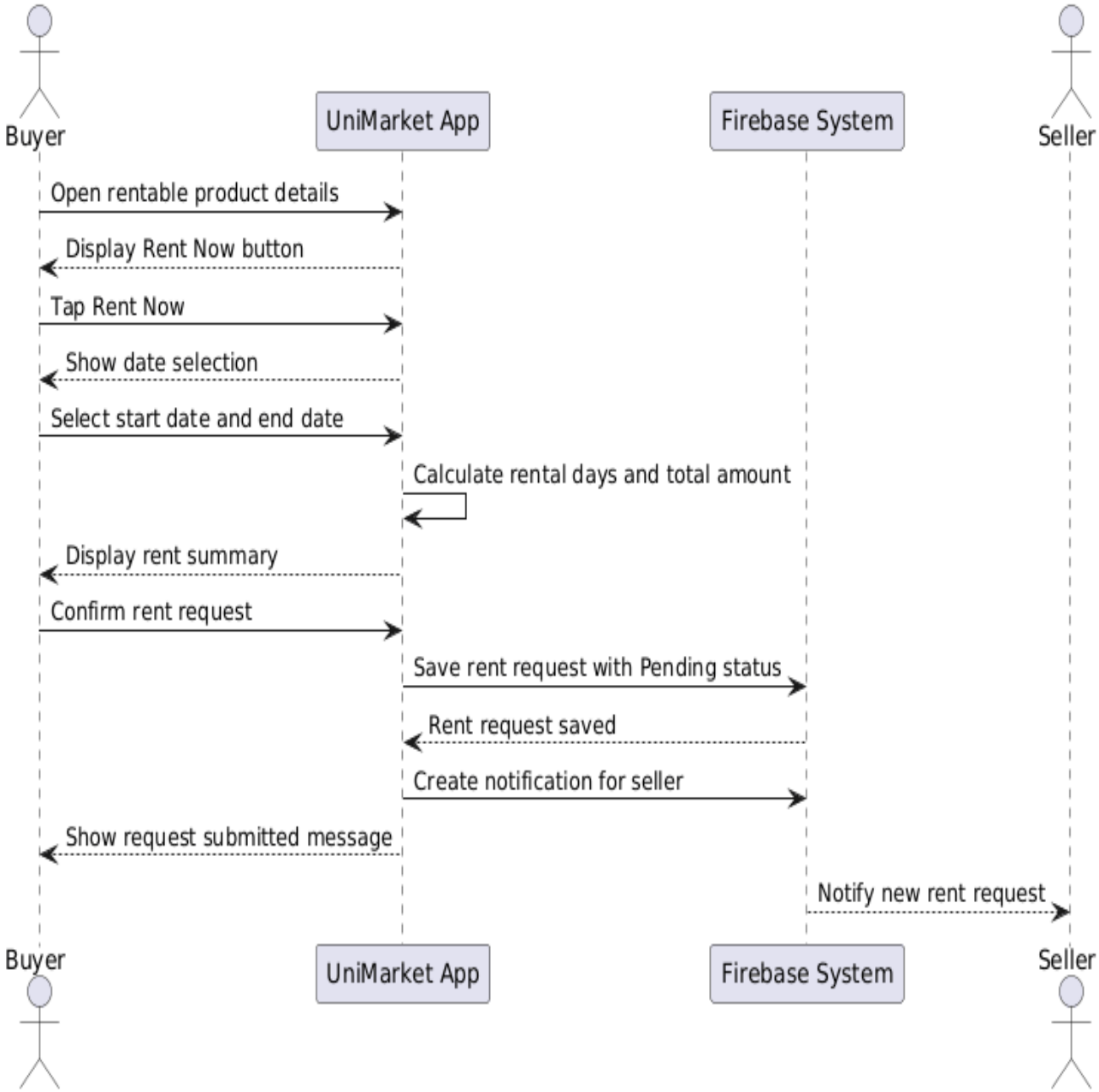


Figure 18:Sequence Diagram of Request Rent

11.9 Sequence Diagram of Request Exchange

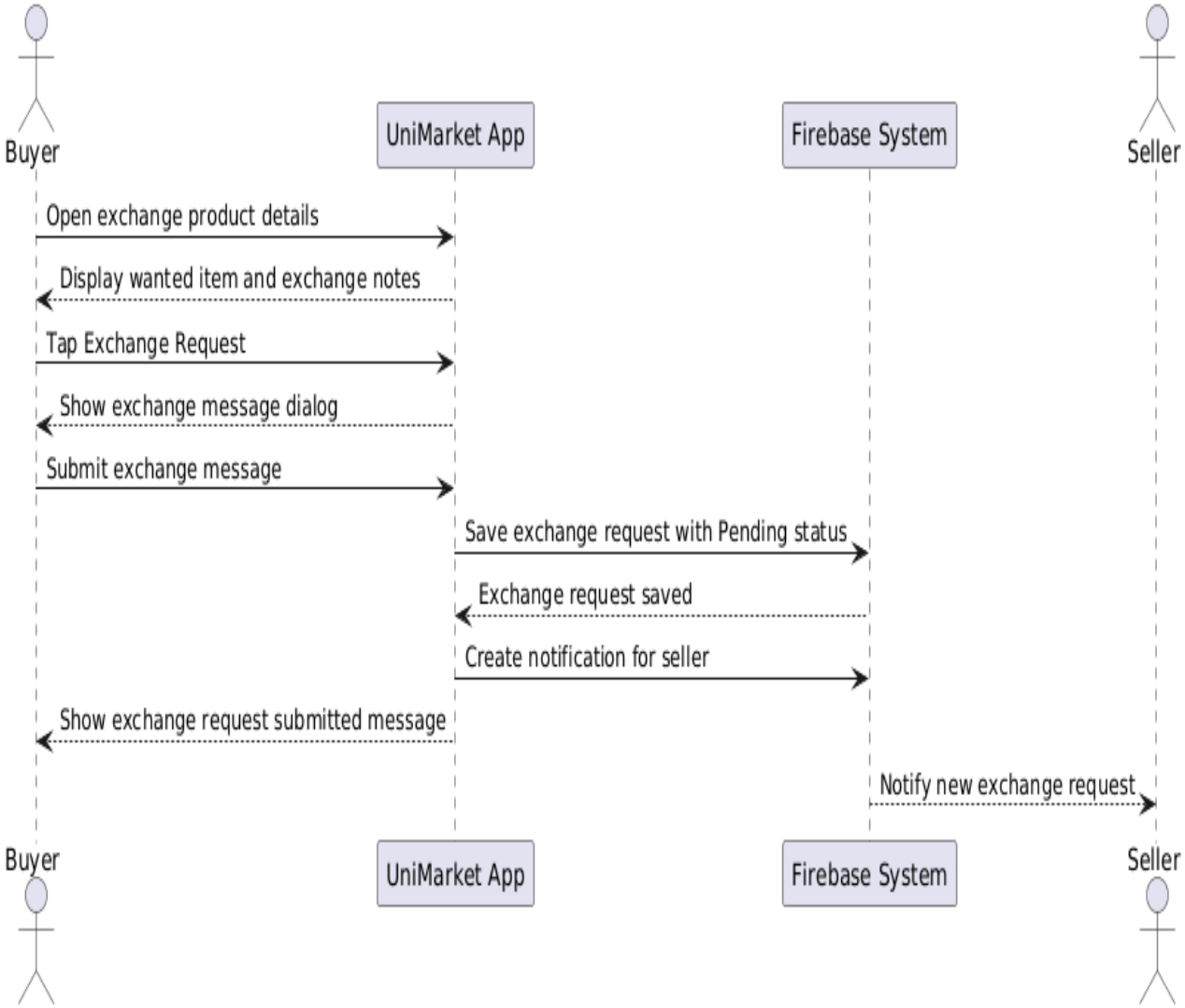


Figure 19: Sequence Diagram of Request Exchange

11.10 Sequence Diagram of Manage Seller Dashboard

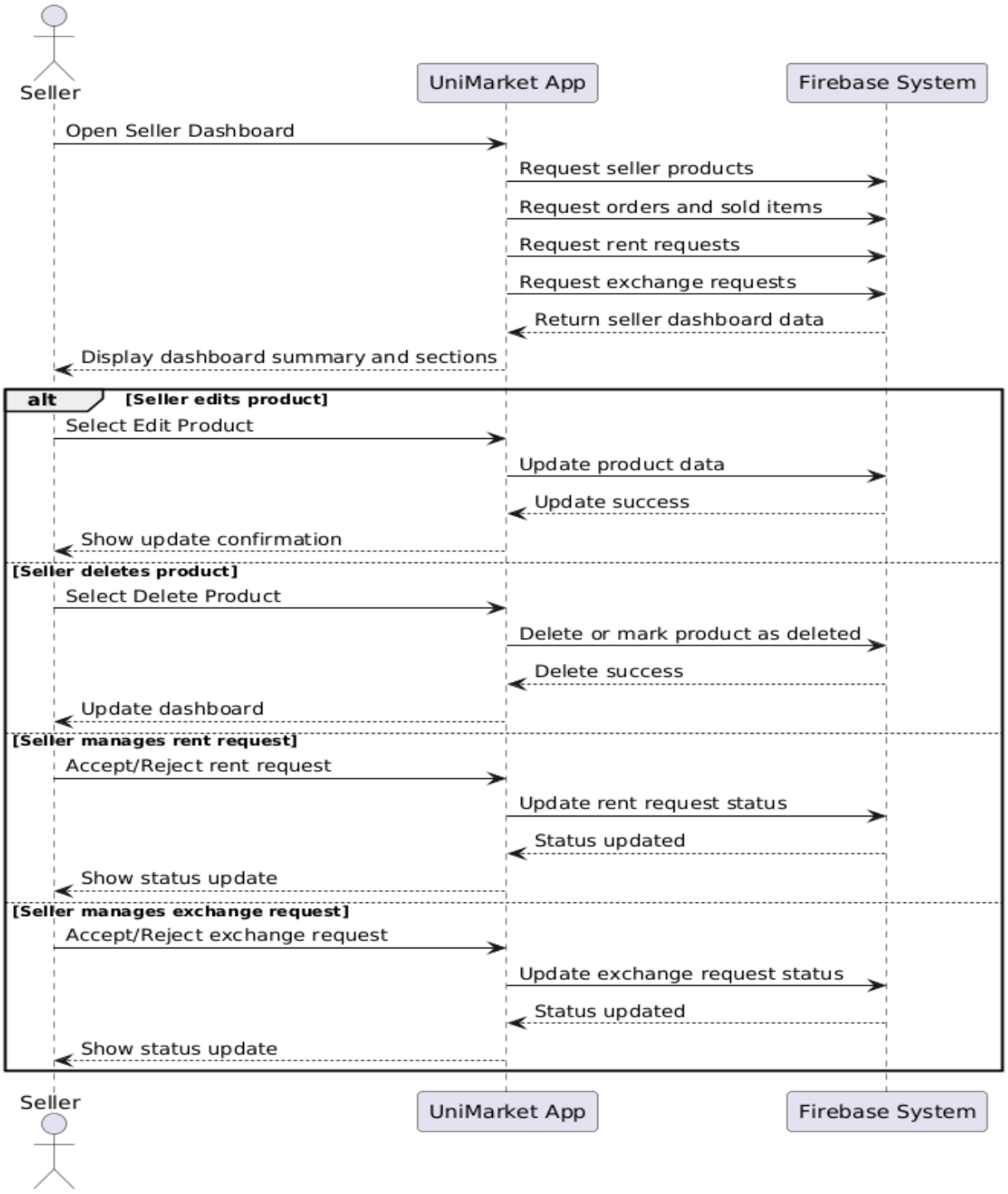


Figure 20:Sequence Diagram of Manage Seller Dashboard

12. Swimlane Diagram

A Swimlane Diagram is a type of activity diagram that separates activities into lanes based on actors or roles. Each swimlane shows who performs which action and when, clearly defining responsibilities. It is useful for understanding workflows involving multiple participants.
All swimlane diagrams are created using PlantUML.

12.1 Swimlane Diagram of Manage Profile

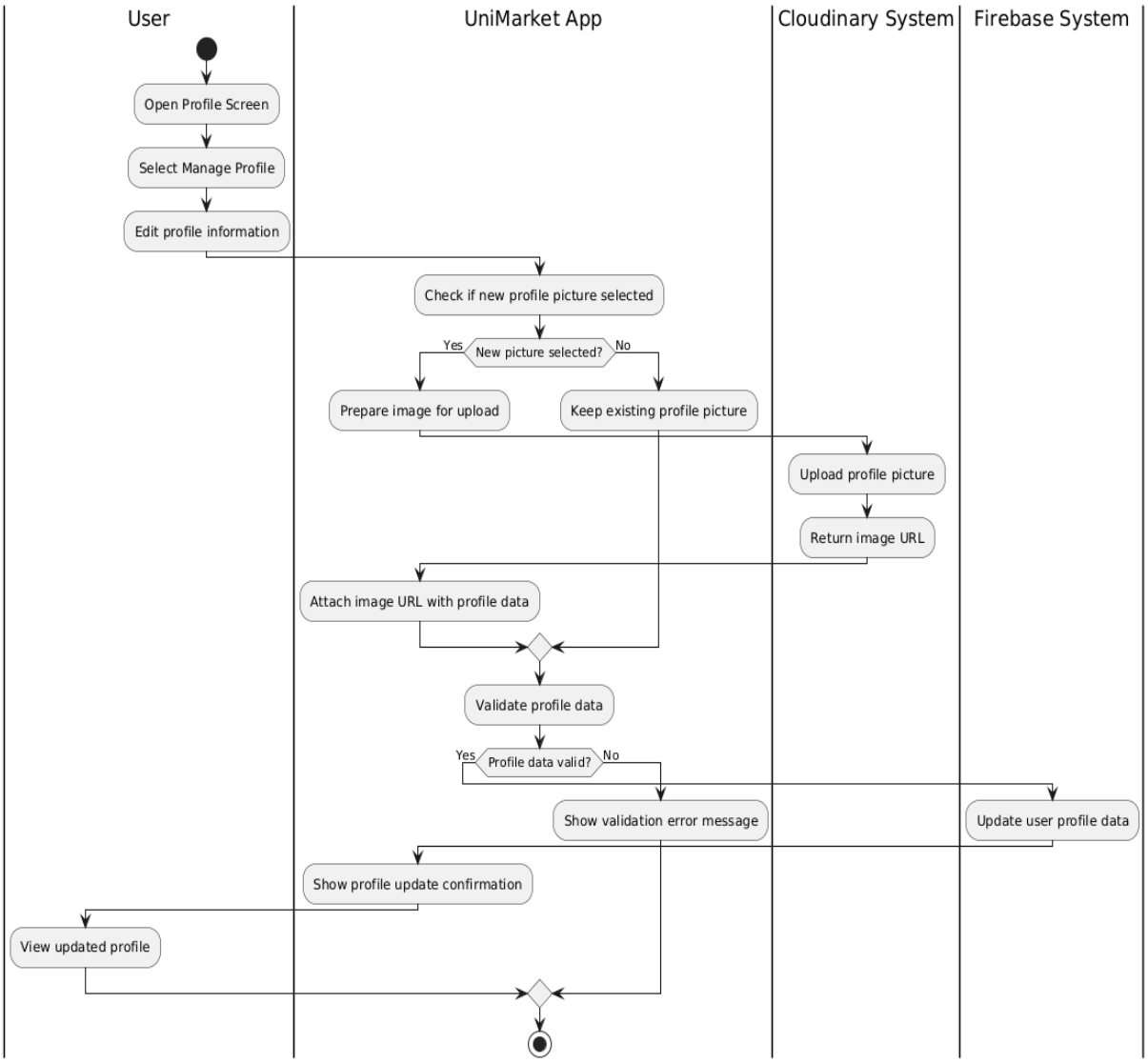


Figure 21: Swimlane Diagram of Manage Profile

12.2 Swimlane Diagram of Search and Filter Product

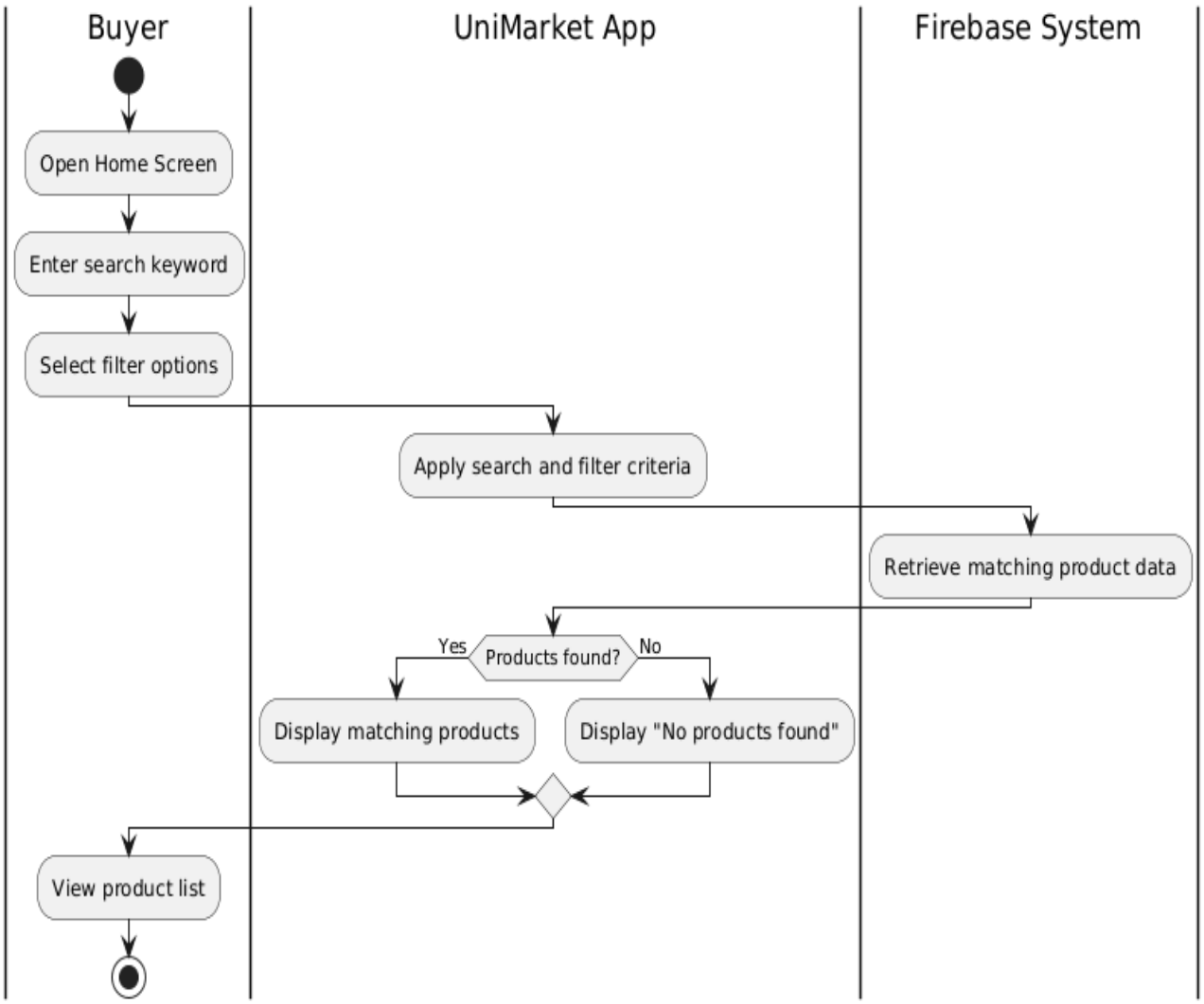


Figure 22: Swimlane Diagram of Search and Filter Product

12.3 Swimlane Diagram of View Product Details

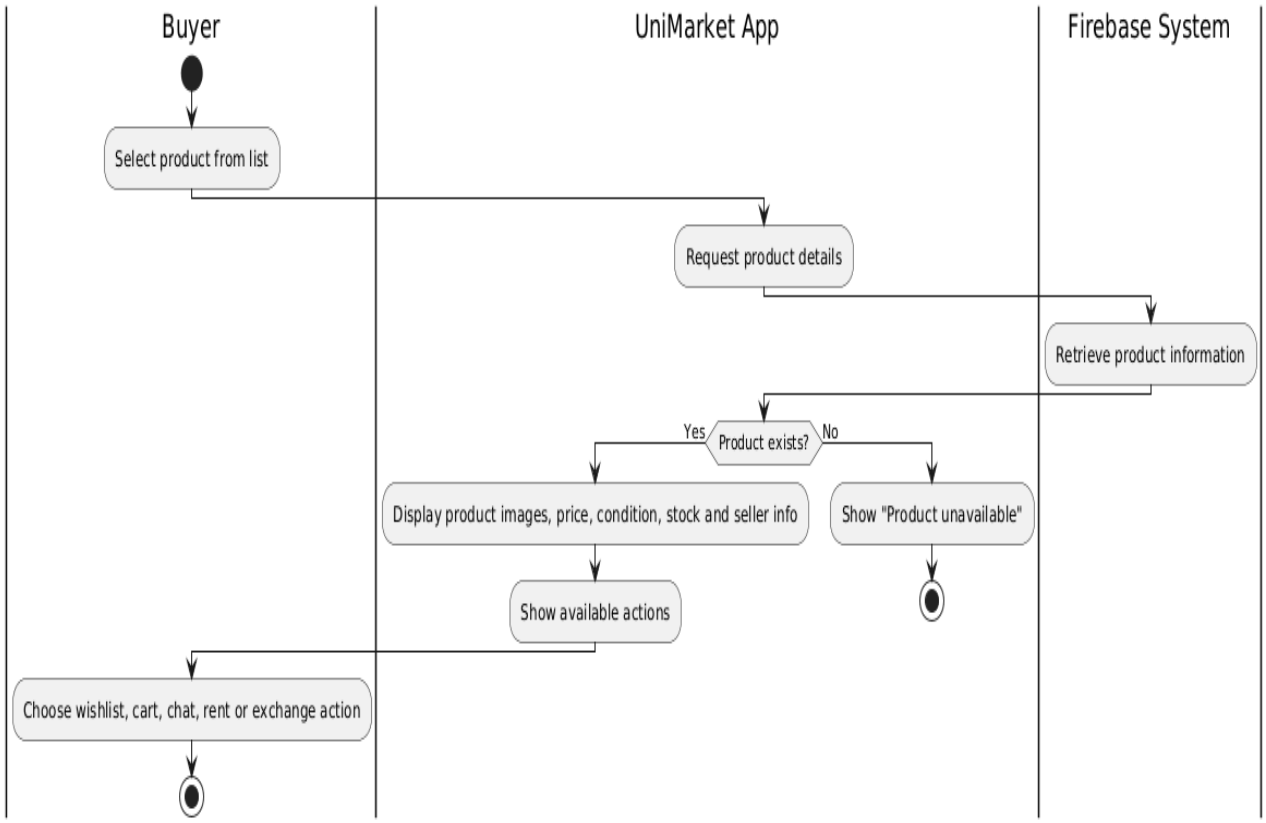


Figure 23:Swimlane Diagram of View Product Details

12.4 Swimlane Diagram of Post Product

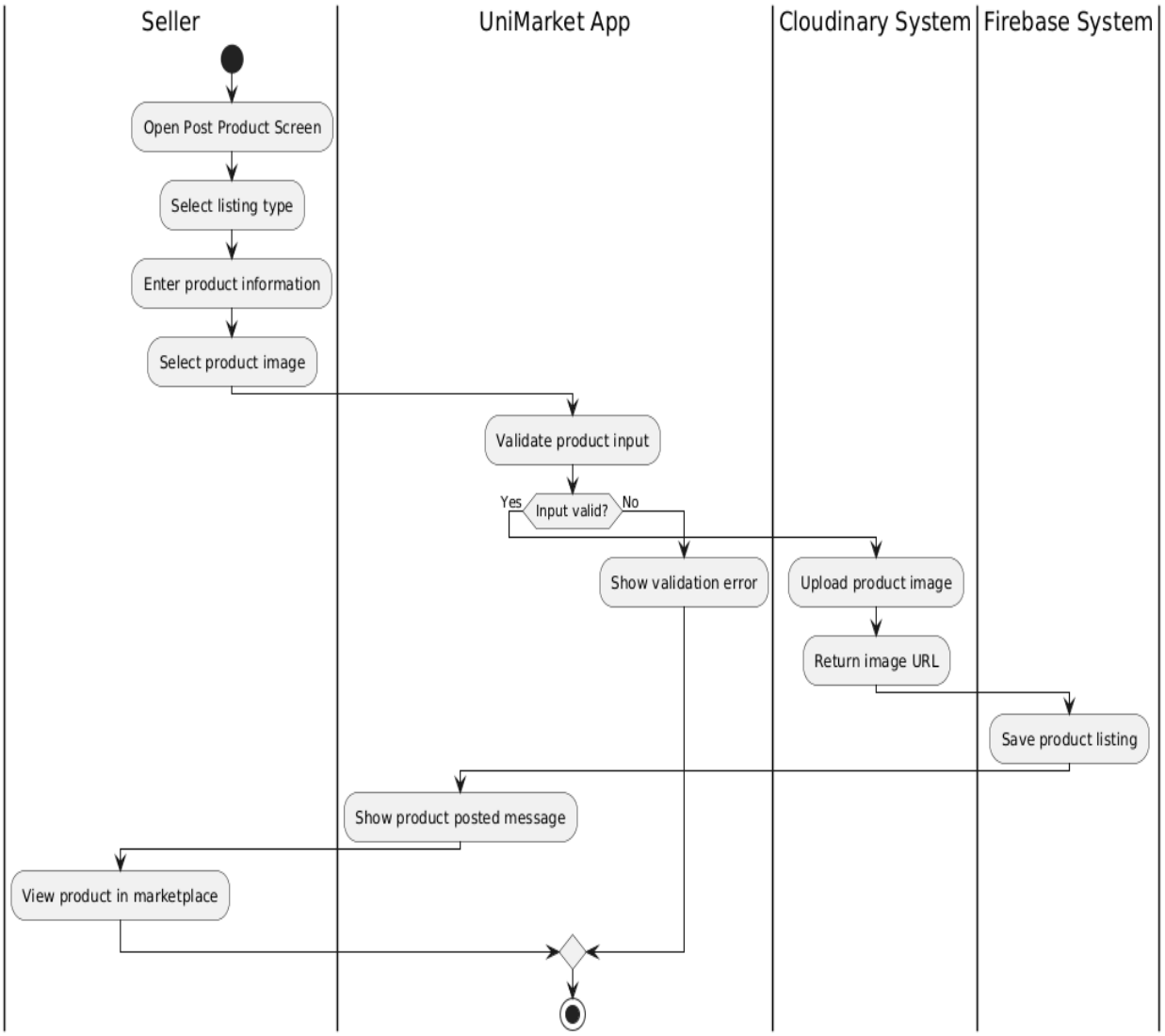


Figure 24:Swimlane Diagram of Post Product

12.5 Swimlane Diagram of Add to Cart and Checkout

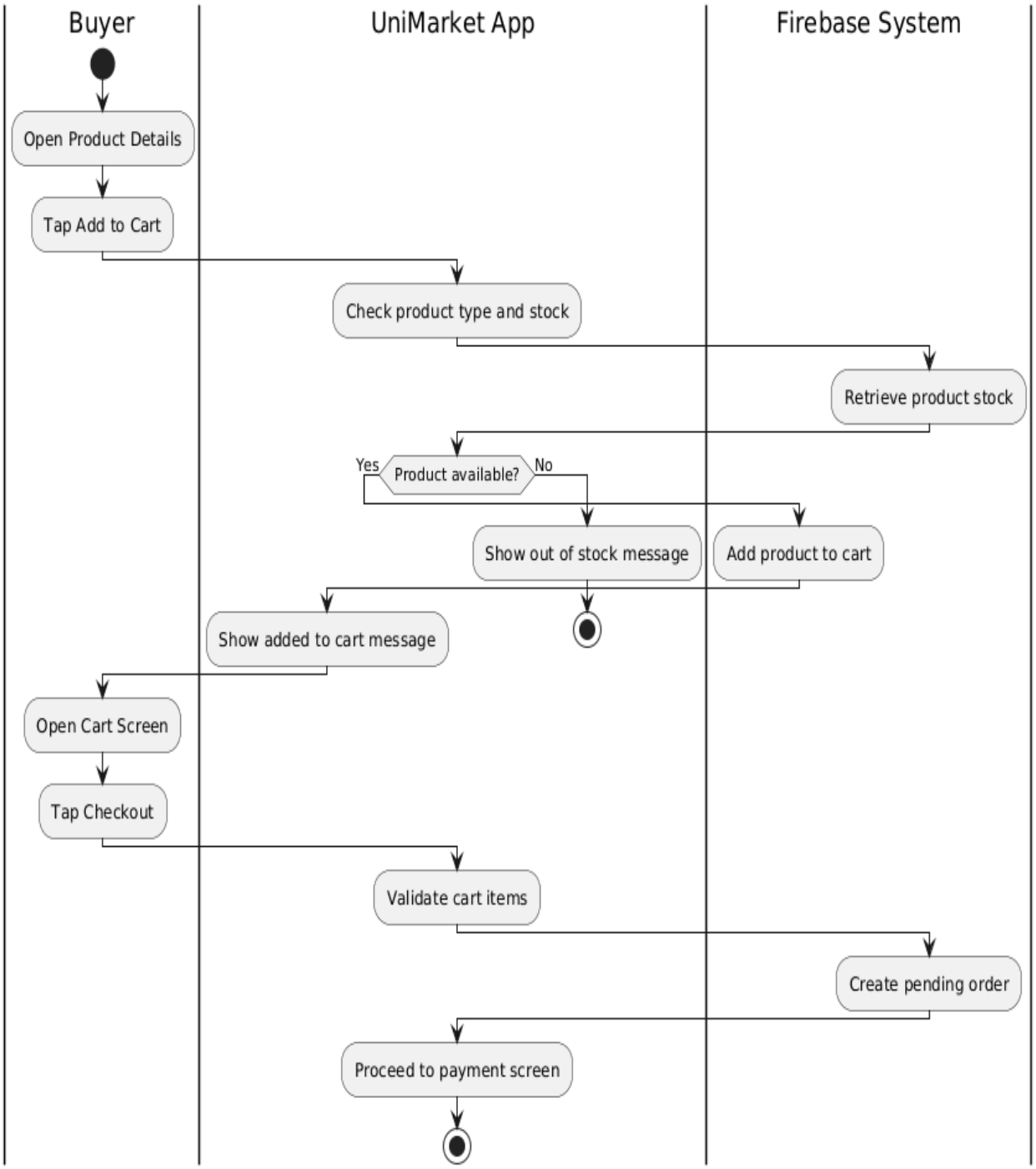


Figure 25:Swimlane Diagram of Add to Cart and Checkout

12.6 Swimlane Diagram of Make Payment

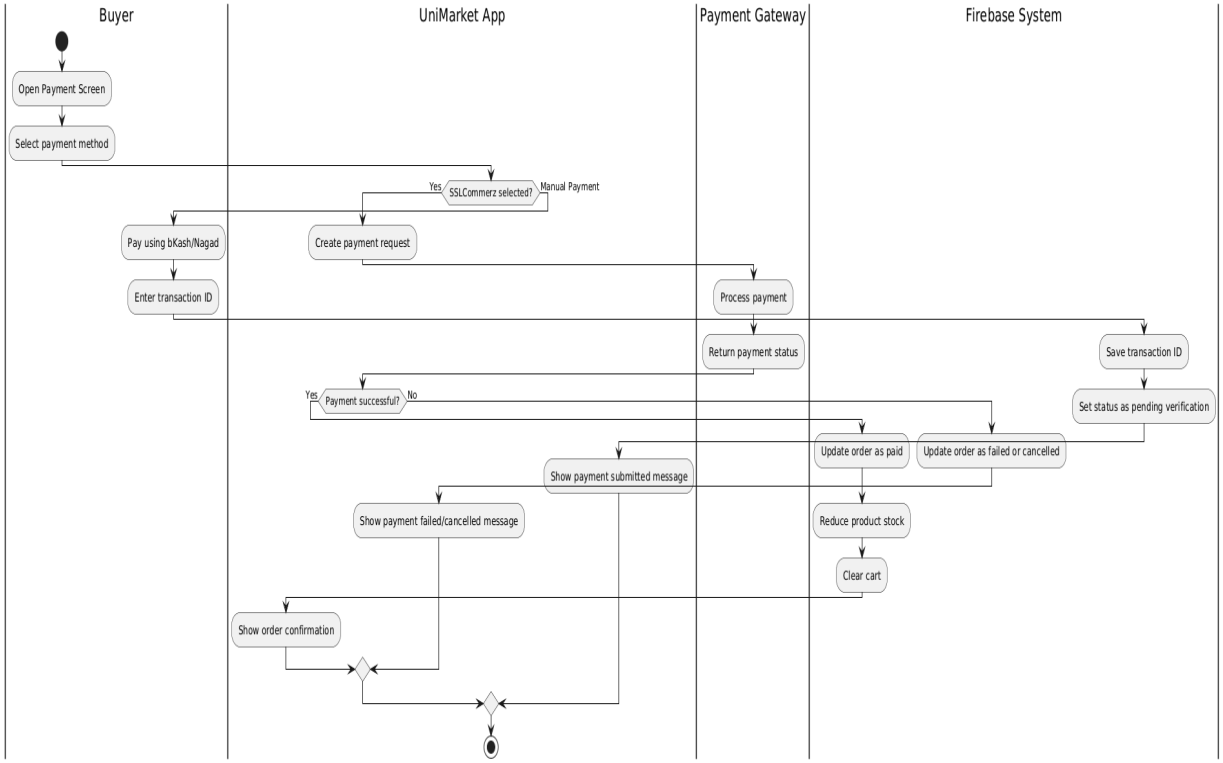


Figure 26: Swimlane Diagram of Make Payment

12.7 Swimlane Diagram of Send Message

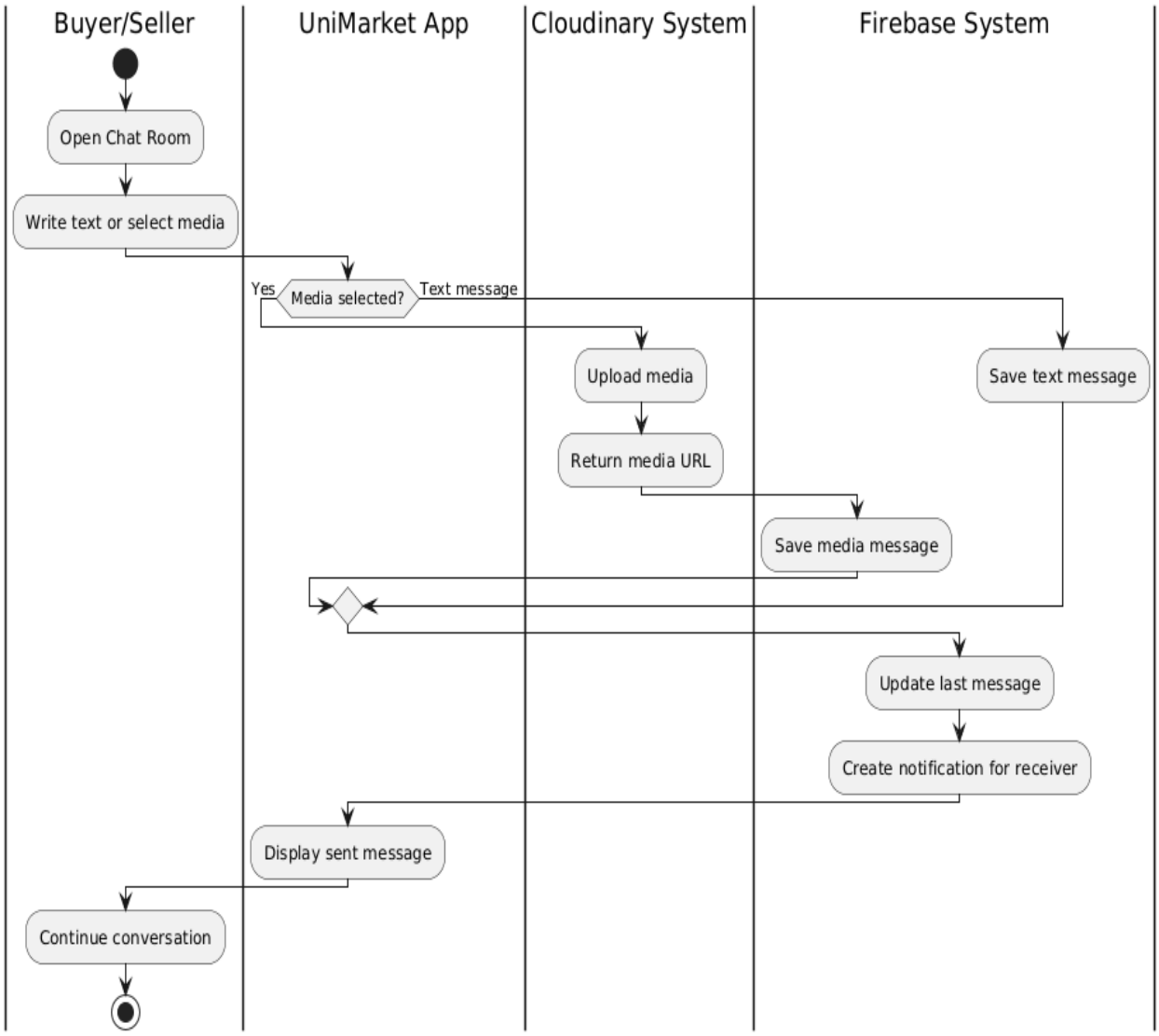


Figure 27:Swimlane Diagram of Send Message

12.8 Swimlane Diagram of Request Rent

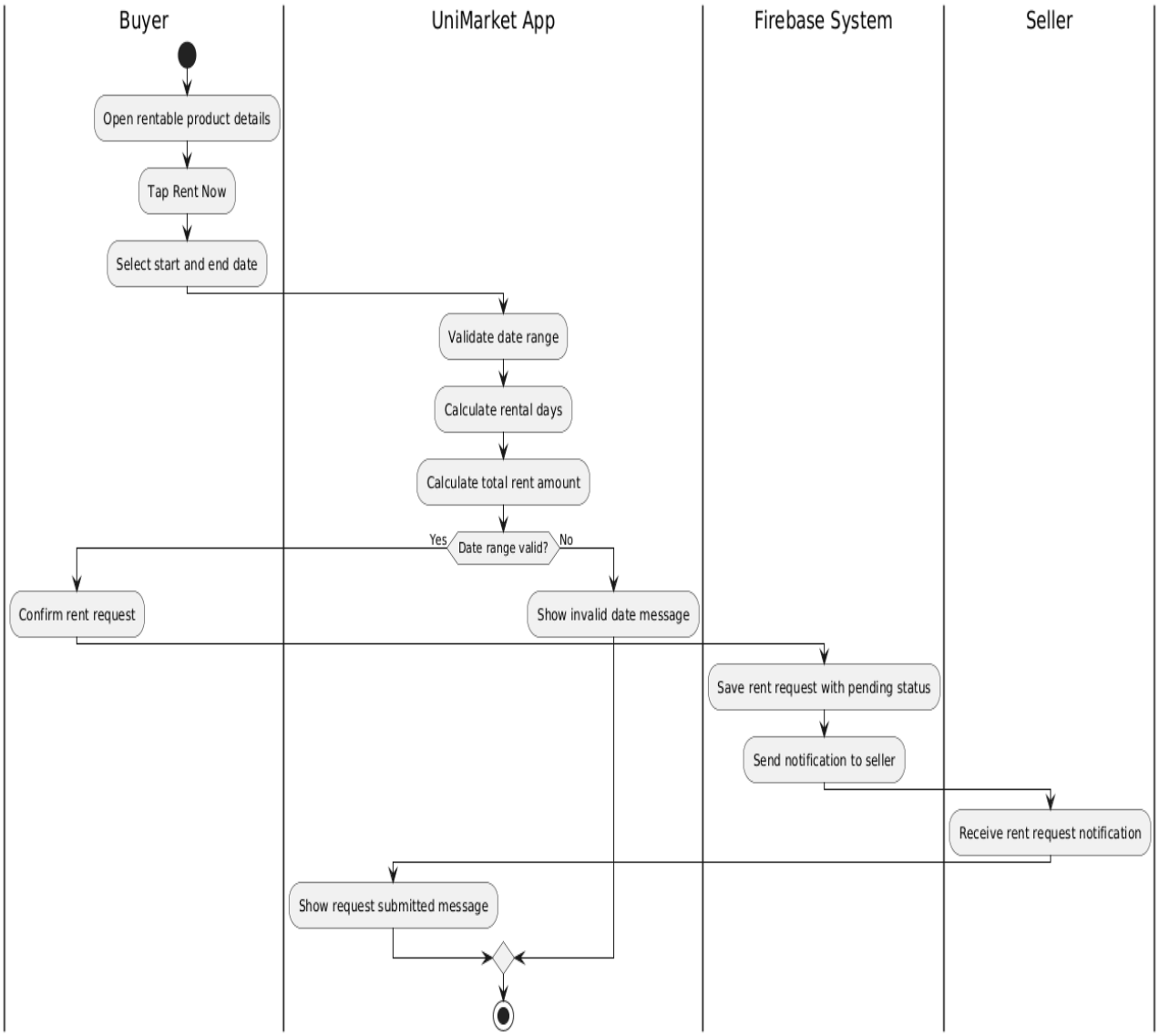


Figure 28:Swimlane Diagram of Request Rent

12.9 Swimlane Diagram of Request Exchange

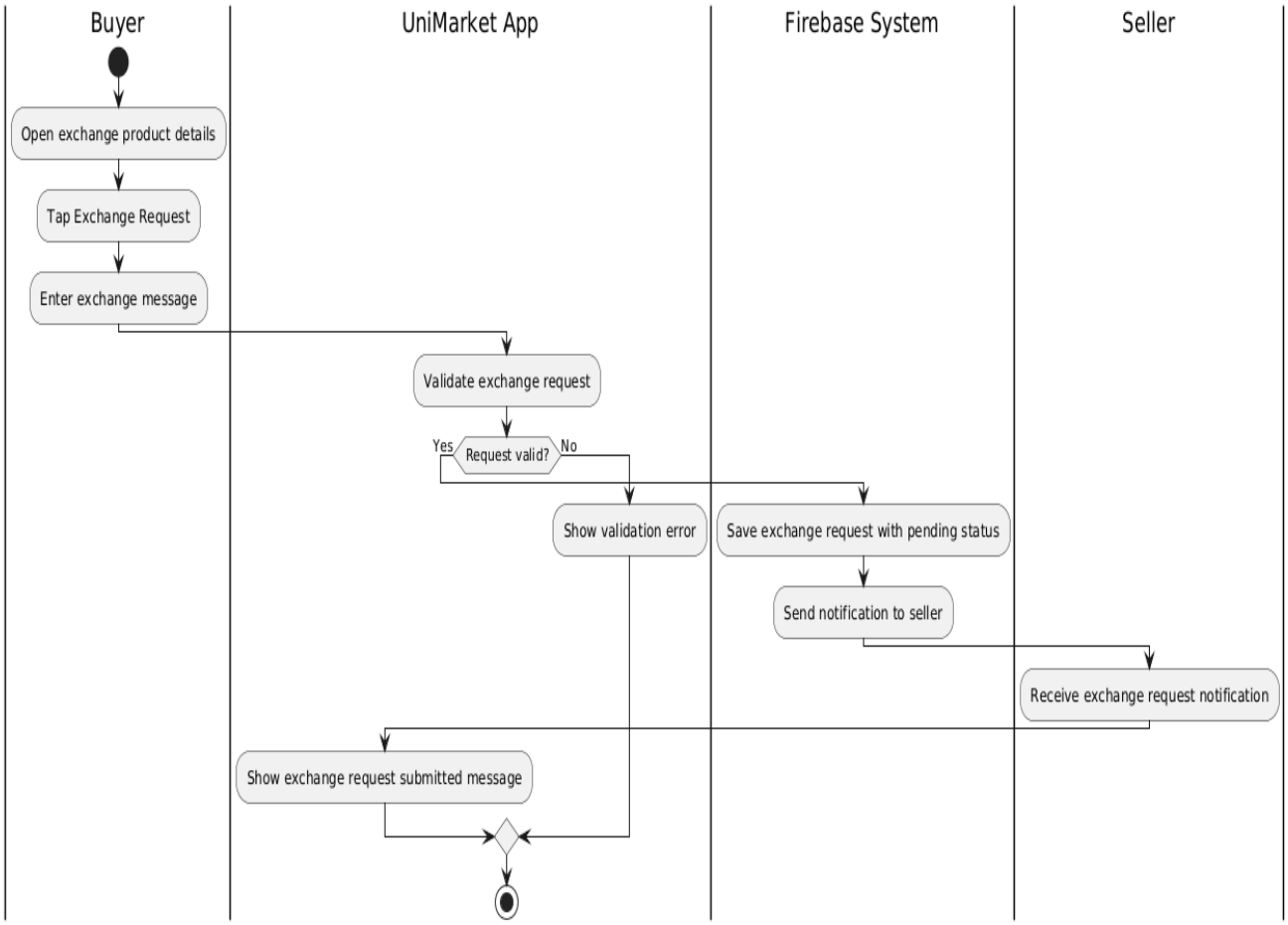


Figure 29:Swimlane Diagram of Request Exchange

12.10 Swimlane Diagram of Manage Seller Dashboard

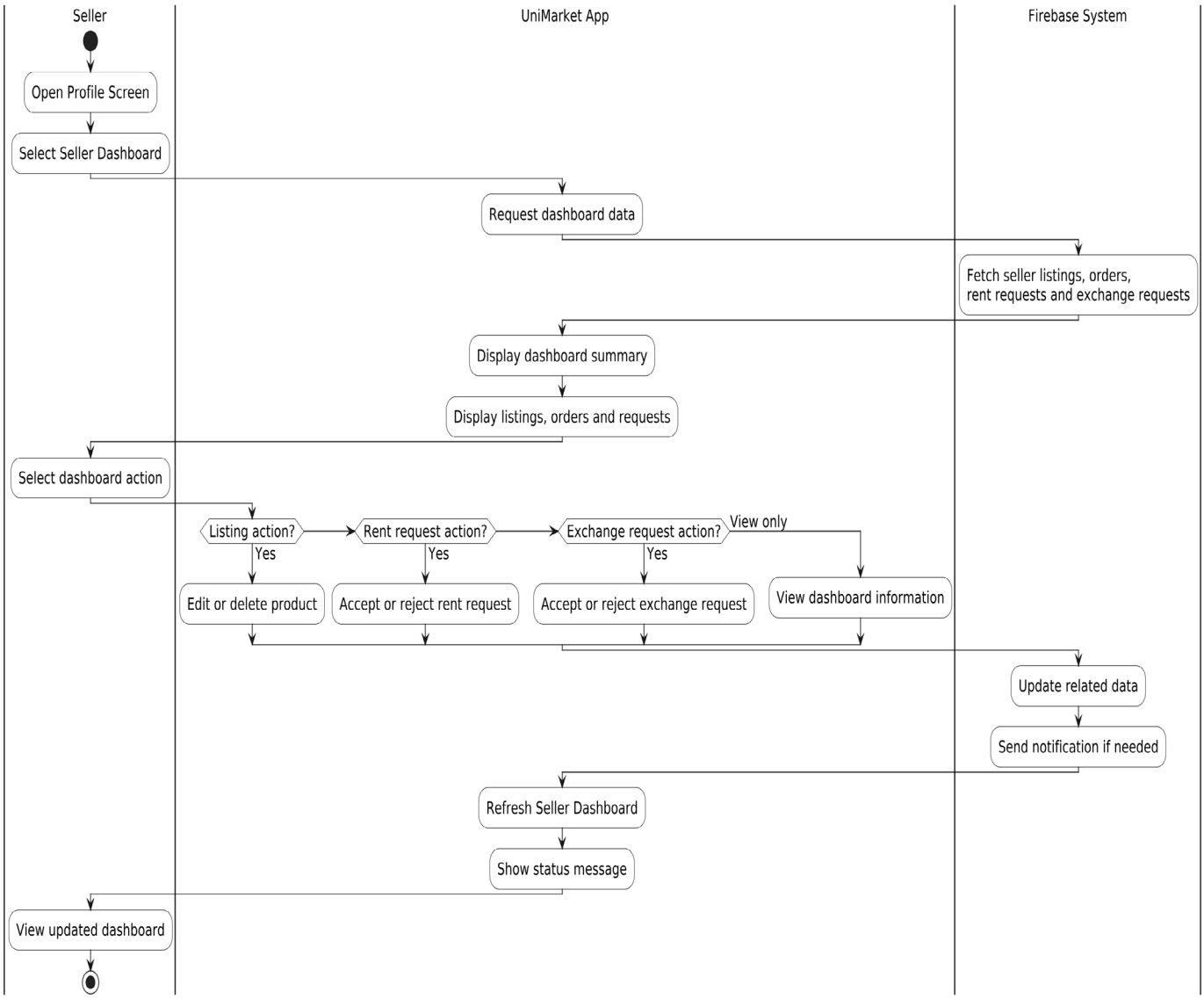


Figure 30:Swimlane Diagram of Manage Seller Dashboard

13. Database Design

13.1 Entity Relationship Diagram

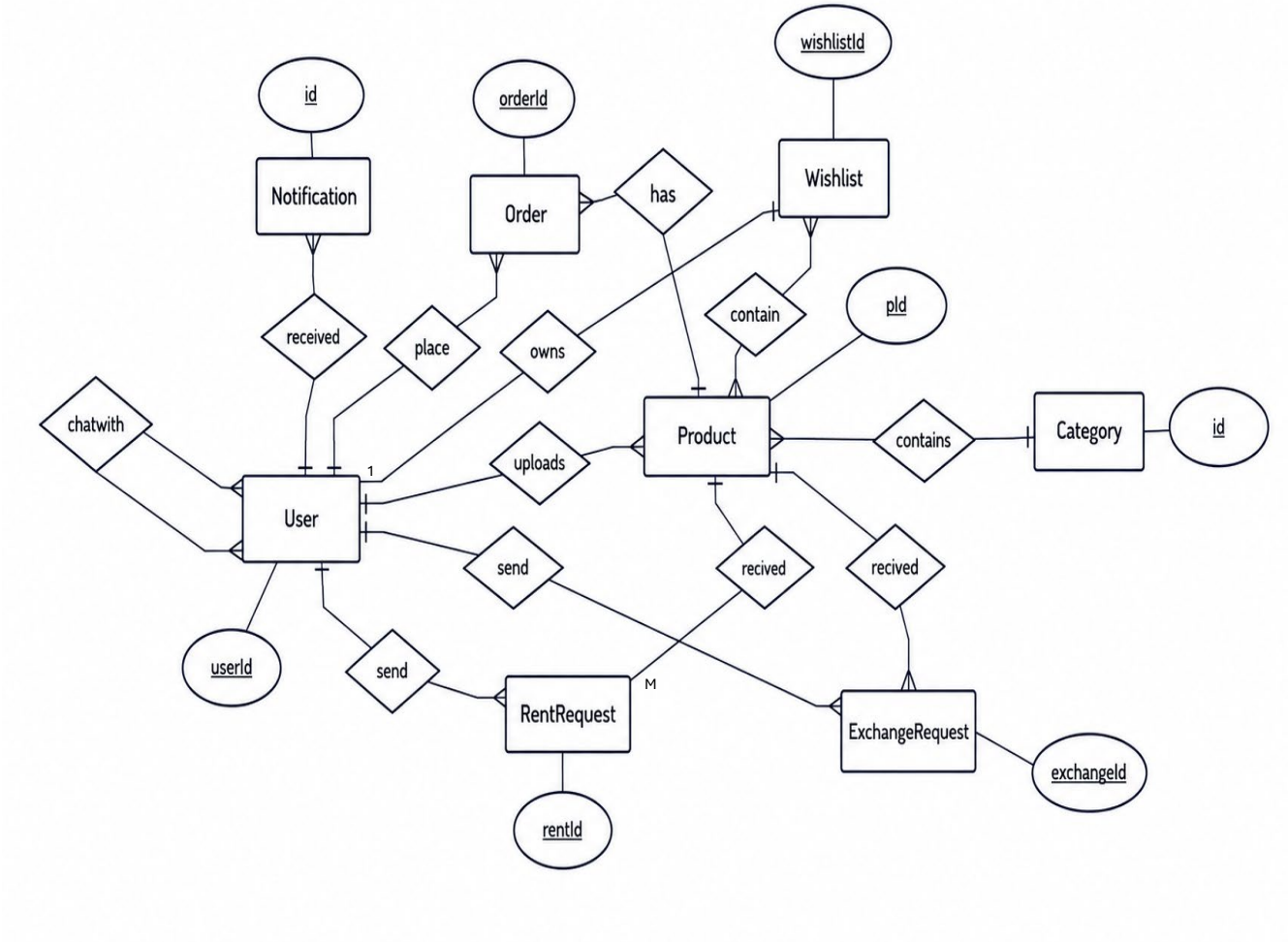


Figure 31:Entity Relationship Diagram of UniMarket

13.2 Relational Schema

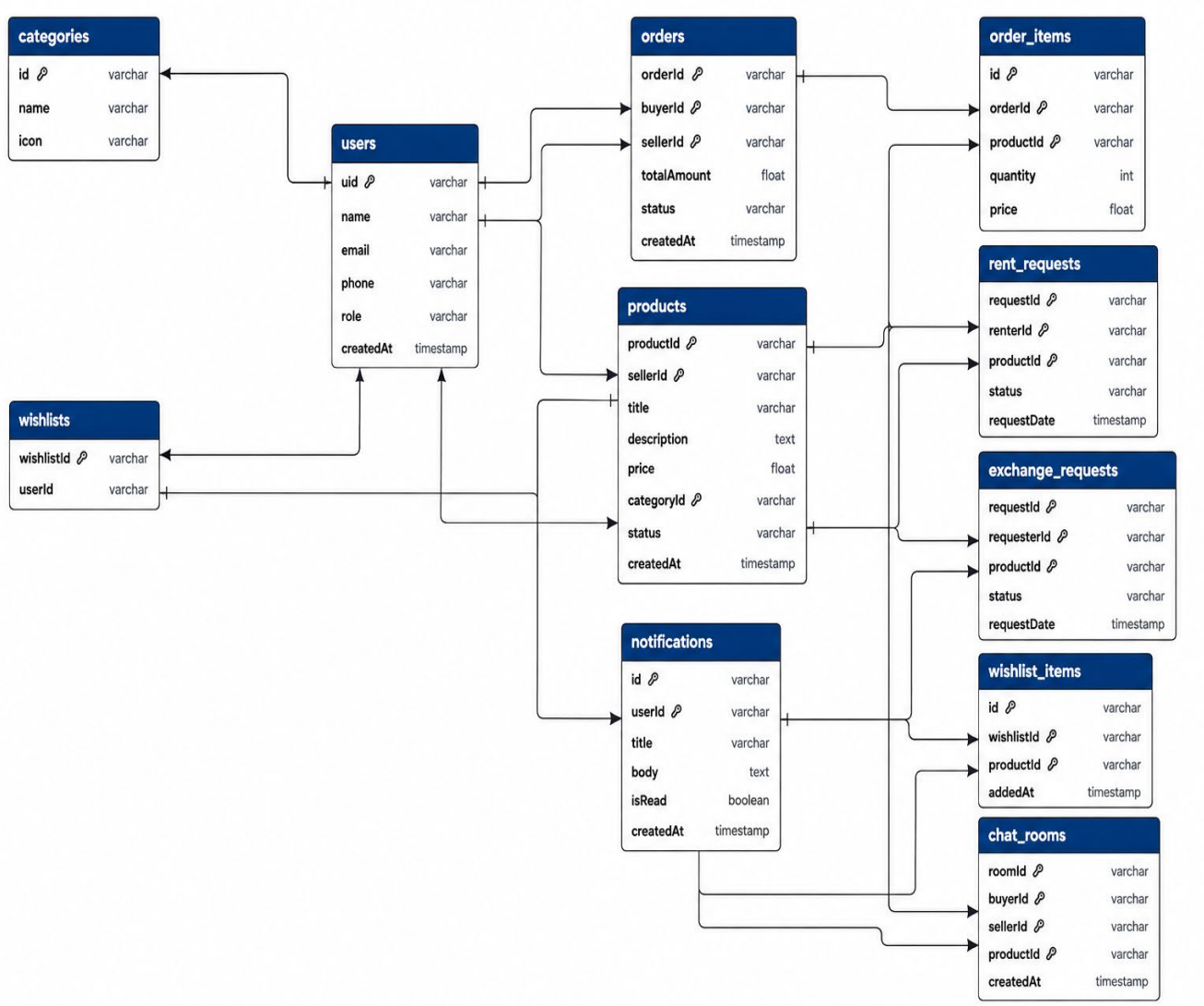


Figure 32:Relational Schema of UniMarket

14.Class Diagram

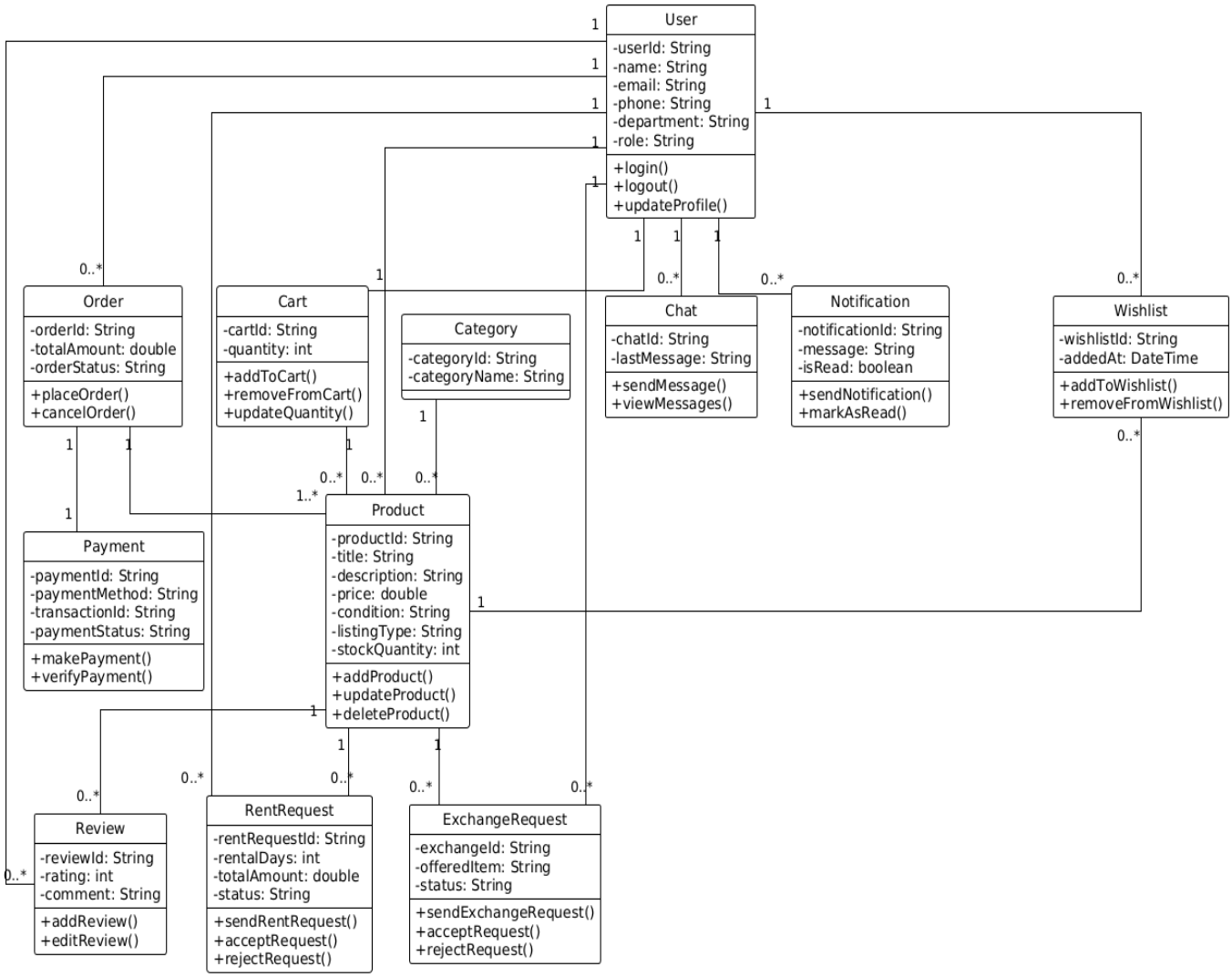


Figure 33:Class Diagram of UniMarket

15. Conclusion

UniMarket is a university-based marketplace application developed to make buying, selling, renting, and exchanging products easier, faster, and more organized for students. This SRS document presents a clear description of the system purpose, project scope, functional requirements, non-functional requirements, system constraints, user interactions, and expected behavior of the UniMarket application. The main objective of the system is to provide a secure and user-friendly digital platform where university students can post products, search and filter items, view product details, communicate with sellers, manage Wishlist and cart items, complete checkout, make payments, send rent or exchange requests, and receive important notifications. The proposed system helps reduce informal and unorganized student-to-student trading by creating a structured campus-based marketplace. Features such as product listing, Wishlist, cart, checkout, manual and online payment, rent request, exchange request, real-time chat, rating, review, stock management, seller dashboard, and notification system make UniMarket useful for both buyers and sellers. This SRS document will work as an important guideline for the design, development, testing, implementation, and future improvement of the UniMarket application.

16. References

- IEEE Std 830-1998: *IEEE Recommended Practice for Software Requirements Specifications*
- ISO/IEC/IEEE 29148:2018 — *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*
- Ian Sommerville, *Software Engineering* (9th Edition), Pearson
- Klaus Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*, Springer
- Abraham Silberschatz, Henry F. Korth, S. Sudarshan, *Database System Concepts* (6th Edition), McGraw-Hill

17.Appendix

17.1 Glossary

This subsection contains definitions of all the terms, acronyms, and abbreviations used in the document. Terms and concepts from the application domain are defined.

- **UniMarket** – A mobile marketplace application designed for university students to buy, sell, rent, and exchange products within a campus-based community.
- **SRS (Software Requirements Specification)** – A document that describes the system's requirements, features, constraints, and expected behavior in detail.
- **Student Marketplace** – A digital platform where students can post, search, buy, sell, rent, or exchange items.
- **User** – Any student or authorized person who uses the UniMarket application.
- **Seller** – A user who posts products for sale, rent, or exchange.
- **Buyer** – A user who searches for products and purchases or requests items from sellers.
- **Product Listing** – A posted item in the UniMarket system containing product name, image, price, description, category, condition, and seller information.
- **Listing Type** – The mode of a product listing, such as Sell Only, Rent Only, Sell & Rent, or Exchange Only.
- **Sell Product** – A product posted by a seller for direct purchase.
- **Rent Product** – A product posted for temporary use based on a daily rental price.
- **Exchange Product** – A product posted for item-to-item exchange without a price field.
- **Wishlist** – A feature that allows users to save favorite products for later viewing.
- **Cart** – A temporary collection of products selected by a buyer before checkout.
- **Checkout** – The process of confirming selected cart items and proceeding toward payment or order placement.
- **Manual Payment** – A payment method where the buyer pays through bKash/Nagad and submits a transaction ID for verification.
- **Transaction ID** – A unique payment reference number provided after a manual payment.
- **Real-Time Chat** – A messaging feature that allows buyers and sellers to communicate instantly.
- **Offer Negotiation** – A conversation between buyer and seller to discuss product price or exchange conditions.
- **Product Details Page** – A screen that displays full product information, image gallery, seller details, and action buttons such as Chat, Add to Cart, Rent Now, or Exchange Request.

- **Rent Request** – A request made by a user to rent a product for a selected duration.
- **Rental Duration** – The number of days for which a product is requested for rent.
- **Daily Rent Price** – The cost of renting a product per day.
- **Exchange Request** – A request sent by one user to another for exchanging items.
- **Wanted Item** – The item that a seller expects in return for an exchange product.
- **Seller Dashboard** – A management screen where sellers can view, edit, delete, and manage their own listings.
- **Admin Panel** – A system module used by administrators to manage users, products, reports, and analytics.
- **Rating** – A 1–5-star score given by a buyer after a transaction.
- **Review** – Text feedback provided by a buyer about a product or seller after a transaction.
- **Stock Management** – A system feature that tracks product quantity and reduces stock after purchase.
- **Auto Expiry** – A system feature that hides or marks listings as expired after a fixed time, such as 30 days.
- **Price Drop Notification** – A notification sent to users when the price of a wishlisted product decreases.
- **Push Notification** – A system-generated alert sent to the user's device.
- **AI Smart Chatbox** – A predefined or AI-assisted quick reply system that helps users send common messages such as "Is it available?" or "Last price?"
- **Authentication** – The process of verifying a user's identity before allowing access to the system.
- **System** – The complete UniMarket software including mobile application, database, cloud services, and backend logic.

17.2 Requirement Elicitation Questionnaire

Questionnaire Link:

https://docs.google.com/forms/d/e/1FAIpQLSeV-IiI1vZbTOfXqztloV4pZu_EG_tHiPpJpKHWUp1DGPbYWg/viewform?usp=header